



# Recursion & Backtracking — Complete Guide

## Placement + Competitive Programming Focused

Covers 16 essential algorithms with intuition, code, dry runs, and interview-ready explanations.

## Table of Contents

1. [Basic Recursion](#)
2. [Recursion Tree](#)
3. [Backtracking Template](#)
4. [Subsets](#)
5. [Permutations](#)
6. [Combinations](#)
7. [N-Queens](#)
8. [Sudoku Solver](#)
9. [Rat in a Maze](#)
10. [Generate Parentheses](#)
11. [Word Search](#)
12. [Palindrome Partitioning](#)
13. [Combination Sum](#)
14. [Hamiltonian Path Basics](#)
15. [Meet-in-the-Middle](#)
16. [Branch and Bound](#)

## 1. Basic Recursion

### 1. Overview

Recursion is a technique where a function calls itself to solve a smaller version of the same problem. The original problem is broken down until it reaches a **base case** — a trivial instance that can be solved directly without further recursion.

### 2. Intuition

#### Simple explanation:

Imagine you're standing in a line at a ticket counter and you want to know your position from the front. You ask the person ahead of you, "What's your position?" They ask the person ahead of them, and so on, until the front person says "I'm 1st." Then each person adds 1 to the answer they received and passes it back. That's recursion — delegate downward, compute upward.

#### Analogy:

Matryoshka (Russian nesting) dolls. To get to the smallest doll, you keep opening outer dolls. The smallest doll is the **base case**. Then you assemble back.

#### Step-by-step reasoning:

1. Identify the **base case** — the simplest input that can be answered directly.
2. Assume the function already works for smaller inputs (the **leap of faith**).
3. Express the answer for the current input in terms of the answer for the smaller input.

**Why it works:**

Each recursive call has its own stack frame with its own local variables. The call stack unwinds naturally when the base case is reached, returning results layer by layer.

3. When to Use It

- Problem has a **recursive structure** (e.g., tree, linked list, divide-and-conquer).
- Problem can be broken into **smaller subproblems** of the same type.
- The input size is small enough that stack depth is not a concern.
- **Trigger phrases:** "compute recursively", "nested structure", "subproblem", "divide and conquer", "tree traversal", "backtracking".

4. When Not to Use It

- **Stack overflow risk:** Deep recursion ( $>10^4$  calls) may crash in C++.
- **Tail recursion not optimized:** C++ does not guarantee tail-call optimization.
- **Better iterative solution exists:** e.g., Fibonacci is better with DP (iterative).
- **Performance-critical code:** Function call overhead matters in tight loops.
- **Overkill for simple loops:** Don't use recursion for a simple `for` loop.

5. Core Concepts

| Concept        | Explanation                                         | Why It Matters                                          |
|----------------|-----------------------------------------------------|---------------------------------------------------------|
| Base Case      | Condition that stops recursion                      | Without it, recursion runs forever (stack overflow)     |
| Recursive Case | The function calling itself with a smaller input    | Breaks the problem into subproblems                     |
| Call Stack     | The stack of function calls waiting to return       | Determines memory usage; deep recursion can overflow    |
| Stack Frame    | Memory allocated per call (local vars, return addr) | Each frame costs $O(1)$ space                           |
| Leap of Faith  | Assume the recursive call works correctly           | Lets you focus on combining results, not tracing        |
| Tail Recursion | Recursive call is the last operation                | Can be optimized to iterative (but not in C++ reliably) |

6. Step-by-Step Algorithm (Factorial Example)

```
Input: n = 5

Step 1: Check base case: n == 0? No.
```

```
Step 2: Return  $n * \text{factorial}(n-1) = 5 * \text{factorial}(4)$ 
Step 3: Check base case:  $n == 0$ ? No.
Step 4: Return  $4 * \text{factorial}(3)$ 
Step 5: ... continue until  $n == 0$  ...
Step 6:  $\text{factorial}(0)$  returns 1
Step 7:  $\text{factorial}(1) = 1 * 1 = 1$ 
Step 8:  $\text{factorial}(2) = 2 * 1 = 2$ 
Step 9:  $\text{factorial}(3) = 3 * 2 = 6$ 
Step 10:  $\text{factorial}(4) = 4 * 6 = 24$ 
Step 11:  $\text{factorial}(5) = 5 * 24 = 120$ 
```

## 7. Dry Run

**Input:** `factorial(4)`

| Call                                | n | Base Case? | Returns                        | Computed Value |
|-------------------------------------|---|------------|--------------------------------|----------------|
| <code>factorial(4)</code>           | 4 | No         | $4 \times \text{factorial}(3)$ | —              |
| <code>factorial(3)</code>           | 3 | No         | $3 \times \text{factorial}(2)$ | —              |
| <code>factorial(2)</code>           | 2 | No         | $2 \times \text{factorial}(1)$ | —              |
| <code>factorial(1)</code>           | 1 | No         | $1 \times \text{factorial}(0)$ | —              |
| <code>factorial(0)</code>           | 0 | <b>Yes</b> | 1                              | 1              |
| ← back to <code>factorial(1)</code> | 1 | —          | $1 \times 1$                   | <b>1</b>       |
| ← back to <code>factorial(2)</code> | 2 | —          | $2 \times 1$                   | <b>2</b>       |
| ← back to <code>factorial(3)</code> | 3 | —          | $3 \times 2$                   | <b>6</b>       |
| ← back to <code>factorial(4)</code> | 4 | —          | $4 \times 6$                   | <b>24</b>      |

**Final answer:** 24

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Basic factorial
long long factorial(int n) {
    // Base case
    if (n <= 1) return 1;
    // Recursive case
    return n * factorial(n - 1);
}

// Fibonacci
int fib(int n) {
    if (n <= 1) return n;
```

```

        return fib(n - 1) + fib(n - 2);
    }

    // Print 1 to N using recursion
    void print1toN(int n) {
        if (n == 0) return;
        print1toN(n - 1);
        cout << n << " ";
    }

    // Sum of array using recursion
    int sumArray(vector<int>& arr, int idx) {
        if (idx == arr.size()) return 0;
        return arr[idx] + sumArray(arr, idx + 1);
    }

    int main() {
        cout << factorial(5) << "\n";           // 120
        cout << fib(7) << "\n";                 // 13
        print1toN(5);                           // 1 2 3 4 5
        cout << "\n";
        vector<int> arr = {1, 2, 3, 4, 5};
        cout << sumArray(arr, 0) << "\n";      // 15
        return 0;
    }

```

## 9. Python Implementation

```

def factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)

def fib(n: int) -> int:
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)

def print_1_to_n(n: int) -> None:
    if n == 0:
        return
    print_1_to_n(n - 1)
    print(n, end=" ")

def sum_array(arr: list, idx: int = 0) -> int:
    if idx == len(arr):
        return 0
    return arr[idx] + sum_array(arr, idx + 1)

# Example usage
print(factorial(5))      # 120

```

```
print(fib(7))           # 13
print_1_to_n(5)         # 1 2 3 4 5
print()
print(sum_array([1, 2, 3, 4, 5])) # 15
```

## 10. Code Explanation

### Factorial:

- `factorial(n)` returns  $n! = n \times (n-1) \times \dots \times 1$
- Base case: `n <= 1` returns 1 ( $0! = 1$ ,  $1! = 1$ )
- Recursive case: `n * factorial(n-1)` — trust that `factorial(n-1)` gives  $(n-1)!$

### Fibonacci:

- `fib(n)` returns the  $n$ th Fibonacci number (0, 1, 1, 2, 3, 5, 8, ...)
- Base case: `n <= 1` returns `n`
- Recursive case: `fib(n-1) + fib(n-2)` — this is exponential; use DP for efficiency

### Print 1 to N:

- The print happens **after** the recursive call — this is why it prints 1, 2, 3, ... N
- If you print before the recursive call, it would print N, N-1, ..., 1

### Sum Array:

- `idx` tracks current position; base case hits when `idx == size`
- Each call adds `arr[idx]` to the sum of the rest

## 11. Complexity Analysis

| Operation         | Time Complexity | Space Complexity (Stack) |
|-------------------|-----------------|--------------------------|
| Factorial         | $O(n)$          | $O(n)$                   |
| Fibonacci (naive) | $O(2^n)$        | $O(n)$                   |
| Print 1 to N      | $O(n)$          | $O(n)$                   |
| Sum Array         | $O(n)$          | $O(n)$                   |

## 12. Common Patterns

| Pattern               | How to Identify                   | Approach                             | Example                   |
|-----------------------|-----------------------------------|--------------------------------------|---------------------------|
| <b>Tail Recursion</b> | Recursive call is last statement  | Can be converted to loop             | Factorial                 |
| <b>Head Recursion</b> | Recursive call before computation | Cannot be tail-optimized             | Print 1 to N              |
| <b>Tree Recursion</b> | Two+ recursive calls              | Creates branching; often exponential | Fibonacci, Tower of Hanoi |

| Pattern                   | How to Identify                                         | Approach                            | Example            |
|---------------------------|---------------------------------------------------------|-------------------------------------|--------------------|
| <b>Indirect Recursion</b> | A calls B, B calls A                                    | Harder to trace; mutually recursive | Syntax parsers     |
| <b>Nested Recursion</b>   | Recursive call is an argument to another recursive call | Rare, complex                       | Ackermann function |

### 13. Common Mistakes

- **Missing base case** → infinite recursion → stack overflow.
- **Base case never reached** → e.g.,  $n > 0$  as base but input is negative.
- **Wrong return type** → integer overflow for large factorial.
- **Recomputing same subproblems** → exponential blowup (Fibonacci without memoization).
- **Modifying shared state** → passing a reference to a container and forgetting to undo.

### 14. Edge Cases

- $n = 0$  (factorial(0) = 1)
- $n = 1$  (base case)
- Very large  $n$  (stack overflow beyond  $\sim 10^5$ )
- Negative input (should be handled or disallowed)

### 15. Variations

| Variation                   | What Changes                  | When Used              | Importance                 |
|-----------------------------|-------------------------------|------------------------|----------------------------|
| <b>With Memoization</b>     | Cache results of subproblems  | DP problems            | Very high                  |
| <b>Tail Recursive</b>       | Accumulator parameter         | Optimization           | Low (C++ doesn't optimize) |
| <b>Mutual Recursion</b>     | Two functions call each other | Parsing, game theory   | Medium                     |
| <b>Continuation-passing</b> | Pass a callback function      | Functional programming | Low                        |

### 16. Related Algorithms

| Algorithm                   | Connection                    | How to Choose                                           |
|-----------------------------|-------------------------------|---------------------------------------------------------|
| <b>Iteration</b>            | Same task, different style    | Use iteration if depth $> 10^4$ or performance critical |
| <b>Dynamic Programming</b>  | Recursion + memoization       | Use DP when overlapping subproblems exist               |
| <b>Divide &amp; Conquer</b> | Recursive division of problem | Use when subproblems are independent                    |

| Algorithm           | Connection                        | How to Choose                        |
|---------------------|-----------------------------------|--------------------------------------|
| <b>Backtracking</b> | Recursion with exploration + undo | Use when exploring all possibilities |

## 17. Practice Problems

### Easy

- **Factorial Trailing Zeroes** — LeetCode — Compute factorial, count trailing zeros (math)
- **Fibonacci Number** — LeetCode — Basic recursion, then DP

### Medium

- **Tower of Hanoi** — GFG — Classic recursion with 3 poles
- **Pow(x, n)** — LeetCode — Recursive binary exponentiation
- **K-th Symbol in Grammar** — LeetCode — Recursive structure in grammar

### Hard

- **Special Binary String** — LeetCode — Complex recursive transformation
- **Basic Calculator** — LeetCode — Recursive descent parsing

## 18. Interview Explanation

"Recursion is a technique where a function calls itself on a smaller subproblem. Every recursive function has two parts: a **base case** that terminates the recursion, and a **recursive case** that reduces the problem. I use the leap of faith — I assume the recursive call works correctly for smaller inputs, and focus on combining the result. The main tradeoff is that recursion uses  $O(\text{depth})$  stack space, so for very deep recursion I'd switch to an iterative approach or use memoization to avoid exponential blowup."

## 19. Revision Notes

- Every recursive function = **Base case** + **Recursive case**
- **Leap of faith:** Trust the recursion, don't trace all calls
- **Stack space:**  $O(\text{depth})$  — risk of overflow for  $\text{depth} > 10^5$
- **Naive Fibonacci** is  $O(2^n)$  — always use DP/memoization
- **Print before call** = decreasing order; **print after call** = increasing order
- **Common trap:** Forgetting to return the base case value

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                    |
|------------------------|----------------------------------------------------------------------------|
| <b>When to use</b>     | Problem has recursive structure, subproblems of same type                  |
| <b>Main operations</b> | Base case check, recursive call(s), combine results                        |
| <b>Complexity</b>      | $O(n)$ time, $O(n)$ stack space for linear; $O(2^n)$ for branching         |
| <b>Key code</b>        | <code>if (base) return baseVal; return combine(recursive(smaller));</code> |

| Aspect     | Summary                                                 |
|------------|---------------------------------------------------------|
| Edge cases | $n=0$ , $n=1$ , negative input, deep recursion overflow |

## 2. Recursion Tree

### 1. Overview

A recursion tree is a visual representation of all recursive calls made during the execution of a recursive function. Each node represents a function call, and its children represent the sub-calls it makes. It helps analyze the flow, branching factor, depth, and complexity of recursive algorithms.

### 2. Intuition

#### Simple explanation:

Think of a family tree. You are the root. Your children are the subproblems you create. Each child has their own children, and so on, until you reach the leaves (base cases). The recursion tree lets you see every call, how many times the function is called, and how the results flow back up.

#### Analogy:

A tournament bracket. Each match (node) feeds into the next round. The final winner is the root's result. Each match represents a recursive call combining two sub-results.

#### Step-by-step reasoning:

1. The root is the initial call with the original input.
2. Each node branches into its recursive calls (children).
3. Leaf nodes are base cases that return directly.
4. Values propagate upward from leaves to root.

#### Why it matters:

Recursion trees help you:

- Understand the **branching factor** (how many recursive calls per node)
- Compute **time complexity** by summing work at each level
- Identify **overlapping subproblems** (nodes with same input at different branches)
- Debug recursive logic by tracing paths

### 3. When to Use It

- **Analyzing** recursive algorithm complexity.
- **Teaching** or **debugging** recursion.
- **Identifying** overlapping subproblems for DP.
- **Visualizing** backtracking search space.
- **Trigger phrases:** "trace the recursion", "draw the recursion tree", "analyze time complexity of recursion".

### 4. When Not to Use It

- **Simple linear recursion:** Factorial's recursion tree is just a chain — not useful.



- **Already familiar with complexity:** If you already know the recurrence, a tree may be overkill.
- **Very deep recursion:** Drawing a tree for depth 1000 is impractical.
- **Memoized recursion:** DP recursion trees are not useful because overlapping subproblems are cached.

5. Core Concepts

| Concept                 | Explanation                              | Why It Matters                              |
|-------------------------|------------------------------------------|---------------------------------------------|
| Root                    | The initial function call                | Represents the original problem             |
| Node                    | A single function call                   | Contains the input size/state               |
| Leaf                    | A base case call (no children)           | Terminates recursion                        |
| Branching Factor        | Number of children per node              | Determines exponential growth               |
| Depth                   | Length of longest path from root to leaf | Determines stack space usage                |
| Level                   | All nodes at the same depth from root    | Work at each level helps compute complexity |
| Overlapping Subproblems | Same node appears in different branches  | Indicates DP opportunity                    |

6. Step-by-Step Algorithm (Building a Recursion Tree)

Input: Recursive function  $f(n)$  that calls  $f(n-1)$  and  $f(n-2)$  (Fibonacci)

Step 1: Draw root node with input  $n = 5$ .

Step 2: From root, draw two children:  $f(4)$  and  $f(3)$ .

Step 3: From  $f(4)$ , draw  $f(3)$  and  $f(2)$ .

Step 4: From  $f(3)$ , draw  $f(2)$  and  $f(1)$ .

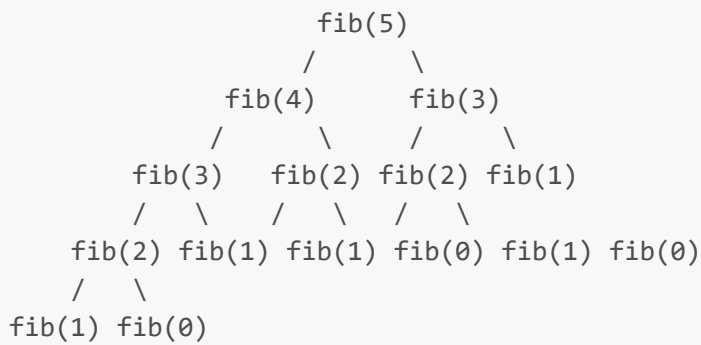
Step 5: Continue until  $f(1)$  and  $f(0)$  are reached (base cases).

Step 6: Leaf nodes return their values.

Step 7: Values propagate upward: each node sums its children.

7. Dry Run

**Function:** `fib(5)` — Fibonacci recursion tree



### Counting calls:

| Level | Nodes | Description                  |
|-------|-------|------------------------------|
| 0     | 1     | fib(5)                       |
| 1     | 2     | fib(4), fib(3)               |
| 2     | 4     | fib(3)×2, fib(2)×2           |
| 3     | 8     | fib(2)×3, fib(1)×3, fib(0)×2 |
| 4     | 5     | fib(1)×3, fib(0)×2           |

**Total calls:**  $1 + 2 + 4 + 8 + 5 = 15$  (actually 15 calls for fib(5))

**Overlapping subproblems:** fib(3) appears twice, fib(2) appears 3 times, fib(1) appears 5 times, fib(0) appears 3 times. This is why memoization helps.

## 8. C++ Implementation (Visualizing with Depth Tracking)

```

#include <bits/stdc++.h>
using namespace std;

int fib(int n, string indent = "") {
    cout << indent << "fib(" << n << ")\n";
    if (n <= 1) {
        cout << indent << "=> " << n << " (base)\n";
        return n;
    }
    int left = fib(n - 1, indent + " ");
    int right = fib(n - 2, indent + " ");
    int result = left + right;
    cout << indent << "=> " << result << "\n";
    return result;
}

int main() {
    fib(5);
}

```

```
    return 0;
}
```

## 9. Python Implementation

```
def fib(n: int, indent: str = ""):
    print(f"{indent}fib({n})")
    if n <= 1:
        print(f"{indent}=> {n} (base)")
        return n
    left = fib(n - 1, indent + " ")
    right = fib(n - 2, indent + " ")
    result = left + right
    print(f"{indent}=> {result}")
    return result

fib(5)
```

## 10. Code Explanation

- The `indent` parameter tracks the depth of the call, creating a visual tree.
- Each call prints its input before recursing.
- Base cases print their return value.
- After both children return, the parent prints its computed result.
- The output directly mirrors the recursion tree structure.

## 11. Complexity Analysis from Recursion Tree

| Aspect           | Value             |
|------------------|-------------------|
| Branching factor | 2 (for Fibonacci) |
| Depth            | n                 |
| Number of nodes  | $O(2^n)$          |
| Work per node    | $O(1)$            |
| Total time       | $O(2^n)$          |
| Stack space      | $O(n)$            |

**General formula:** For a recursion tree with branching factor `b` and depth `d`, worst-case nodes =  $O(b^d)$ .

## 12. Common Patterns

| Pattern      | Description           | Example                 |
|--------------|-----------------------|-------------------------|
| Linear Chain | Single recursive call | Factorial, Sum of array |

| Pattern             | Description           | Example                         |
|---------------------|-----------------------|---------------------------------|
| <b>Binary Tree</b>  | Two recursive calls   | Fibonacci, Merge Sort           |
| <b>Ternary Tree</b> | Three recursive calls | Some divide-and-conquer         |
| <b>N-ary Tree</b>   | N recursive calls     | Subset generation, Backtracking |
| <b>Pruned Tree</b>  | Some branches are cut | Backtracking with constraints   |

### 13. Common Mistakes

- **Confusing depth with number of nodes:** Depth =  $O(n)$ , nodes =  $O(b^n)$ .
- **Ignoring overlapping subproblems:** The recursion tree might have many shared nodes — DP can reduce complexity.
- **Assuming all leaves are at same depth:** Some branches may terminate earlier.
- **Not counting work per node:** If each node does  $O(n)$  work, total is different from  $O(1)$  per node.

### 14. Edge Cases

- **$n = 0$ :** Only one node (root is leaf).
- **$n = 1$ :** Root is leaf.
- **No branching:** Linear chain — recursion tree is just a path.
- **All leaves at different depths:** E.g., Quicksort worst-case vs best-case.

### 15. Variations

| Variation              | What Changes                      | When Used                | Importance |
|------------------------|-----------------------------------|--------------------------|------------|
| <b>Recursion Stack</b> | Shows only active calls           | Debugging stack overflow | Medium     |
| <b>Call Graph</b>      | Shows all calls (not just active) | Static analysis          | Low        |
| <b>DAG</b>             | Recursion tree with merged nodes  | DP optimization          | High       |

### 16. Related Concepts

| Concept                     | Connection                                          |
|-----------------------------|-----------------------------------------------------|
| <b>Recurrence Relations</b> | Math behind recursion tree; $T(n) = aT(n/b) + f(n)$ |
| <b>Master Theorem</b>       | Solves recurrence for divide-and-conquer trees      |
| <b>Memoization</b>          | Overlapping subproblems identified from tree        |
| <b>Branch and Bound</b>     | Pruning branches of the recursion tree              |

### 17. Practice Problems

#### Easy

- **Fibonacci Number** — LeetCode — Draw the recursion tree manually
- **Climbing Stairs** — LeetCode — Similar tree to Fibonacci

## Medium

- **Generate Parentheses** — LeetCode — Recursion tree with pruning
- **Subsets** — LeetCode — Binary decision tree

## Hard

- **N-Queens** — LeetCode — Complex branching with pruning
- **Word Search II** — LeetCode — Trie + recursion tree

## 18. Interview Explanation

"A recursion tree is a visual tool where each node represents a function call. The root is the initial call, children are the sub-calls, and leaves are base cases. I use it to analyze time complexity — the branching factor and depth tell me if the algorithm is  $O(n)$ ,  $O(2^n)$ , or  $O(n!)$ . It also helps me identify overlapping subproblems, which tells me whether I can optimize with dynamic programming."

## 19. Revision Notes

- **Root = initial call, leaves = base cases**
- **Branching factor**  $\times$  **depth** determines complexity
- **Overlapping subproblems**  $\rightarrow$  DP opportunity
- **Linear chain**  $\rightarrow O(n)$  depth,  $O(n)$  nodes
- **Binary tree**  $\rightarrow O(2^n)$  nodes for Fibonacci
- **Pruned tree**  $\rightarrow$  backtracking (much fewer nodes in practice)

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                              |
|------------------------|--------------------------------------------------------------------------------------|
| <b>When to use</b>     | Analyzing recursive complexity, visualizing backtracking                             |
| <b>Main operations</b> | Draw root, branch to children, label leaves                                          |
| <b>Complexity</b>      | Nodes = $O(b^{\text{root}})$ where $b$ = branching factor, stack = $O(\text{depth})$ |
| <b>Key insight</b>     | Overlapping subproblems $\rightarrow$ DP; pruning $\rightarrow$ backtracking         |
| <b>Edge cases</b>      | $n=0$ , $n=1$ , no branching, varying leaf depths                                    |

## 3. Backtracking Template

### 1. Overview

Backtracking is a systematic way to explore all possible solutions to a problem by incrementally building candidates and abandoning ("backtracking" from) a candidate as soon as it's determined to be invalid. It's essentially a depth-first search of the **state-space tree** with pruning.

### 2. Intuition

#### Simple explanation:

Imagine you're trying to solve a maze. You walk forward, and at each intersection you choose a path. If you hit

a dead end, you go back to the last intersection and try a different path. This "try → if fail → undo and try another" is backtracking.

**Analogy:**

A chess player thinking ahead: "If I move my knight here, then my opponent can move there, then I can move here..." They explore branches mentally and discard losing lines.

**Step-by-step reasoning:**

- 1. **Choose:** Make a decision (add an element, place a queen, etc.).
- 2. **Explore:** Recursively solve the remaining problem.
- 3. **Un-choose:** Undo the decision (backtrack) to try another option.

**Why it works:**

Backtracking systematically explores the entire search space without repeating work. The pruning (abandoning invalid branches early) is what makes it practical — it distinguishes backtracking from brute-force enumeration.

3. When to Use It

- **All possible solutions** are required (subsets, permutations, combinations).
- **Constraint satisfaction** problems (N-Queens, Sudoku, graph coloring).
- **Combinatorial optimization** (Knapsack, TSP with small input).
- **Trigger phrases:** "generate all", "find all possible", "print all combinations", "place such that", "arrange without conflict", "backtrack".

4. When Not to Use It

- **Counting only** → use combinatorics formulas ( $nCr$ ,  $nPr$ ).
- **Input size is large** ( $> 20$ - $30$  for combinations,  $> 10$ - $12$  for full permutations).
- **One optimal solution needed** → use greedy or DP instead.
- **Problem has optimal substructure** → DP is better.
- **Overlapping subproblems exist** → DP with memoization.

5. Core Concepts

| Concept          | Explanation                             | Why It Matters                 |
|------------------|-----------------------------------------|--------------------------------|
| State Space Tree | Tree of all possible states             | Defines the search space       |
| Pruning          | Cutting off invalid branches early      | Makes backtracking practical   |
| Choice Point     | A decision to make (branch)             | Determines branching factor    |
| Constraint       | Rule that must be satisfied             | Used to prune branches         |
| Goal             | Condition for a valid complete solution | Terminates successful branches |
| Undo (Backtrack) | Revert the last decision                | Allows exploring other options |
| Partial Solution | Incomplete solution being built         | Current state of exploration   |

6. Step-by-Step Algorithm (Generic Template)

Input: Problem parameters, current state, partial solution

Step 1: Check if the current state is a valid complete solution (goal).  
If yes, add to results and return.

Step 2: Iterate over all possible choices at this state.

Step 3: For each choice:

- a. Check if the choice is valid (constraint check).  
If not valid, skip (prune).
- b. Make the choice (update state, add to partial solution).
- c. Recursively call for the next state.
- d. Undo the choice (backtrack).

Step 4: Return (all choices explored).

## 7. Dry Run

**Problem:** Generate all subsets of [1, 2] (Subsets problem)

Initial: idx=0, current=[]

At idx=0:

Choice 1: Exclude 1 → current=[], recurse idx=1

At idx=1:

Choice 1: Exclude 2 → current=[], idx=2 (base) → add []

Choice 2: Include 2 → current=[2], idx=2 (base) → add [2]

Backtrack: undo 2 → current=[]

Choice 2: Include 1 → current=[1], recurse idx=1

At idx=1:

Choice 1: Exclude 2 → current=[1], idx=2 (base) → add [1]

Choice 2: Include 2 → current=[1,2], idx=2 (base) → add [1,2]

Backtrack: undo 2 → current=[1]

Backtrack: undo 1 → current=[]

Result: [], [2], [1], [1,2]

## 8. C++ Implementation (Generic Template)

```
#include <bits/stdc++.h>
using namespace std;

// Generic backtracking template
class BacktrackingSolver {
    vector<vector<int>> result;
    vector<int> current;

    void backtrack(vector<int>& candidates, int start, int target) {
```

```

        // Goal check
        if (target == 0) {
            result.push_back(current);
            return;
        }

        // Iterate over choices
        for (int i = start; i < candidates.size(); i++) {
            // Prune invalid branches
            if (candidates[i] > target) continue;

            // Make choice
            current.push_back(candidates[i]);

            // Recurse
            backtrack(candidates, i, target - candidates[i]);

            // Undo choice (backtrack)
            current.pop_back();
        }
    }

public:
    vector<vector<int>> solve(vector<int>& candidates, int target) {
        result.clear();
        current.clear();
        sort(candidates.begin(), candidates.end());
        backtrack(candidates, 0, target);
        return result;
    }
};

// Example usage
int main() {
    BacktrackingSolver solver;
    vector<int> candidates = {2, 3, 6, 7};
    auto res = solver.solve(candidates, 7);

    for (auto& comb : res) {
        cout << "[";
        for (int x : comb) cout << x << " ";
        cout << "]\n";
    }
    // Output: [2 2 3 ] [7 ]
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

```



```

class BacktrackingSolver:
    def __init__(self):
        self.result = []
        self.current = []

    def backtrack(self, candidates: List[int], start: int, target: int):
        # Goal check
        if target == 0:
            self.result.append(self.current[:])
            return

        # Iterate over choices
        for i in range(start, len(candidates)):
            # Prune invalid branches
            if candidates[i] > target:
                continue

            # Make choice
            self.current.append(candidates[i])

            # Recurse (allow reuse of same element)
            self.backtrack(candidates, i, target - candidates[i])

            # Undo choice (backtrack)
            self.current.pop()

    def solve(self, candidates: List[int], target: int) -> List[List[int]]:
        self.result = []
        self.current = []
        candidates.sort()
        self.backtrack(candidates, 0, target)
        return self.result

# Example usage
solver = BacktrackingSolver()
print(solver.solve([2, 3, 6, 7], 7)) # [[2, 2, 3], [7]]

```

## 10. Code Explanation

- **result**: Stores all valid complete solutions.
- **current**: Tracks the partial solution being built.
- **backtrack()**: The recursive function that explores the state space.
- **Goal check**: If `target == 0`, we've found a valid combination.
- **Pruning**: if `(candidates[i] > target)` `continue`; — skip impossible choices.
- **Make choice**: `current.push_back(candidates[i])` — add to partial solution.
- **Recurse**: `backtrack(candidates, i, target - candidates[i])` — explore with reduced target.
- **Undo**: `current.pop_back()` — remove the choice to try another.

## 11. Complexity Analysis

| Aspect                 | Complexity                                                      |
|------------------------|-----------------------------------------------------------------|
| Time (worst-case)      | $O(\text{branches}^{\text{depth}})$ — often $O(2^n)$ or $O(n!)$ |
| Space (stack)          | $O(\text{depth})$ — typically $O(n)$                            |
| Space (result storage) | $O(\text{number of solutions} \times \text{solution size})$     |
| Pruning effect         | Can reduce time drastically in practice                         |

## 12. Common Patterns

| Pattern                 | Description                           | Example                    |
|-------------------------|---------------------------------------|----------------------------|
| Decision at each level  | Include/exclude each element          | Subsets                    |
| Permutation             | Choose an unused element at each step | Permutations               |
| Combination             | Choose k elements in order            | Combinations               |
| Constraint Satisfaction | Place items with constraints          | N-Queens, Sudoku           |
| Path Finding            | Explore paths in a grid               | Rat in a Maze, Word Search |

## 13. Common Mistakes

- **Forgetting to undo** → state corruption across branches.
- **Not cloning the result** → storing reference to **current** which gets modified.
- **Wrong pruning condition** → missing valid solutions or including invalid ones.
- **Infinite recursion** → no progress toward base case.
- **Modifying shared global state** → affects other branches.
- **Using return after pruning** → may skip other valid choices in the loop.

## 14. Edge Cases

- **Empty input** → should return `[]` for subsets (empty set is valid).
- **No valid solution** → return empty list.
- **All elements identical** → may produce duplicates unless handled.
- **Large input with heavy pruning** → runtime varies drastically.
- **Target = 0** → empty combination is valid.

## 15. Variations

| Variation              | What Changes                            | When Used             | Importance |
|------------------------|-----------------------------------------|-----------------------|------------|
| With Duplicates        | Skip same elements at same level        | Input has duplicates  | High       |
| With Repetition        | Allow reusing same element              | Combination Sum       | High       |
| With Pruning (B&B)     | Bound function to cut branches          | Optimization problems | High       |
| Iterative Backtracking | Use explicit stack instead of recursion | Avoid stack overflow  | Medium     |

## 16. Related Algorithms

| Algorithm                 | Connection                           | How to Choose                         |
|---------------------------|--------------------------------------|---------------------------------------|
| <b>DFS</b>                | Backtracking = DFS on state space    | Backtracking = DFS with pruning       |
| <b>DP</b>                 | DP = Backtracking + memoization      | Use DP for overlapping subproblems    |
| <b>BFS</b>                | BFS for shortest path in state space | Use BFS when shortest solution needed |
| <b>Branch &amp; Bound</b> | Backtracking + cost bound            | Optimization problems with bounds     |

## 17. Practice Problems

### Easy

- **Subsets** — LeetCode — Basic include/exclude backtracking
- **Generate Parentheses** — LeetCode — Backtracking with string building

### Medium

- **Permutations** — LeetCode — Classic permutation backtracking
- **Combination Sum** — LeetCode — Backtracking with repetition allowed
- **N-Queens** — LeetCode — Constraint satisfaction backtracking

### Hard

- **Sudoku Solver** — LeetCode — Complex constraint satisfaction
- **Word Search II** — LeetCode — Trie + backtracking

## 18. Interview Explanation

"Backtracking is a systematic way to explore all possible solutions by building candidates incrementally. The core pattern is **choose-explore-undo**: I make a choice, recurse with the new state, then undo the choice to try another option. The key to making it efficient is **pruning** — cutting off branches as soon as they violate constraints. I use it for problems like generating all subsets, permutations, combinations, and constraint satisfaction problems like N-Queens and Sudoku."

## 19. Revision Notes

- **Template:** `if (goal) → add to result; for each choice: if valid → make choice → recurse → undo choice`
- **Pruning** is what makes backtracking practical
- **Always undo** the choice — this is the "backtrack" part
- **Clone results** when adding to answer list
- **Avoid duplicates** by sorting and skipping same elements at same level
- **Complexity:**  $O(\text{choices}^{\text{depth}})$  worst-case, but pruning helps

## 20. Final Cheat Sheet

| Aspect             | Summary                                                      |
|--------------------|--------------------------------------------------------------|
| <b>When to use</b> | All solutions, constraint satisfaction, combinatorial search |

| Aspect          | Summary                                               |
|-----------------|-------------------------------------------------------|
| Main operations | Choose → Explore → Undo (backtrack)                   |
| Complexity      | $O(b^d)$ worst-case, $O(d)$ stack space               |
| Key code        | <code>makeChoice(); backtrack(); undoChoice();</code> |
| Edge cases      | Empty input, no solution, duplicates, target=0        |

## 4. Subsets

### 1. Overview

The Subsets problem (also called "Power Set") asks: given an array of unique elements, return **all possible subsets** (the power set). The order of elements in a subset doesn't matter, and the empty set is always included.

### 2. Intuition

#### Simple explanation:

For each element, you have two choices: **include it** or **exclude it**. If you make this decision for every element, the set of all combinations of choices gives you all subsets.

#### Analogy:

A restaurant menu with  $n$  items. For each item, you decide yes/no. Every possible combination of items you could order is a subset of the menu.

#### Step-by-step reasoning:

1. Start with an empty subset.
2. Look at the first element. You can either take it or skip it.
3. For each of those choices, look at the next element and repeat.
4. When you've considered all elements, you have a complete subset.

#### Why it works:

The power set has exactly  $2^n$  subsets. Each subset is uniquely determined by which elements are included. The recursive decision tree explores all  $2^n$  possibilities.

### 3. When to Use It

- Need to generate **all subsets** of a set.
- Need to check **all possible selections** from a set.
- **Subproblems** that involve selecting a subset satisfying constraints.
- **Trigger phrases:** "power set", "all subsets", "all possible combinations of elements", "subsequence", "subarray".

### 4. When Not to Use It

- Only need **count** of subsets ( $2^n$ ) — just use math.
- Need **subarrays** (contiguous) — use sliding window.

- Need **subsequences** (ordered, not necessarily contiguous) — still backtracking, but different.
- Input size  $n > 25$  —  $2^{25} = 33$  million, too large for enumeration.

5. Core Concepts

| Concept                  | Explanation                     | Why It Matters                    |
|--------------------------|---------------------------------|-----------------------------------|
| Power Set                | Set of all subsets              | Contains $2^n$ elements           |
| Include/Exclude Decision | Binary choice per element       | Each element doubles the subsets  |
| Lexicographic Order      | Subsets in sorted order         | Often required by problems        |
| Bitmask Representation   | Integer with bits for inclusion | Efficient enumeration (iterative) |

6. Step-by-Step Algorithm

```
Input: nums = [1, 2, 3]

Step 1: Start with idx=0, current=[]
Step 2: At idx=0 (element=1):
  - Exclude: recurse with idx=1, current=[]
  - Include: recurse with idx=1, current=[1]
Step 3: At idx=1 (element=2):
  - For current=[]: exclude → []; include → [2]
  - For current=[1]: exclude → [1]; include → [1,2]
Step 4: At idx=2 (element=3):
  - For each of the 4 subsets, exclude/include 3 → 8 subsets
Step 5: When idx == n, add current to result

Result: [[], [3], [2], [2,3], [1], [1,3], [1,2], [1,2,3]]
```

7. Dry Run

Input: `nums = [1, 2]`

| idx | current | Action          | Result Added |
|-----|---------|-----------------|--------------|
| 0   | []      | Exclude 1       | —            |
| 1   | []      | Exclude 2       | —            |
| 2   | []      | Base case       | ✓ []         |
| —   | []      | Backtrack       | —            |
| 1   | []      | Include 2 → [2] | —            |
| 2   | [2]     | Base case       | ✓ [2]        |
| —   | [2]     | Backtrack → []  | —            |
| 0   | []      | Include 1 → [1] | —            |

| idx | current | Action            | Result Added |
|-----|---------|-------------------|--------------|
| 1   | [1]     | Exclude 2         | —            |
| 2   | [1]     | Base case         | ✓ [1]        |
| —   | [1]     | Backtrack         | —            |
| 1   | [1]     | Include 2 → [1,2] | —            |
| 2   | [1,2]   | Base case         | ✓ [1,2]      |

**Final result:** `[], [2], [1], [1,2]`

### 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Subsets {
    vector<vector<int>> result;
    vector<int> current;

    void backtrack(vector<int>& nums, int idx) {
        // Base case: all elements processed
        if (idx == nums.size()) {
            result.push_back(current);
            return;
        }

        // Option 1: Exclude current element
        backtrack(nums, idx + 1);

        // Option 2: Include current element
        current.push_back(nums[idx]);
        backtrack(nums, idx + 1);

        // Backtrack
        current.pop_back();
    }

public:
    vector<vector<int>> generate(vector<int>& nums) {
        result.clear();
        current.clear();
        backtrack(nums, 0);
        return result;
    }
};

// Iterative approach (bitmask)
vector<vector<int>> subsetsIterative(vector<int>& nums) {
    int n = nums.size();
```

```

vector<vector<int>> result;

for (int mask = 0; mask < (1 << n); mask++) {
    vector<int> subset;
    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) {
            subset.push_back(nums[i]);
        }
    }
    result.push_back(subset);
}
return result;
}

int main() {
    Subsets solver;
    vector<int> nums = {1, 2, 3};
    auto res = solver.generate(nums);

    cout << "Total subsets: " << res.size() << "\n";
    for (auto& subset : res) {
        cout << "[";
        for (int x : subset) cout << x << " ";
        cout << "]\n";
    }
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class Subsets:
    def __init__(self):
        self.result = []
        self.current = []

    def backtrack(self, nums: List[int], idx: int):
        # Base case: all elements processed
        if idx == len(nums):
            self.result.append(self.current[:])
            return

        # Option 1: Exclude current element
        self.backtrack(nums, idx + 1)

        # Option 2: Include current element
        self.current.append(nums[idx])
        self.backtrack(nums, idx + 1)

        # Backtrack

```

```

        self.current.pop()

    def generate(self, nums: List[int]) -> List[List[int]]:
        self.result = []
        self.current = []
        self.backtrack(nums, 0)
        return self.result

# Iterative approach (bitmask)
def subsets_iterative(nums: List[int]) -> List[List[int]]:
    n = len(nums)
    result = []
    for mask in range(1 << n):
        subset = []
        for i in range(n):
            if mask & (1 << i):
                subset.append(nums[i])
        result.append(subset)
    return result

# Example usage
solver = Subsets()
print(solver.generate([1, 2, 3]))
# [[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]

```

## 10. Code Explanation

### Recursive approach:

- `backtrack(nums, idx)` processes element at index `idx`.
- **Base case:** When `idx == n`, all decisions are made — add `current` to result.
- **Exclude:** Simply recurse to next index without adding the element.
- **Include:** Add `nums[idx]` to current, recurse, then pop (backtrack).
- The order of exclude-then-include determines the order of subsets in the result.

### Iterative (bitmask) approach:

- Each mask from `0` to  $2^n - 1$  represents a subset.
- Bit `i` set to 1 means include `nums[i]`.
- More efficient (no recursion overhead), but doesn't demonstrate backtracking.

## 11. Complexity Analysis

| Aspect                | Complexity                                                           |
|-----------------------|----------------------------------------------------------------------|
| <b>Time</b>           | $O(n \times 2^n)$ — $n$ elements, $2^n$ subsets, $O(n)$ to copy each |
| <b>Space (stack)</b>  | $O(n)$ — recursion depth                                             |
| <b>Space (result)</b> | $O(n \times 2^n)$ — storing all subsets                              |
| <b>Bitmask time</b>   | $O(n \times 2^n)$ — same                                             |



## 12. Common Patterns

| Pattern                        | Description                           | Example              |
|--------------------------------|---------------------------------------|----------------------|
| <b>Standard Subsets</b>        | Unique elements, any order            | LeetCode 78          |
| <b>Subsets with Duplicates</b> | Input has duplicates, skip same level | LeetCode 90          |
| <b>Subset Sum</b>              | Find subsets with sum = target        | Variation            |
| <b>Bitmask Generation</b>      | Iterative using bit operations        | Fast for $n \leq 20$ |

## 13. Common Mistakes

- **Forgetting to sort** before handling duplicates (for LeetCode 90).
- **Not cloning the result** — storing reference to `current`.
- **Wrong order** — order of include/exclude affects lexicographic order.
- **Integer overflow** for  $1 \ll n$  when  $n > 30$  (use  $1LL \ll n$ ).

## 14. Edge Cases

- **Empty array** →  `[[] ]` (one subset: empty set).
- **Single element** →  `[ [] , [ 1 ] ]`.
- **n = 20** → 1,048,576 subsets — manageable but large.
- **n = 25** → 33 million subsets — too large for most problems.

## 15. Variations

| Variation                      | What Changes                                   | Importance |
|--------------------------------|------------------------------------------------|------------|
| <b>Subsets with Duplicates</b> | Skip same element at same level                | High       |
| <b>Subset Sum</b>              | Track sum, prune if sum > target               | Medium     |
| <b>Subsets of size K</b>       | Add only when <code>current.size() == k</code> | Medium     |
| <b>Lexicographic Order</b>     | Include before exclude                         | Low        |

## 16. Related Algorithms

| Algorithm           | Connection                          |
|---------------------|-------------------------------------|
| <b>Combinations</b> | Subsets of fixed size k             |
| <b>Permutations</b> | Different ordering matters          |
| <b>Bitmask DP</b>   | DP over subsets (TSP, matching)     |
| <b>Backtracking</b> | Generalization of subset generation |

## 17. Practice Problems

### Easy

- **Subsets** — LeetCode 78 — Basic subset generation
- **Subset Sums** — GFG — Generate all subset sums

## Medium

- **Subsets II** — LeetCode 90 — Subsets with duplicates
- **Sum of Subset XOR Totals** — LeetCode 1863 — Bitmask approach
- **Partition Equal Subset Sum** — LeetCode 416 — Subset sum DP

## Hard

- **Subsets with target sum** — Variation — Subset sum with backtracking
- **Maximum XOR of Two Numbers** — LeetCode 421 — Subset XOR with trie

## 18. Interview Explanation

"The subsets problem asks for all possible subsets of a given set. I solve it using backtracking: for each element, I make a binary decision — include it or exclude it. When I've processed all elements, I add the current subset to the result. There are exactly  $2^n$  subsets, and the time complexity is  $O(n \times 2^n)$ . For small  $n$ , I can also use a bitmask approach — iterate over all integers from 0 to  $2^n-1$ , and use the set bits to indicate included elements."

## 19. Revision Notes

- **$2^n$  subsets** total
- **Include/exclude** decision per element
- **Backtracking:** exclude → include → undo include
- **Bitmask:** `mask = 0` to  $2^n-1$ , check `mask & (1<<i)`
- **Duplicates:** sort, skip `nums[i] == nums[i-1]` at same level
- **Complexity:**  $O(n \times 2^n)$

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------|
| <b>When to use</b>     | Generate all subsets, power set, all selections                                                        |
| <b>Main operations</b> | Include/Exclude each element → recurse → undo                                                          |
| <b>Complexity</b>      | $O(n \times 2^n)$ time, $O(n)$ stack                                                                   |
| <b>Key code</b>        | <code>backtrack(idx+1); current.push_back(nums[idx]); backtrack(idx+1);<br/>current.pop_back();</code> |
| <b>Edge cases</b>      | Empty array → <code>[]</code> , $n > 20$ → consider bitmask                                            |

## 5. Permutations

### 1. Overview

Given an array of distinct integers, return **all possible arrangements** (permutations) of the elements. Order matters — `[1,2,3]` and `[3,2,1]` are different permutations. There are  $n!$  permutations for  $n$  elements.

## 2. Intuition

### Simple explanation:

Imagine you have  $n$  boxes and  $n$  distinct balls. For the first box, you can choose any of the  $n$  balls. For the second box, you can choose any of the remaining  $n-1$  balls. Continue until all boxes are filled.

### Analogy:

Arranging books on a shelf. For the first position, you pick any book. For the second, you pick from the remaining books, and so on. Each different arrangement is a permutation.

### Step-by-step reasoning:

1. For position 0, try each element as the first element.
2. For position 1, try each remaining element as the second element.
3. Continue until all positions are filled.
4. Use a "used" array (or swapping) to track which elements are already placed.

### Why it works:

The number of permutations is  $n! = n \times (n-1) \times \dots \times 1$ . The branching factor decreases by 1 at each level. This naturally models the "pick one from remaining" process.

## 3. When to Use It

- Need **all arrangements** of elements.
- Need **all orderings** that satisfy certain constraints.
- **Trigger phrases:** "all permutations", "all arrangements", "all possible orders", "rearrange", "distinct arrangements".

## 4. When Not to Use It

- Input size  $n > 10$  —  $10! = 3.6\text{M}$ ,  $11! = 39.9\text{M}$  (too large).
- Only need **count** of permutations ( $n!$ ).
- Only need **next permutation** (lexicographic) — use `next_permutation()`.
- Elements are **not distinct** — need to handle duplicates carefully.
- Only need **k-th permutation** — use factorial number system (LeetCode 60).

## 5. Core Concepts

| Concept                             | Explanation                                          | Why It Matters             |
|-------------------------------------|------------------------------------------------------|----------------------------|
| <b><math>n!</math> permutations</b> | Total count = $n \times (n-1) \times \dots \times 1$ | Grows extremely fast       |
| <b>Used array</b>                   | Tracks which elements are already placed             | $O(n)$ space, $O(1)$ check |
| <b>Swapping approach</b>            | Swap elements in-place to generate permutations      | $O(1)$ extra space         |
| <b>Lexicographic Order</b>          | Sorted order of permutations                         | Often required             |

## 6. Step-by-Step Algorithm

Input: nums = [1, 2, 3]

Step 1: Start with empty permutation, used = [F, F, F]

Step 2: Pick first element:

- Pick 1: perm=[1], used=[T,F,F], recurse
  - Pick 2: perm=[1,2], used=[T,T,F], recurse
    - Pick 3: perm=[1,2,3], used=[T,T,T] → add to result
  - Pick 3: perm=[1,3], used=[T,F,T], recurse
    - Pick 2: perm=[1,3,2], used=[T,T,T] → add to result
- Pick 2: perm=[2], used=[F,T,F], recurse
  - Pick 1: perm=[2,1], used=[T,T,F], recurse
    - Pick 3: perm=[2,1,3] → add
  - Pick 3: perm=[2,3], used=[F,T,T], recurse
    - Pick 1: perm=[2,3,1] → add
- Pick 3: perm=[3], used=[F,F,T], recurse
  - Pick 1: perm=[3,1], used=[T,F,T], recurse
    - Pick 2: perm=[3,1,2] → add
  - Pick 2: perm=[3,2], used=[F,T,T], recurse
    - Pick 1: perm=[3,2,1] → add

Result: [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]

## 7. Dry Run (Swapping Approach)

Input: nums = [1, 2, 3]

| Level | Array State | Fixed Prefix | Action                    |
|-------|-------------|--------------|---------------------------|
| 0     | [1,2,3]     | []           | swap(0,0) → [1,2,3]       |
| 1     | [1,2,3]     | [1]          | swap(1,1) → [1,2,3]       |
| 2     | [1,2,3]     | [1,2]        | swap(2,2) → [1,2,3] → add |
| —     | —           | —            | swap back                 |
| 1     | [1,2,3]     | [1]          | swap(1,2) → [1,3,2]       |
| 2     | [1,3,2]     | [1,3]        | swap(2,2) → [1,3,2] → add |
| —     | —           | —            | swap back                 |
| 0     | [1,2,3]     | []           | swap(0,1) → [2,1,3]       |
| 1     | [2,1,3]     | [2]          | swap(1,1) → [2,1,3]       |
| 2     | [2,1,3]     | [2,1]        | swap(2,2) → [2,1,3] → add |
| ...   | ...         | ...          | ...                       |

**Final result:** [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Permutations {
    vector<vector<int>> result;

    void backtrack(vector<int>& nums, int start) {
        // Base case: all positions filled
        if (start == nums.size()) {
            result.push_back(nums);
            return;
        }

        for (int i = start; i < nums.size(); i++) {
            // Swap current element to position 'start'
            swap(nums[start], nums[i]);

            // Recurse for next position
            backtrack(nums, start + 1);

            // Backtrack (swap back)
            swap(nums[start], nums[i]);
        }
    }

public:
    vector<vector<int>> permute(vector<int>& nums) {
        result.clear();
        backtrack(nums, 0);
        return result;
    }
};

// Using visited array (more intuitive)
vector<vector<int>> permuteVisited(vector<int>& nums) {
    vector<vector<int>> result;
    vector<int> current;
    vector<bool> used(nums.size(), false);

    function<void()> backtrack = [&]() {
        if (current.size() == nums.size()) {
            result.push_back(current);
            return;
        }
        for (int i = 0; i < nums.size(); i++) {
            if (!used[i]) {
                used[i] = true;
                current.push_back(nums[i]);
                backtrack();
                current.pop_back();
                used[i] = false;
            }
        }
    };
    backtrack();
    return result;
}
```

```

    }
};

    backtrack();
    return result;
}

int main() {
    Permutations solver;
    vector<int> nums = {1, 2, 3};
    auto res = solver.permute(nums);

    cout << "Total permutations: " << res.size() << "\n";
    for (auto& perm : res) {
        cout << "[";
        for (int x : perm) cout << x << " ";
        cout << "]\n";
    }
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class Permutations:
    def __init__(self):
        self.result = []

    def backtrack(self, nums: List[int], start: int):
        # Base case: all positions filled
        if start == len(nums):
            self.result.append(nums[:])
            return

        for i in range(start, len(nums)):
            # Swap current element to position 'start'
            nums[start], nums[i] = nums[i], nums[start]

            # Recurse for next position
            self.backtrack(nums, start + 1)

            # Backtrack (swap back)
            nums[start], nums[i] = nums[i], nums[start]

    def permute(self, nums: List[int]) -> List[List[int]]:
        self.result = []
        self.backtrack(nums, 0)
        return self.result

# Using visited array (more intuitive)

```

```
def permute_visited(nums: List[int]) -> List[List[int]]:
    result = []
    current = []
    used = [False] * len(nums)

    def backtrack():
        if len(current) == len(nums):
            result.append(current[:])
            return
        for i in range(len(nums)):
            if not used[i]:
                used[i] = True
                current.append(nums[i])
                backtrack()
                current.pop()
                used[i] = False

    backtrack()
    return result

# Example usage
solver = Permutations()
print(solver.permute([1, 2, 3]))
# [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

## 10. Code Explanation

### Swapping approach:

- `start` indicates the position being filled.
- For each `i` from `start` to `n-1`, swap `nums[start]` with `nums[i]`.
- The prefix `nums[0...start]` is the fixed part of the permutation.
- After recursion, swap back to restore the array.
- **Advantage:**  $O(1)$  extra space, no used array.
- **Caveat:** Modifies the input array.

### Visited array approach:

- `used[i]` tracks whether `nums[i]` is already placed.
- `current` builds the permutation incrementally.
- At each step, pick any unused element.
- **Advantage:** Doesn't modify input, more intuitive.
- **Caveat:**  $O(n)$  extra space for `used` array.

## 11. Complexity Analysis

| Aspect        | Complexity                                                      |
|---------------|-----------------------------------------------------------------|
| Time          | $O(n \times n!)$ — $n!$ permutations, each takes $O(n)$ to copy |
| Space (stack) | $O(n)$ — recursion depth                                        |

| Aspect            | Complexity                                  |
|-------------------|---------------------------------------------|
| Space (result)    | $O(n \times n!)$ — storing all permutations |
| Swapping approach | Same complexity, but no extra used array    |

## 12. Common Patterns

| Pattern                      | Description                           | Example     |
|------------------------------|---------------------------------------|-------------|
| Standard Permutations        | All arrangements of distinct elements | LeetCode 46 |
| Permutations with Duplicates | Skip same elements at same level      | LeetCode 47 |
| Next Permutation             | Lexicographically next permutation    | LeetCode 31 |
| K-th Permutation             | Factorial number system               | LeetCode 60 |
| Permutation of String        | Generate all permutations of a string | GFG         |

## 13. Common Mistakes

- **Not handling duplicates** → duplicates in result when input has duplicates.
- **Forgetting to swap back** → state corruption in swapping approach.
- **Modifying result** after adding to list (store a copy).
- **Using `next_permutation` incorrectly** — it modifies the input.
- **n! overflow** —  $13! > 6$  billion, too large to store.

## 14. Edge Cases

- **n = 0** →  `[[] ]` (one empty permutation).
- **n = 1** →  `[[1] ]`.
- **n = 10** → 3,628,800 permutations — manageable but large.
- **n = 12** → 479M — too large for most practical purposes.
- **Duplicate elements** → need deduplication logic.

## 15. Variations

| Variation                    | What Changes                           | Importance |
|------------------------------|----------------------------------------|------------|
| Permutations with Duplicates | Sort + skip same element at same level | High       |
| Permutations of multiset     | Count-based generation (less frequent) | Medium     |
| String permutations          | Same logic, string instead of array    | High       |
| Circular permutations        | $(n-1)!$ permutations                  | Low        |

## 16. Related Algorithms

| Algorithm    | Connection                              |
|--------------|-----------------------------------------|
| Combinations | Order doesn't matter; k elements from n |



| Algorithm                      | Connection                                |
|--------------------------------|-------------------------------------------|
| <b>Subsets</b>                 | Any subset, not just full size            |
| <b>Next Permutation</b>        | Single permutation in lexicographic order |
| <b>Factorial Number System</b> | Direct k-th permutation computation       |

## 17. Practice Problems

### Easy

- **Permutations** — LeetCode 46 — Basic permutation generation
- **Build Array from Permutation** — LeetCode 1920 — Simple permutation application

### Medium

- **Permutations II** — LeetCode 47 — Permutations with duplicates
- **Next Permutation** — LeetCode 31 — Single next permutation in  $O(n)$
- **Permutation in String** — LeetCode 567 — Sliding window + permutation check

### Hard

- **K-th Permutation** — LeetCode 60 — Factorial number system
- **Palindrome Permutation II** — LeetCode 267 — Generate palindrome permutations

## 18. Interview Explanation

"For generating all permutations of distinct elements, I use a backtracking approach. The key insight is that at each position, I can place any remaining element. I use either a swapping approach — swap each element to the current position, recurse, and swap back — or a visited array to track used elements. The time complexity is  $O(n \times n!)$  because there are  $n!$  permutations and each takes  $O(n)$  to copy. For  $n > 10$ , I'd consider whether we really need all permutations or just a specific one."

## 19. Revision Notes

- **$n!$  permutations** total
- **Swapping approach:** swap → recurse → swap back ( $O(1)$  extra space)
- **Visited array approach:** more intuitive,  $O(n)$  extra space
- **Duplicates:** sort + skip `nums[i] == nums[i-1]` when `!used[i-1]`
- **$n \leq 10$**  for practical enumeration
- **Alternative:** `next_permutation()` for lexicographic order

## 20. Final Cheat Sheet

| Aspect                 | Summary                                      |
|------------------------|----------------------------------------------|
| <b>When to use</b>     | All arrangements, all orderings              |
| <b>Main operations</b> | Pick from remaining → place → recurse → undo |

| Aspect     | Summary                                                                                  |
|------------|------------------------------------------------------------------------------------------|
| Complexity | $O(n \times n!)$ time, $O(n)$ stack                                                      |
| Key code   | <code>swap(nums[start], nums[i]); backtrack(start+1); swap(nums[start], nums[i]);</code> |
| Edge cases | $n=0 \rightarrow [[]]$ , $n=1 \rightarrow [[1]]$ , duplicates need handling              |

## 6. Combinations

### 1. Overview

Given two integers  $n$  and  $k$ , return **all combinations** of  $k$  numbers chosen from the range  $[1, n]$ . Order doesn't matter —  $[1, 2]$  and  $[2, 1]$  are the same combination. The total number is  $C(n, k) = n! / (k! \times (n-k)!)$ .

### 2. Intuition

#### Simple explanation:

Choose  $k$  items from  $n$  items where order doesn't matter. You pick items in increasing order to avoid duplicates. Once you pick a number, subsequent picks must be larger.

#### Analogy:

Choosing a team of 3 players from a group of 10. The order of selection doesn't matter — {Alice, Bob, Charlie} is the same team as {Charlie, Bob, Alice}. To avoid counting the same team multiple times, you always pick players in alphabetical order.

#### Step-by-step reasoning:

1. Start with the smallest available number.
2. Pick a number, then pick the next number from numbers greater than it.
3. Continue until you've picked  $k$  numbers.
4. The "increasing order" constraint prevents duplicates.

#### Why it works:

By enforcing that we always pick numbers in increasing order, each combination is generated exactly once. This is the key difference from permutations — order doesn't matter, so we fix an ordering.

### 3. When to Use It

- Need **all subsets of size  $k$** .
- Need **all teams/groups** of fixed size.
- **Trigger phrases:** "combinations", "choose  $k$ ", "all possible selections of  $k$ ", "team formation", " $k$ -combinations".

### 4. When Not to Use It

- Only need **count**  $C(n, k)$  — use math formula.
- Order matters — use permutations.
- Need **all subsets** (any size) — use Subsets.

- $n$  is large ( $n > 30$ ) —  $C(n, k)$  can be huge.
- Need **one optimal combination** — use DP or greedy.

5. Core Concepts

| Concept                 | Explanation                                        | Why It Matters                             |
|-------------------------|----------------------------------------------------|--------------------------------------------|
| <b>C(n, k)</b>          | Number of combinations = $n! / (k! \times (n-k)!)$ | Grows quickly but slower than permutations |
| <b>Increasing Order</b> | Pick elements in sorted order                      | Prevents duplicates                        |
| <b>Start Index</b>      | Only pick from elements $\geq$ start               | Ensures increasing order                   |
| <b>Pruning</b>          | Stop if remaining elements $<$ needed              | Optimizes the search                       |

6. Step-by-Step Algorithm

```
Input: n = 4, k = 2

Step 1: Start with start=1, current=[]
Step 2: Pick 1: current=[1], recurse with start=2
  - Pick 2: current=[1,2], size=k → add [1,2]
  - Pick 3: current=[1,3], size=k → add [1,3]
  - Pick 4: current=[1,4], size=k → add [1,4]
Step 3: Pick 2: current=[2], recurse with start=3
  - Pick 3: current=[2,3], size=k → add [2,3]
  - Pick 4: current=[2,4], size=k → add [2,4]
Step 4: Pick 3: current=[3], recurse with start=4
  - Pick 4: current=[3,4], size=k → add [3,4]
Step 5: Pick 4: can't reach k=2 (only one element left) → pruned

Result: [[1,2], [1,3], [1,4], [2,3], [2,4], [3,4]]
```

7. Dry Run

Input:  $n = 5, k = 3$

| start | current | Choices   | Added?    |
|-------|---------|-----------|-----------|
| 1     | []      | 1,2,3,4,5 | —         |
| 2     | [1]     | 2,3,4,5   | —         |
| 3     | [1,2]   | 3,4,5     | —         |
| 4     | [1,2,3] | —         | ✓ [1,2,3] |
| 4     | [1,2,4] | —         | ✓ [1,2,4] |
| 5     | [1,2,5] | —         | ✓ [1,2,5] |

| start | current | Choices | Added?    |
|-------|---------|---------|-----------|
| 3     | [1,3]   | 4,5     | —         |
| 4     | [1,3,4] | —       | ✓ [1,3,4] |
| 5     | [1,3,5] | —       | ✓ [1,3,5] |
| 4     | [1,4]   | 5       | —         |
| 5     | [1,4,5] | —       | ✓ [1,4,5] |
| 2     | [2]     | 3,4,5   | —         |
| 3     | [2,3]   | 4,5     | —         |
| ...   | ...     | ...     | ...       |

**Total combinations:**  $C(5,3) = 10$

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class Combinations {
    vector<vector<int>>> result;
    vector<int> current;

    void backtrack(int n, int k, int start) {
        // Base case: selected k elements
        if (current.size() == k) {
            result.push_back(current);
            return;
        }

        // Prune: not enough remaining elements
        int remaining = k - current.size();
        if (remaining > n - start + 1) return;

        for (int i = start; i <= n; i++) {
            current.push_back(i);
            backtrack(n, k, i + 1);
            current.pop_back();
        }
    }

public:
    vector<vector<int>>> combine(int n, int k) {
        result.clear();
        current.clear();
        backtrack(n, k, 1);
        return result;
    }
}
```

```
};

int main() {
    Combinations solver;
    auto res = solver.combine(4, 2);

    cout << "Total combinations: " << res.size() << "\n";
    for (auto& comb : res) {
        cout << "[";
        for (int x : comb) cout << x << " ";
        cout << "]\n";
    }
    // Output: [1 2] [1 3] [1 4] [2 3] [2 4] [3 4]
    return 0;
}
```

## 9. Python Implementation

```
from typing import List

class Combinations:
    def __init__(self):
        self.result = []
        self.current = []

    def backtrack(self, n: int, k: int, start: int):
        # Base case: selected k elements
        if len(self.current) == k:
            self.result.append(self.current[:])
            return

        # Prune: not enough remaining elements
        remaining = k - len(self.current)
        if remaining > n - start + 1:
            return

        for i in range(start, n + 1):
            self.current.append(i)
            self.backtrack(n, k, i + 1)
            self.current.pop()

    def combine(self, n: int, k: int) -> List[List[int]]:
        self.result = []
        self.current = []
        self.backtrack(n, k, 1)
        return self.result

# Example usage
solver = Combinations()
```

```
print(solver.combine(4, 2))
# [[1, 2], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]]
```

## 10. Code Explanation

- **backtrack(n, k, start)**: **start** is the smallest number we can pick next.
- **Base case**: When `current.size() == k`, we have a complete combination.
- **Pruning**: `remaining > n - start + 1` — if we need more elements than available, stop.
- **Loop**: Try each number from **start** to **n**.
- **Recurse**: `backtrack(n, k, i + 1)` — next pick must be greater than **i**.
- **Backtrack**: Pop the last element after recursion.

## 11. Complexity Analysis

| Aspect         | Complexity                                                      |
|----------------|-----------------------------------------------------------------|
| Time           | $O(k \times C(n, k))$ — for each combination, copy $k$ elements |
| Space (stack)  | $O(k)$ — recursion depth                                        |
| Space (result) | $O(k \times C(n, k))$                                           |
| Pruning        | Reduces practical runtime significantly                         |

## 12. Common Patterns

| Pattern                      | Description                 | Example     |
|------------------------------|-----------------------------|-------------|
| Standard Combinations        | $C(n, k)$ from $[1..n]$     | LeetCode 77 |
| Combination Sum              | Numbers that sum to target  | LeetCode 39 |
| Letter Combinations          | Phone number mapping        | LeetCode 17 |
| Combinations with Repetition | Allow reuse of same element | Variation   |

## 13. Common Mistakes

- **Starting from 0 instead of 1** — depends on problem (index vs value).
- **Not pruning** — unnecessary work for invalid branches.
- **Wrong loop bounds** — using **n** instead of `n - k + current.size() + 1`.
- **Forgetting to pop** — state corruption.
- **Confusing combinations with permutations** — combinations use **start** parameter.

## 14. Edge Cases

- **k = 0** → `[]` (empty combination).
- **k = n** → `[1, 2, ..., n]` (one combination: all elements).
- **k > n** → `[]` (no valid combinations).
- **n = 0** → `[]` (no range).
- **Large n, k** —  $C(n, k)$  may overflow memory.

## 15. Variations

| Variation                           | What Changes                      | Importance |
|-------------------------------------|-----------------------------------|------------|
| <b>Combinations with Repetition</b> | Allow reuse (no $i+1$ , use $i$ ) | High       |
| <b>Letter Combinations of Phone</b> | Mapping digits to letters         | High       |
| <b>Combination Sum</b>              | Sum to target, with/without reuse | High       |
| <b>k-combinations from array</b>    | Input array instead of $1..n$     | Medium     |

## 16. Related Algorithms

| Algorithm           | Connection                           |
|---------------------|--------------------------------------|
| <b>Subsets</b>      | Combinations are subsets of size $k$ |
| <b>Permutations</b> | Order matters in permutations        |
| <b>Subset Sum</b>   | Combinations that sum to target      |
| <b>Backtracking</b> | General framework for combinations   |

## 17. Practice Problems

### Easy

- **Combinations** — LeetCode 77 — Basic  $C(n, k)$
- **Letter Combinations of a Phone Number** — LeetCode 17 — Mapping combinations

### Medium

- **Combination Sum** — LeetCode 39 — Combinations summing to target
- **Combination Sum II** — LeetCode 40 — With duplicates, no reuse
- **Combination Sum III** — LeetCode 216 —  $K$  numbers from  $1-9$  summing to  $n$

### Hard

- **Factor Combinations** — LeetCode 254 — Factor product combinations
- **Combinations with duplicates** — Variation

## 18. Interview Explanation

"For combinations, I use backtracking with a start index to ensure combinations are generated in increasing order, which prevents duplicates. At each step, I pick a number, recurse for the next position starting from the next number, then backtrack. I also prune branches where there aren't enough remaining numbers to reach size  $k$ . The time complexity is  $O(k \times C(n,k))$ . The key difference from permutations is that I use a start parameter to enforce ordering, not a used array."

## 19. Revision Notes

- **$C(n, k) = n! / (k! \times (n-k)!)$**
- **Start index** prevents duplicates (enforces increasing order)

- **Pruning:**  $\text{remaining} > n - \text{start} + 1 \rightarrow \text{skip}$
- $k = 0 \rightarrow [[]]$ ,  $k = n \rightarrow [[1..n]]$
- **Complexity:**  $O(k \times C(n, k))$
- **Key difference from permutations:** order doesn't matter

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                           |
|------------------------|-----------------------------------------------------------------------------------|
| <b>When to use</b>     | All subsets of size $k$ , team selection                                          |
| <b>Main operations</b> | Pick element $\rightarrow$ recurse with next start $\rightarrow$ undo             |
| <b>Complexity</b>      | $O(k \times C(n, k))$ time, $O(k)$ stack                                          |
| <b>Key code</b>        | <code>for i in [start..n]: current.push(i); backtrack(i+1); current.pop();</code> |
| <b>Edge cases</b>      | $k=0 \rightarrow [[]]$ , $k>n \rightarrow []$ , $k=n \rightarrow [[1..n]]$        |

## 7. N-Queens

### 1. Overview

The N-Queens problem asks: place  $N$  queens on an  $N \times N$  chessboard such that no two queens attack each other. Queens attack along rows, columns, and diagonals. Return all distinct board configurations.

### 2. Intuition

#### Simple explanation:

Place queens one row at a time. For each row, try every column. If a queen can be placed safely (no other queen in the same column or diagonal), place it and move to the next row. If no safe column exists in a row, backtrack to the previous row.

#### Analogy:

You're arranging VIPs in a theater row by row. Each VIP hates every other VIP in the same column or diagonal. Once seated, you cannot move them. If a row has no valid seat, you ask the previous row's VIP to move to another seat.

#### Step-by-step reasoning:

1. Start with row 0.
2. For the current row, try each column from 0 to  $N-1$ .
3. Check if placing a queen at (row, col) is safe.
4. If safe, place the queen and recurse to the next row.
5. If the recursion succeeds, record the solution.
6. If it fails, remove the queen and try the next column.
7. If no column works, backtrack to the previous row.

#### Why it works:

By placing one queen per row (which is required since each row must have exactly one queen), we reduce the search space from  $N^2$  to  $N^N$ . The column and diagonal checks further prune the tree.



### 3. When to Use It

- **Constraint satisfaction** problems with placement constraints.
- **Board games** with attack/conflict rules.
- **Trigger phrases:** "N-Queens", "place N queens", "no two attack each other", "non-attacking placements", "chessboard".

### 4. When Not to Use It

- $N > 15$  — search space grows too large ( $N!$  for N-Queens).
- Only need **count** — use closed-form formulas or bitmask DP (faster).
- Similar problem with **different constraints** — e.g., rooks (easy, just permutations).
- Only need **one solution** — newer algorithms exist for larger N.

### 5. Core Concepts

| Concept              | Explanation                                         | Why It Matters            |
|----------------------|-----------------------------------------------------|---------------------------|
| Row-by-row placement | One queen per row                                   | Reduces branching factor  |
| Column tracking      | Track which columns are used                        | Prevents column attacks   |
| Diagonal tracking    | Two diagonals (main and anti)                       | Prevents diagonal attacks |
| Safe check           | Queen at (r,c) is safe if no queen in same col/diag | Core constraint           |
| Board representation | String/vector of queen positions                    | Required for output       |

### 6. Step-by-Step Algorithm

```
Input: N = 4

Step 1: Row 0:
  Try col 0: safe → place queen at (0,0)
Row 1:
  Try col 0: not safe (col 0)
  Try col 1: not safe (diag)
  Try col 2: safe → place queen at (1,2)
Row 2:
  Try col 0: not safe (col 0)
  Try col 1: not safe (diag)
  Try col 2: not safe (col 2)
  Try col 3: not safe (diag)
  → No safe column, backtrack
Remove queen at (1,2)
Try col 3: safe → place queen at (1,3)
Row 2:
  Try col 0: not safe (col 0)
  Try col 1: safe → place queen at (2,1)
Row 3:
  Try col 0: not safe (col 0)
  Try col 1: not safe (col 1)
```

```
    Try col 2: not safe (diag)
    Try col 3: not safe (diag)
    → No safe column, backtrack
  Remove queen at (2,1)
  ... no more columns → backtrack
  Remove queen at (1,3)
  → No more columns, backtrack
  Remove queen at (0,0)

  Try col 1: safe → place queen at (0,1)
  ... (continues until solution found)

Solution 1: [[.Q..], [...Q], [Q...], [..Q.]]
Solution 2: [[..Q.], [Q...], [...Q], [.Q..]]
```

7. Dry Run

Input: N = 4, tracking columns and diagonals

| Row | Col | Cols Used | Main Diag (r-c+N-1) | Anti Diag (r+c) | Safe?  | Action    |
|-----|-----|-----------|---------------------|-----------------|--------|-----------|
| 0   | 0   | {}        | {}                  | {}              | ✓      | Place     |
| 1   | 0   | {0}       | {3}                 | {0}             | ✗ col  | Skip      |
| 1   | 1   | {0}       | {3}                 | {0}             | ✗ anti | Skip      |
| 1   | 2   | {0}       | {3}                 | {0}             | ✓      | Place     |
| 2   | 0   | {0,2}     | {3,2}               | {0,3}           | ✗ col  | Skip      |
| 2   | 1   | {0,2}     | {3,2}               | {0,3}           | ✗ anti | Skip      |
| 2   | 2   | {0,2}     | {3,2}               | {0,3}           | ✗ col  | Skip      |
| 2   | 3   | {0,2}     | {3,2}               | {0,3}           | ✗ anti | Skip      |
| —   | —   | —         | —                   | —               | —      | Backtrack |
| 1   | 3   | {0}       | {3}                 | {0}             | ✓      | Place     |
| 2   | 0   | {0,3}     | {3,2}               | {0,4}           | ✗ col  | Skip      |
| 2   | 1   | {0,3}     | {3,2}               | {0,4}           | ✓      | Place     |
| 3   | 0   | {0,3,1}   | {3,2,0}             | {0,4,3}         | ✗ col  | Skip      |
| 3   | 1   | {0,3,1}   | {3,2,0}             | {0,4,3}         | ✗ col  | Skip      |
| 3   | 2   | {0,3,1}   | {3,2,0}             | {0,4,3}         | ✗ anti | Skip      |
| 3   | 3   | {0,3,1}   | {3,2,0}             | {0,4,3}         | ✗ anti | Skip      |
| —   | —   | —         | —                   | —               | —      | Backtrack |

Final result for N=4: 2 solutions.

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class NQueens {
    vector<vector<string>> result;
    vector<bool> cols, diag1, diag2;
    vector<int> queens; // queens[row] = col

    bool isSafe(int row, int col, int n) {
        return !cols[col] && !diag1[row - col + n - 1] && !diag2[row + col];
    }

    void placeQueen(int row, int col, int n) {
        queens[row] = col;
        cols[col] = true;
        diag1[row - col + n - 1] = true;
        diag2[row + col] = true;
    }

    void removeQueen(int row, int col, int n) {
        queens[row] = -1;
        cols[col] = false;
        diag1[row - col + n - 1] = false;
        diag2[row + col] = false;
    }

    void buildBoard(int n) {
        vector<string> board(n, string(n, '.'));
        for (int r = 0; r < n; r++) {
            board[r][queens[r]] = 'Q';
        }
        result.push_back(board);
    }

    void backtrack(int row, int n) {
        if (row == n) {
            buildBoard(n);
            return;
        }

        for (int col = 0; col < n; col++) {
            if (isSafe(row, col, n)) {
                placeQueen(row, col, n);
                backtrack(row + 1, n);
                removeQueen(row, col, n);
            }
        }
    }

public:
```

```

vector<vector<string>> solveNQueens(int n) {
    result.clear();
    cols.assign(n, false);
    diag1.assign(2 * n - 1, false);
    diag2.assign(2 * n - 1, false);
    queens.assign(n, -1);
    backtrack(0, n);
    return result;
}

};

int main() {
    NQueens solver;
    auto res = solver.solveNQueens(4);

    cout << "Total solutions for N=4: " << res.size() << "\n";
    for (int i = 0; i < res.size(); i++) {
        cout << "Solution " << i + 1 << ":\n";
        for (string& row : res[i]) {
            cout << row << "\n";
        }
        cout << "\n";
    }
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class NQueens:
    def __init__(self):
        self.result = []

    def solveNQueens(self, n: int) -> List[List[str]]:
        self.result = []
        self.cols = [False] * n
        self.diag1 = [False] * (2 * n - 1) # r - c + n - 1
        self.diag2 = [False] * (2 * n - 1) # r + c
        self.queens = [-1] * n # queens[row] = col

        self.backtrack(0, n)
        return self.result

    def backtrack(self, row: int, n: int):
        if row == n:
            # Build board
            board = ['. ' * n for _ in range(n)]
            for r in range(n):
                board[r] = board[r][:self.queens[r]] + 'Q' + board[r][self.queens[r]+1:]

```

```

        self.result.append(board)
        return

    for col in range(n):
        if self.is_safe(row, col, n):
            self.place_queen(row, col, n)
            self.backtrack(row + 1, n)
            self.remove_queen(row, col, n)

    def is_safe(self, row: int, col: int, n: int) -> bool:
        return not (self.cols[col] or
                    self.diag1[row - col + n - 1] or
                    self.diag2[row + col])

    def place_queen(self, row: int, col: int, n: int):
        self.queens[row] = col
        self.cols[col] = True
        self.diag1[row - col + n - 1] = True
        self.diag2[row + col] = True

    def remove_queen(self, row: int, col: int, n: int):
        self.queens[row] = -1
        self.cols[col] = False
        self.diag1[row - col + n - 1] = False
        self.diag2[row + col] = False

# Example usage
solver = NQueens()
solutions = solver.solveNQueens(4)
print(f"Total solutions for N=4: {len(solutions)}")
for i, board in enumerate(solutions):
    print(f"Solution {i + 1}:")
    for row in board:
        print(row)
    print()

```

## 10. Code Explanation

- **cols[col]**: Tracks which columns are occupied.
- **diag1[r-c+N-1]**: Tracks main diagonal (top-left to bottom-right). All cells on the same main diagonal have the same **r-c** value.
- **diag2[r+c]**: Tracks anti-diagonal (top-right to bottom-left). All cells on the same anti-diagonal have the same **r+c** value.
- **isSafe()**: Check if column and both diagonals are free.
- **placeQueen()** / **removeQueen()**: Update tracking arrays.
- **buildBoard()**: Convert queen positions to string board representation.
- **Backtracking**: Row by row, try each column, recurse, undo.

## 11. Complexity Analysis

| Aspect                         | Complexity                                       |
|--------------------------------|--------------------------------------------------|
| <b>Time (worst-case)</b>       | $O(N!)$ — $N$ queens, decreasing choices per row |
| <b>Time (with pruning)</b>     | Much less than $N!$ in practice                  |
| <b>Space (stack)</b>           | $O(N)$                                           |
| <b>Space (tracking arrays)</b> | $O(N)$                                           |
| <b>Space (result)</b>          | $O(N^2 \times \text{number of solutions})$       |

## 12. Common Patterns

| Pattern                        | Description                       | Example           |
|--------------------------------|-----------------------------------|-------------------|
| <b>Row-by-row placement</b>    | One queen per row                 | Standard N-Queens |
| <b>Bitmask optimization</b>    | Use bitsets for columns/diagonals | Fast N-Queens     |
| <b>N-Queens II</b>             | Count only, no board construction | LeetCode 52       |
| <b>N-Queens with obstacles</b> | Some cells blocked                | Variation         |

## 13. Common Mistakes

- **Off-by-one in diagonal indices:** `diag1` size =  $2N-1$ , index =  $r-c+N-1$ .
- **Not using `const` reference** for result strings.
- **Forgetting to backtrack** after recursion.
- **Checking entire board** for safety each time ( $O(N)$  per check) — use tracking arrays for  $O(1)$ .
- **Wrong row/col ordering** — row is the recursion level, col is the loop variable.

## 14. Edge Cases

- **$N = 1$**  → `[[Q]]` (one solution).
- **$N = 2, 3$**  → no solutions.
- **$N = 0$**  → `[][]` (empty board).
- **Large  $N$**  ( $N > 15$ ) → too many solutions, impractical.

## 15. Variations

| Variation                        | What Changes                    | Importance               |
|----------------------------------|---------------------------------|--------------------------|
| <b>N-Queens II</b> (LeetCode 52) | Count only                      | High                     |
| <b>N-Queens with bitmask</b>     | Use int bitmasks for cols/diags | High (CP)                |
| <b>N-Queens with obstacles</b>   | Some cells blocked              | Medium                   |
| <b>N-Rooks</b>                   | No diagonal constraints         | Easy (just permutations) |
| <b>N-Knights</b>                 | Different movement rules        | Medium                   |

## 16. Related Algorithms

| Algorithm                      | Connection                           |
|--------------------------------|--------------------------------------|
| <b>Sudoku Solver</b>           | Similar constraint satisfaction      |
| <b>Graph Coloring</b>          | N-Queens is a graph coloring variant |
| <b>Backtracking</b>            | General framework                    |
| <b>Constraint Satisfaction</b> | N-Queens is a classic CSP            |

## 17. Practice Problems

### Easy

- **N-Queens II** — LeetCode 52 — Count solutions (easier than constructing boards)
- **N-Queens** — LeetCode 51 — Construct all solutions

### Medium

- **Non-Attacking Knights** — Variation — Knights instead of queens
- **N-Queens with bitset** — CP — Optimized with bit operations

### Hard

- **N-Queens with obstacle** — Variation — Some cells blocked
- **N-Queens for N up to 100** — Advanced — Requires heuristic algorithms

## 18. Interview Explanation

"N-Queens is a classic constraint satisfaction problem solved with backtracking. I place queens one row at a time. For each row, I try every column and check if the position is safe using three tracking arrays — one for columns, two for diagonals. The key insight is that cells on the same main diagonal have the same  $r-c$  value, and cells on the same anti-diagonal have the same  $r+c$  value. This gives me  $O(1)$  safety checks. The time complexity is  $O(N!)$  worst-case, but pruning makes it much faster in practice."

## 19. Revision Notes

- **Row-by-row placement** = recursion level
- **Safety check:** `cols[col]`, `diag1[r-c+N-1]`, `diag2[r+c]`
- **N=1** → 1 solution; **N=2,3** → 0 solutions; **N=4** → 2 solutions
- **Complexity:**  $O(N!)$  worst-case,  $O(N)$  space
- **Key formula:** `diag1` index =  $r-c+N-1$ , `diag2` index =  $r+c$

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                 |
|------------------------|---------------------------------------------------------|
| <b>When to use</b>     | Placement problems, constraint satisfaction, chessboard |
| <b>Main operations</b> | Check safety → place → recurse → remove                 |
| <b>Complexity</b>      | $O(N!)$ time, $O(N)$ space                              |

| Aspect     | Summary                                                                  |
|------------|--------------------------------------------------------------------------|
| Key code   | <pre>if (isSafe(row,col)) { place(); backtrack(row+1); remove(); }</pre> |
| Edge cases | N=1 → 1 soln, N=2,3 → 0 solns, N=4 → 2 solns                             |

## 8. Sudoku Solver

### 1. Overview

Given a partially filled 9×9 Sudoku board, fill all empty cells such that each row, each column, and each of the nine 3×3 sub-boxes contains digits 1-9 exactly once.

### 2. Intuition

#### Simple explanation:

For each empty cell, try placing digits 1 through 9. After placing a digit, check if it violates any Sudoku rule. If not, move to the next empty cell. If a dead end is reached (no digit works for a cell), backtrack to the previous cell and try a different digit.

#### Analogy:

Like filling a crossword puzzle. You try a word in a slot. If it fits, you move to the next slot. If later you can't find a word that fits, you erase the previous word and try a different one.

#### Step-by-step reasoning:

1. Find the next empty cell.
2. For each digit 1-9, check if placing it is valid:
  - Not already in the same row.
  - Not already in the same column.
  - Not already in the same 3×3 box.
3. If valid, place the digit and recurse.
4. If the recursion returns true, we're done.
5. If not, remove the digit and try the next digit.
6. If no digit works, return false (backtrack).

#### Why it works:

The Sudoku constraints (row, column, box) create a well-defined search space. The pruning is very effective — typically only a few choices per cell are valid.

### 3. When to Use It

- **Solving Sudoku puzzles** of any size.
- **Constraint satisfaction** with multi-dimensional constraints.
- **Trigger phrases:** "Sudoku", "solve Sudoku", "fill the board", "9x9 grid with digits".

### 4. When Not to Use It

- **Validating a Sudoku** — just check constraints, no solving needed.
- **Generating Sudoku puzzles** — different algorithm needed.



- **Other grid puzzles** like Nonograms — use different techniques.
- **Very large Sudoku** (16×16 or larger) — may need more advanced techniques.

5. Core Concepts

| Concept                        | Explanation                               | Why It Matters     |
|--------------------------------|-------------------------------------------|--------------------|
| Row constraint                 | Each row has digits 1-9 exactly once      | Core rule          |
| Column constraint              | Each column has digits 1-9 exactly once   | Core rule          |
| Box constraint                 | Each 3×3 box has digits 1-9 exactly once  | Core rule          |
| Empty cell                     | Cell with value '.' or 0                  | Target for filling |
| Valid placement                | Digit doesn't violate any constraint      | Pruning condition  |
| MRV (Minimum Remaining Values) | Fill cell with fewest valid options first | Optimization       |

6. Step-by-Step Algorithm

```
Input: Partially filled 9×9 board

Step 1: Find the first empty cell (row, col).
Step 2: For digit = 1 to 9:
  a. Check if placing digit at (row, col) is valid.
  b. If valid:
    - Place digit.
    - Recursively solve from this state.
    - If recursive call returns true, return true.
    - Otherwise, remove digit (backtrack) and try next digit.
Step 3: If no digit works, return false.
Step 4: If no empty cells remain, return true (puzzle solved).
```

7. Dry Run

**Input:** Partially filled board (simplified 4×4 for demonstration)

```
Initial board:
[1, ., ., 4]
[., ., 3, .]
[., 1, ., .]
[4, ., ., 2]

Step 1: Find empty cell at (0,1)
Step 2: Try digit 2:
  Row: 1,2,4 -> 2 ok
  Col: .,.,1,4 -> 2 ok
  Box (0-1,0-1): 1,.,.,. -> 2 ok
  Place 2 at (0,1)
  Recurse: find empty cell at (0,2)
```

```

Try digit 3:
  Row: 1,2,3,4 -> 3 ok
  Col: .,.,.,. -> 3 ok
  Box: 1,2,.,. -> 3 ok
  Place 3 at (0,2)
  ... continue until solved or backtrack

```

Final solved board:

```

[1, 2, 3, 4]
[3, 4, 1, 2]
[2, 1, 4, 3]
[4, 3, 2, 1]

```

## 8. C++ Implementation

```

#include <bits/stdc++.h>
using namespace std;

class SudokuSolver {
    vector<vector<char>> board;

    bool isValid(int row, int col, char ch) {
        for (int i = 0; i < 9; i++) {
            // Check row
            if (board[row][i] == ch) return false;
            // Check column
            if (board[i][col] == ch) return false;
            // Check 3x3 box
            int boxRow = 3 * (row / 3) + i / 3;
            int boxCol = 3 * (col / 3) + i % 3;
            if (board[boxRow][boxCol] == ch) return false;
        }
        return true;
    }

    bool solve() {
        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                if (board[row][col] == '.') {
                    for (char ch = '1'; ch <= '9'; ch++) {
                        if (isValid(row, col, ch)) {
                            board[row][col] = ch;
                            if (solve()) return true;
                            board[row][col] = '.'; // Backtrack
                        }
                    }
                    return false; // No digit works
                }
            }
        }
        return true; // All cells filled
    }
};

```

```

    }

public:
    void solveSudoku(vector<vector<char>>& b) {
        board = b;
        solve();
        b = board;
    }
};

int main() {
    vector<vector<char>> board = {
        {'5','3','.','.','7','.','.','.','.'},
        {'6','.','.','1','9','5','.','.','.'},
        {'.','9','8','.','.','.','.','6','.'},
        {'8','.','.','.','6','.','.','.','3'},
        {'4','.','.','8','.','3','.','.','1'},
        {'7','.','.','.','2','.','.','.','6'},
        {'.','6','.','.','.','.','.','2','8','.'},
        {'.','.'.','.','4','1','9','.','.','5'},
        {'.','.'.','.','.','8','.','.','7','9'}
    };

    SudokuSolver solver;
    solver.solveSudoku(board);

    cout << "Solved Sudoku:\n";
    for (auto& row : board) {
        for (char c : row) cout << c << " ";
        cout << "\n";
    }
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class SudokuSolver:
    def solveSudoku(self, board: List[List[str]]) -> None:
        self.board = board
        self.solve()

    def solve(self) -> bool:
        for row in range(9):
            for col in range(9):
                if self.board[row][col] == '.':
                    for ch in '123456789':
                        if self.is_valid(row, col, ch):
                            self.board[row][col] = ch
                            if self.solve():
                                return True
                    return False
        return True

    def is_valid(self, row, col, ch):
        # Check row and column
        for i in range(9):
            if i != row and self.board[i][col] == ch:
                return False
            if i != col and self.board[row][i] == ch:
                return False
        # Check 3x3 box
        box_row, box_col = row // 3, col // 3
        for i in range(3):
            for j in range(3):
                if (box_row * 3 + i != row and box_col * 3 + j != col and
                    self.board[box_row * 3 + i][box_col * 3 + j] == ch):
                    return False
        return True

```

```

        return True
        self.board[row][col] = '.' # Backtrack
    return False # No digit works
return True # All cells filled

def is_valid(self, row: int, col: int, ch: str) -> bool:
    for i in range(9):
        # Check row
        if self.board[row][i] == ch:
            return False
        # Check column
        if self.board[i][col] == ch:
            return False
        # Check 3x3 box
        box_row = 3 * (row // 3) + i // 3
        box_col = 3 * (col // 3) + i % 3
        if self.board[box_row][box_col] == ch:
            return False
    return True

# Example usage
solver = SudokuSolver()
board = [
    ["5", "3", ".", ".", "7", ".", ".", ".", "."],
    ["6", ".", ".", "1", "9", "5", ".", ".", "."],
    [".", "9", "8", ".", ".", ".", ".", "6", "."],
    ["8", ".", ".", ".", "6", ".", ".", ".", "3"],
    ["4", ".", ".", "8", ".", "3", ".", ".", "1"],
    ["7", ".", ".", ".", "2", ".", ".", ".", "6"],
    [".", "6", ".", ".", ".", ".", "2", "8", "."],
    [".", ".", ".", "4", "1", "9", ".", ".", "5"],
    [".", ".", ".", ".", "8", ".", ".", "7", "9"]
]
solver.solveSudoku(board)
for row in board:
    print(" ".join(row))

```

## 10. Code Explanation

- **solve()**: Iterates through the board to find the first empty cell. For each empty cell, tries digits 1-9. Uses recursion — if placing a digit leads to a solution, returns true; otherwise, backtracks.
- **isValid()**: Checks if placing **ch** at (**row**, **col**) violates any constraint. The loop from 0 to 8 checks all three constraints simultaneously.
- **Box indexing**:  $3 * (\text{row}/3) + i/3$  gives the row within the 3×3 box, and  $3 * (\text{col}/3) + i\%3$  gives the column.
- **Backtracking**: The `board[row][col] = '.'` restores the state when a digit doesn't lead to a solution.

## 11. Complexity Analysis

| Aspect                     | Complexity                                                      |
|----------------------------|-----------------------------------------------------------------|
| <b>Time (worst-case)</b>   | $O(9^{\text{(empty cells)}})$ — extremely large without pruning |
| <b>Time (with pruning)</b> | Much faster in practice due to constraints                      |
| <b>Space (stack)</b>       | $O(\text{empty cells})$ — up to 81                              |
| <b>Space (board)</b>       | $O(81) = O(1)$ — fixed size                                     |

## 12. Common Patterns

| Pattern                    | Description                         | Example         |
|----------------------------|-------------------------------------|-----------------|
| <b>Standard Sudoku</b>     | 9×9 board, digits 1-9               | LeetCode 37     |
| <b>Sudoku with bitmask</b> | Use bitsets for faster validation   | CP optimization |
| <b>MRV Heuristic</b>       | Fill cell with fewest valid options | Optimization    |
| <b>N×N Sudoku</b>          | Generalized Sudoku                  | Variation       |

## 13. Common Mistakes

- **Incorrect box indexing** —  $3 * (\text{row}/3) + \text{i}/3$  and  $3 * (\text{col}/3) + \text{i}\%3$ .
- **Not checking all three constraints** — must check row, column, and box.
- **Forgetting to backtrack** — must restore the cell to '.'.
- **Returning true prematurely** — only return true when all cells are filled.
- **Using int instead of char** — board is typically char/string.

## 14. Edge Cases

- **Empty board** (all '.') — should find a valid solution.
- **Already solved board** — should return immediately.
- **Invalid initial board** — may return false (no solution).
- **Single empty cell** — should fill it quickly.

## 15. Variations

| Variation                  | What Changes                        | Importance |
|----------------------------|-------------------------------------|------------|
| <b>Sudoku with bitmask</b> | Use bitsets for $O(1)$ validation   | High (CP)  |
| <b>MRV heuristic</b>       | Pick cell with fewest valid options | Medium     |
| <b>N×N Sudoku</b>          | Generalize to any N                 | Low        |
| <b>Sudoku Validator</b>    | Just check if board is valid        | High       |

## 16. Related Algorithms

| Algorithm       | Connection                      |
|-----------------|---------------------------------|
| <b>N-Queens</b> | Similar constraint satisfaction |

| Algorithm                     | Connection                           |
|-------------------------------|--------------------------------------|
| <b>Backtracking</b>           | General framework                    |
| <b>Constraint Propagation</b> | Advanced Sudoku solving              |
| <b>Dancing Links</b>          | Algorithm X for exact cover problems |

## 17. Practice Problems

### Easy

- **Valid Sudoku** — LeetCode 36 — Validate a Sudoku board
- **Sudoku Solver** — LeetCode 37 — Solve a Sudoku puzzle

### Medium

- **Sudoku with bitmask** — CP — Optimized with bit operations
- **Sudoku Generator** — Variation — Create valid puzzles

### Hard

- **N×N Sudoku** — Variation — Generalize to any size
- **Sudoku with constraints** — Variation — Additional constraints

## 18. Interview Explanation

"For solving Sudoku, I use backtracking. I find the first empty cell and try digits 1 through 9. For each digit, I check if it's valid — not already in the same row, column, or 3×3 box. If valid, I place it and recursively solve the rest. If the recursion fails, I undo and try the next digit. The box index is calculated as  $3 * (\text{row}/3) + i/3$  for row and  $3 * (\text{col}/3) + i\%3$  for column. The time complexity is  $O(9^m)$  where  $m$  is the number of empty cells, but constraints make it much faster in practice."

## 19. Revision Notes

- **Find empty cell** → try digits 1-9 → check validity → place → recurse → undo
- **Box indexing:**  $\text{boxRow} = 3 * (\text{row}/3) + i/3$ ,  $\text{boxCol} = 3 * (\text{col}/3) + i\%3$
- **Three constraints:** row, column, box
- **Complexity:**  $O(9^m)$  worst-case, but practically fast
- **Return true** only when all cells filled

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                          |
|------------------------|----------------------------------------------------------------------------------|
| <b>When to use</b>     | Solving Sudoku, grid constraint satisfaction                                     |
| <b>Main operations</b> | Find empty → try digits → validate → place → recurse → undo                      |
| <b>Complexity</b>      | $O(9^m)$ worst-case, $O(m)$ stack                                                |
| <b>Key code</b>        | <code>for ch in '1'..'9': if valid: place; if solve(): return true; undo;</code> |

| Aspect     | Summary                                            |
|------------|----------------------------------------------------|
| Edge cases | Empty board, already solved, invalid initial board |

## 9. Rat in a Maze

### 1. Overview

A rat starts at the top-left corner (0,0) of an  $N \times N$  maze and needs to reach the bottom-right corner (N-1,N-1). The maze has blocked cells (0) and open cells (1). The rat can move in four directions (up, down, left, right). Find all possible paths from source to destination.

### 2. Intuition

#### Simple explanation:

The rat tries to move in one direction. If it hits a wall or goes out of bounds, it tries another direction. If it reaches the destination, it records the path. It marks visited cells to avoid loops.

#### Analogy:

A person in a hedge maze. At each intersection, they try a path. If it's a dead end, they return to the intersection and try another path. They mark visited paths with chalk to avoid going in circles.

#### Step-by-step reasoning:

1. Start at (0,0).
2. Mark the current cell as visited.
3. If the current cell is the destination, record the path.
4. Otherwise, try moving in each direction (D, L, R, U):
  - Check if the next cell is within bounds, not blocked, and not visited.
  - If valid, move there and recurse.
5. After returning from the move, unmark the cell (backtrack).

#### Why it works:

DFS explores all possible paths. The visited array prevents cycles. The path is built incrementally as we move.

### 3. When to Use It

- **Grid path finding** with obstacles.
- **Finding all paths** from source to destination.
- **Trigger phrases:** "rat in a maze", "find all paths", "grid with obstacles", "moving in 4 directions".

### 4. When Not to Use It

- **Shortest path needed** — use BFS or Dijkstra.
- **Only one path needed** — DFS is fine, but BFS is better for shortest.
- **Very large grid** ( $N > 20$ ) — number of paths can be exponential.
- **Weighted grid** — use Dijkstra or A\*.

### 5. Core Concepts

| Concept         | Explanation                          | Why It Matters                |
|-----------------|--------------------------------------|-------------------------------|
| Visited Array   | Tracks visited cells to avoid cycles | Prevents infinite loops       |
| Direction Array | Arrays for row/col deltas (dx, dy)   | Clean way to try moves        |
| Boundary Check  | Ensures next cell is within grid     | Prevents out-of-bounds errors |
| Path String     | Records the sequence of moves        | Required output format        |
| Backtracking    | Unmark visited cell after recursion  | Allows exploring other paths  |

6. Step-by-Step Algorithm

```
Input: maze = [[1,0,0,0],
               [1,1,0,1],
               [1,1,0,0],
               [0,1,1,1]]
```

Step 1: Start at (0,0). Mark visited.  
Step 2: Current = (0,0). Not destination.  
Step 3: Try D (1,0): valid (1,1, not visited). Move to (1,0).  
Step 4: Current = (1,0). Not destination.  
Step 5: Try D (2,0): valid (1,1, not visited). Move to (2,0).  
Step 6: Current = (2,0). Not destination.  
Step 7: Try D (3,0): blocked (0). Skip.  
Step 8: Try L (-1,0): out of bounds. Skip.  
Step 9: Try R (2,1): valid (1,1, not visited). Move to (2,1).  
... continues until destination or dead end  
Step 10: Backtrack to (2,0). Unmark (2,1).  
... continue exploring other paths from (2,0)  
Step 11: Backtrack to (1,0). Unmark (2,0).  
Step 12: Try R (1,1): valid. Move to (1,1).  
... continue

Possible paths: DDRDRR, DRDDRR

7. Dry Run

**Input:** Simple 2×2 maze

```
[1, 1]
[1, 1]
```

| Step | Current | Path | Visited  | Moves Tried                                                                                   | Action |
|------|---------|------|----------|-----------------------------------------------------------------------------------------------|--------|
| 1    | (0,0)   | ""   | {{(0,0)} | —                                                                                             | Start  |
| 2    | (0,0)   | ""   | {{(0,0)} | D → (1,0)  | Move D |



| Step | Current | Path | Visited             | Moves Tried         | Action       |
|------|---------|------|---------------------|---------------------|--------------|
| 3    | (1,0)   | "D"  | {(0,0),(1,0)}       | D → (2,0) ❌ OOB     | Skip         |
|      |         |      |                     | L → (1,-1) ❌ OOB    | Skip         |
|      |         |      |                     | R → (1,1) ✅         | Move R       |
| 4    | (1,1)   | "DR" | {(0,0),(1,0),(1,1)} | Destination!        | ✅ Record     |
| 5    | (1,1)   | "DR" | {(0,0),(1,0)}       | Backtrack           | Unmark (1,1) |
| 6    | (1,0)   | "D"  | {(0,0),(1,0)}       | U → (0,0) ❌ visited | Skip         |
| 7    | (1,0)   | "D"  | {(0,0)}             | Backtrack           | Unmark (1,0) |
| 8    | (0,0)   | ""   | {(0,0)}             | D → tried           | —            |
| 9    | (0,0)   | ""   | {(0,0)}             | L → (-1,0) ❌ OOB    | Skip         |
| 10   | (0,0)   | ""   | {(0,0)}             | R → (0,1) ✅         | Move R       |
| 11   | (0,1)   | "R"  | {(0,0),(0,1)}       | D → (1,1) ✅         | Move D       |
| 12   | (1,1)   | "RD" | {(0,0),(0,1),(1,1)} | Destination!        | ✅ Record     |

Paths found: "DR", "RD"

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class RatInMaze {
    vector<string> result;
    vector<vector<int>>> maze;
    vector<vector<bool>>> visited;
    int n;

    // Direction arrays: D, L, R, U
    int dx[4] = {1, 0, 0, -1};
    int dy[4] = {0, -1, 1, 0};
    char dir[4] = {'D', 'L', 'R', 'U'};

    bool isValid(int x, int y) {
        return x >= 0 && x < n && y >= 0 && y < n
            && maze[x][y] == 1 && !visited[x][y];
    }

    void solve(int x, int y, string path) {
        // Destination reached
        if (x == n - 1 && y == n - 1) {
            result.push_back(path);
            return;
        }
    }
}
```

```

        // Try all four directions
        for (int i = 0; i < 4; i++) {
            int nx = x + dx[i];
            int ny = y + dy[i];

            if (isValid(nx, ny)) {
                visited[nx][ny] = true;
                solve(nx, ny, path + dir[i]);
                visited[nx][ny] = false; // Backtrack
            }
        }
    }

public:
    vector<string> findPath(vector<vector<int>>& m) {
        result.clear();
        maze = m;
        n = m.size();

        if (n == 0 || m[0][0] == 0 || m[n-1][n-1] == 0) {
            return result; // No path possible
        }

        visited.assign(n, vector<bool>(n, false));
        visited[0][0] = true;
        solve(0, 0, "");
        sort(result.begin(), result.end()); // Lexicographic order
        return result;
    }
};

int main() {
    vector<vector<int>> maze = {
        {1, 0, 0, 0},
        {1, 1, 0, 1},
        {1, 1, 0, 0},
        {0, 1, 1, 1}
    };

    RatInMaze solver;
    auto paths = solver.findPath(maze);

    cout << "Paths found: ";
    for (string& p : paths) cout << p << " ";
    cout << "\n";
    // Output: DDRDRR DRDDRR
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class RatInMaze:
    def __init__(self):
        self.result = []
        self.dx = [1, 0, 0, -1] # D, L, R, U
        self.dy = [0, -1, 1, 0]
        self.dir = ['D', 'L', 'R', 'U']

    def findPath(self, maze: List[List[int]]) -> List[str]:
        self.result = []
        n = len(maze)

        if n == 0 or maze[0][0] == 0 or maze[n-1][n-1] == 0:
            return self.result

        visited = [[False] * n for _ in range(n)]
        visited[0][0] = True

        self.solve(maze, visited, 0, 0, "", n)
        self.result.sort() # Lexicographic order
        return self.result

    def solve(self, maze, visited, x, y, path, n):
        # Destination reached
        if x == n - 1 and y == n - 1:
            self.result.append(path)
            return

        # Try all four directions
        for i in range(4):
            nx = x + self.dx[i]
            ny = y + self.dy[i]

            if self.is_valid(maze, visited, nx, ny, n):
                visited[nx][ny] = True
                self.solve(maze, visited, nx, ny, path + self.dir[i], n)
                visited[nx][ny] = False # Backtrack

    def is_valid(self, maze, visited, x, y, n):
        return (0 <= x < n and 0 <= y < n and
                maze[x][y] == 1 and not visited[x][y])

# Example usage
solver = RatInMaze()
maze = [
    [1, 0, 0, 0],
    [1, 1, 0, 1],
    [1, 1, 0, 0],
    [0, 1, 1, 1]
]

```

```
paths = solver.findPath(maze)
print("Paths found:", paths) # ['DDRDRR', 'DRDDRR']
```

## 10. Code Explanation

- **dx/dy arrays:** Store row and column deltas for each direction. Direction order (D, L, R, U) ensures lexicographic order.
- **isValid():** Checks bounds, open cell, and not visited.
- **solve():** Recursive function. If destination reached, add path. Otherwise, try all 4 directions.
- **Visited array:** `visited[nx][ny] = true` before recursion, `false` after (backtracking).
- **Edge case:** If start or end is blocked, return empty immediately.

## 11. Complexity Analysis

| Aspect                     | Complexity                             |
|----------------------------|----------------------------------------|
| <b>Time (worst-case)</b>   | $O(4^N)$ — each cell has 4 choices     |
| <b>Time (with pruning)</b> | Much less in practice due to obstacles |
| <b>Space (stack)</b>       | $O(N)$ — recursion depth = path length |
| <b>Space (visited)</b>     | $O(N^2)$                               |
| <b>Space (result)</b>      | $O(N^2 \times \text{number of paths})$ |

## 12. Common Patterns

| Pattern                         | Description                       | Example   |
|---------------------------------|-----------------------------------|-----------|
| <b>Standard Rat in a Maze</b>   | All paths from (0,0) to (N-1,N-1) | GFG       |
| <b>Shortest Path in Maze</b>    | BFS for shortest path             | LeetCode  |
| <b>Maze with Multiple Moves</b> | Knight moves, diagonal moves      | Variation |
| <b>Maze with Jumps</b>          | Can jump multiple cells           | Variation |

## 13. Common Mistakes

- **Not checking if start/end is blocked** — should return empty immediately.
- **Wrong direction order** — lexicographic order required (D, L, R, U).
- **Forgetting to backtrack** — visited cell not unmarked.
- **Not sorting result** — many problems expect sorted output.
- **Off-by-one in bounds check** — use `>=` and `<` correctly.

## 14. Edge Cases

- **1x1 grid** → `[""]` (empty path, already at destination).
- **Start or end blocked** → `[]` (no path).
- **No path exists** → `[]`.
- **All cells open** → many paths, memory-heavy.

- **Large grid** ( $N > 10$ ) → exponential paths, may overflow.

## 15. Variations

| Variation                         | What Changes          | Importance |
|-----------------------------------|-----------------------|------------|
| <b>Shortest Path</b>              | BFS instead of DFS    | High       |
| <b>Maze with Obstacles</b>        | Some cells blocked    | High       |
| <b>Maze with Keys/Doors</b>       | Need to collect keys  | Medium     |
| <b>Maze with Multiple Sources</b> | Multiple start points | Low        |
| <b>Maze with Jumps</b>            | Can jump 1-2 cells    | Medium     |

## 16. Related Algorithms

| Algorithm           | Connection               |
|---------------------|--------------------------|
| <b>DFS</b>          | Rat in a Maze uses DFS   |
| <b>BFS</b>          | Better for shortest path |
| <b>Backtracking</b> | General framework        |
| <b>Flood Fill</b>   | Similar grid traversal   |

## 17. Practice Problems

### Easy

- **Rat in a Maze** — GFG — Standard problem
- **Flood Fill** — LeetCode 733 — Similar grid traversal

### Medium

- **Shortest Path in Binary Matrix** — LeetCode 1091 — BFS variant
- **Unique Paths III** — LeetCode 980 — Paths covering all cells
- **Maze with Jumps** — Variation — Multiple step sizes

### Hard

- **Maze with Keys and Doors** — LeetCode 864 — Collect keys to open doors
- **The Maze III** — LeetCode 499 — Ball rolling in maze

## 18. Interview Explanation

"Rat in a Maze is a classic backtracking problem. I use DFS from the start cell, marking visited cells to avoid cycles. At each cell, I try all four directions in lexicographic order (D, L, R, U). When I reach the destination, I record the path. The key is backtracking — I unmark a cell after returning from a recursive call so other paths can use it. The time complexity is  $O(4^N)$  worst-case, but obstacles provide significant pruning. I also handle edge cases like blocked start or end cells."

## 19. Revision Notes

- **DFS from (0,0) to (N-1,N-1)**
- **4 directions:** D, L, R, U (lexicographic order)
- **Visited array** prevents cycles; **backtrack** to unmark
- **Edge cases:** blocked start/end → return empty
- **Complexity:**  $O(4^N)$  worst-case
- **String path** records moves

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                          |
|------------------------|------------------------------------------------------------------|
| <b>When to use</b>     | Grid path finding, all paths from source to destination          |
| <b>Main operations</b> | Move in 4 directions → mark visited → recurse → unmark           |
| <b>Complexity</b>      | $O(4^N)$ time, $O(N^2)$ space                                    |
| <b>Key code</b>        | <code>dx={1,0,0,-1}, dy={0,-1,1,0}, dir={'D','L','R','U'}</code> |
| <b>Edge cases</b>      | 1×1 grid, blocked start/end, no path exists                      |

## 10. Generate Parentheses

### 1. Overview

Given `n` pairs of parentheses, generate **all valid** combinations of parentheses strings. A valid string has matching opening and closing brackets in the correct order.

### 2. Intuition

#### Simple explanation:

We have `n` opening brackets and `n` closing brackets to place. We can place an opening bracket if we still have any left. We can place a closing bracket only if it wouldn't create an imbalance — the number of closing brackets placed must never exceed the number of opening brackets.

#### Analogy:

Think of a stack. Every opening bracket pushes onto the stack, every closing bracket pops. A valid string never tries to pop from an empty stack, and the stack is empty at the end.

#### Step-by-step reasoning:

1. Keep track of `open` (used so far) and `close` (used so far).
2. If `open < n`, we can add '('.
3. If `close < open`, we can add ').
4. When `open == close == n`, we have a valid string.

#### Why it works:

The constraints `open <= n` and `close <= open` ensure we never generate an invalid string. This is a classic backtracking problem where the branching is constrained by the current state.

3. When to Use It

- **Generating balanced parentheses** strings.
- **Generating valid bracket sequences** of any type.
- **Trigger phrases:** "generate parentheses", "valid parentheses", "balanced brackets", "all combinations of parentheses".

4. When Not to Use It

- **Only need count** — Catalan number  $C_n = \frac{(2n)!}{n!(n+1)!}$ .
- **Validating a parentheses string** — use stack counter.
- **Only need longest valid parentheses** — use stack/DP.
- **n > 15** — number of strings = Catalan number, grows as  $\sim 4^n/(n\sqrt{n})$ .

5. Core Concepts

| Concept             | Explanation                                                   | Why It Matters                  |
|---------------------|---------------------------------------------------------------|---------------------------------|
| Catalan Number      | Number of valid combinations = $C_n = \frac{(2n)!}{n!(n+1)!}$ | Grows as $\sim 4^n/(n\sqrt{n})$ |
| Open count          | Number of '(' placed so far                                   | Must not exceed n               |
| Close count         | Number of ')' placed so far                                   | Must not exceed open count      |
| Balance             | open - close at any point                                     | Must never be negative          |
| Lexicographic Order | All '(' before ')' for same prefix                            | Natural output order            |

6. Step-by-Step Algorithm

```
Input: n = 3

Step 1: open=0, close=0, current=""
Step 2: open < n → add '(' → "(", open=1
Step 3: open < n → add '(' → "(", open=2
Step 4: open < n → add '(' → "(((", open=3
Step 5: open == n, close < open → add ')' → "((((", close=1
Step 6: close < open → add ')' → "(((())", close=2
Step 7: close < open → add ')' → "((()))", close=3 → VALID
Step 8: Backtrack, try other branches...

All valid strings for n=3:
((())) ((()()) ((())()) ()(()) ()()()
```

7. Dry Run

Input: n = 2

| open | close | current | Condition         | Action    | Result  |
|------|-------|---------|-------------------|-----------|---------|
| 0    | 0     | ""      | open < 2          | Add '('   | "( "    |
| 1    | 0     | "("     | open < 2          | Add '('   | "(( "   |
| 2    | 0     | "(("    | close < 2         | Add ')'   | "(() "  |
| 2    | 1     | "(())"  | close < 2         | Add ')'   | "(())"  |
| 2    | 2     | "(())"  | open==close==2    | ✔ Add     | —       |
| —    | —     | —       | Backtrack         | Pop ')'   | "(())"  |
| 2    | 1     | "(())"  | open==2, close==1 | Backtrack | Pop ')' |
| 2    | 0     | "((("   | open==2, close==0 | Backtrack | Pop '(' |
| 1    | 0     | "(("    | close < open      | Add ')'   | "()"    |
| 1    | 1     | "()"    | open < 2          | Add '('   | "()("   |
| 2    | 1     | "()("   | close < 2         | Add ')'   | "()()"  |
| 2    | 2     | "()()"  | open==close==2    | ✔ Add     | —       |

Result: ["(())", "()()"]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class GenerateParentheses {
    vector<string> result;

    void backtrack(string current, int open, int close, int n) {
        // Base case: used all n pairs
        if (open == n && close == n) {
            result.push_back(current);
            return;
        }

        // Can add opening bracket
        if (open < n) {
            backtrack(current + '(', open + 1, close, n);
        }

        // Can add closing bracket (only if it won't create imbalance)
        if (close < open) {
            backtrack(current + ')', open, close + 1, n);
        }
    }

public:
```



```

    vector<string> generate(int n) {
        result.clear();
        backtrack("", 0, 0, n);
        return result;
    }
};

// Iterative approach using stack
vector<string> generateParenthesesIterative(int n) {
    vector<string> result;
    stack<tuple<string, int, int>> st; // (current, open, close)
    st.push("", 0, 0);

    while (!st.empty()) {
        auto [curr, open, close] = st.top();
        st.pop();

        if (open == n && close == n) {
            result.push_back(curr);
            continue;
        }

        if (close < open) {
            st.push({curr + '}', open, close + 1});
        }
        if (open < n) {
            st.push({curr + '(', open + 1, close});
        }
    }

    return result;
}

int main() {
    GenerateParentheses solver;
    auto res = solver.generate(3);

    cout << "Total for n=3: " << res.size() << "\n";
    for (string& s : res) {
        cout << s << "\n";
    }
    // Output: ((( ))) (( ) ) ( ) ( ) ( ) ( ) ( ) ( )
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class GenerateParentheses:
    def __init__(self):

```

```

        self.result = []

    def generate(self, n: int) -> List[str]:
        self.result = []
        self.backtrack("", 0, 0, n)
        return self.result

    def backtrack(self, current: str, open: int, close: int, n: int):
        # Base case: used all n pairs
        if open == n and close == n:
            self.result.append(current)
            return

        # Can add opening bracket
        if open < n:
            self.backtrack(current + '(', open + 1, close, n)

        # Can add closing bracket (only if it won't create imbalance)
        if close < open:
            self.backtrack(current + ')', open, close + 1, n)

# Example usage
solver = GenerateParentheses()
print(solver.generate(3))
# ['((()))', '(()())', '()()()', '()(())', '()()()']

```

## 10. Code Explanation

- **backtrack(current, open, close, n)**: Builds the string incrementally.
- **open**: Number of '(' placed so far.
- **close**: Number of ')' placed so far.
- **Base case**: `open == n && close == n` — all parentheses used.
- **Add '('**: Only if `open < n` — we still have '(' to place.
- **Add ')'**: Only if `close < open` — ensures we never have more ')' than '('.
- **No explicit undo**: The string is passed by value (copy), so backtracking is automatic.

## 11. Complexity Analysis

| Aspect                | Complexity                                                  |
|-----------------------|-------------------------------------------------------------|
| <b>Time</b>           | $O(4^n/\sqrt{n})$ — Catalan number $C_n = (2n)!/(n!(n+1)!)$ |
| <b>Space (stack)</b>  | $O(n)$ — recursion depth = $2n$                             |
| <b>Space (result)</b> | $O(n \times C_n)$ — storing all strings                     |

## 12. Common Patterns

| Pattern                     | Description                | Example     |
|-----------------------------|----------------------------|-------------|
| <b>Standard Parentheses</b> | Generate all valid strings | LeetCode 22 |

| Pattern                        | Description               | Example     |
|--------------------------------|---------------------------|-------------|
| <b>Different Bracket Types</b> | {, [], ()                 | Variation   |
| <b>Parentheses + Words</b>     | Add words to parentheses  | Variation   |
| <b>Valid Parentheses Check</b> | Verify if string is valid | LeetCode 20 |

### 13. Common Mistakes

- **Wrong condition for ')'** — must check `close < open`, not `close < n`.
- **Forgetting base case** — infinite recursion.
- **Using string reference** — need to backtrack by popping.
- **Off-by-one** — `open == n && close == n` not `open == n-1`.
- **Not recognizing Catalan numbers** — count formula is useful.

### 14. Edge Cases

- **n = 0** → `[]` (empty string is valid).
- **n = 1** → `[]`.
- **n = 8** → 1430 strings (manageable).
- **n = 15** → 9,694,845 strings (too many).

### 15. Variations

| Variation                        | What Changes                       | Importance |
|----------------------------------|------------------------------------|------------|
| <b>Different bracket types</b>   | Multiple bracket types, must match | High       |
| <b>Parentheses with words</b>    | Insert words between brackets      | Medium     |
| <b>Score of Parentheses</b>      | Compute score based on nesting     | Medium     |
| <b>Longest Valid Parentheses</b> | Find longest valid substring       | High       |

### 16. Related Algorithms

| Algorithm              | Connection                          |
|------------------------|-------------------------------------|
| <b>Catalan Numbers</b> | Count formula for valid parentheses |
| <b>Backtracking</b>    | General framework                   |
| <b>Stack</b>           | Validating parentheses uses stack   |
| <b>DP</b>              | Longest valid parentheses uses DP   |

### 17. Practice Problems

#### Easy

- **Generate Parentheses** — LeetCode 22 — Standard problem
- **Valid Parentheses** — LeetCode 20 — Validate a string

## Medium

- **Different Ways to Add Parentheses** — LeetCode 241 — Parentheses with operators
- **Remove Invalid Parentheses** — LeetCode 301 — BFS to remove minimum
- **Score of Parentheses** — LeetCode 856 — Compute score

## Hard

- **Longest Valid Parentheses** — LeetCode 32 — DP with stack
- **Parentheses with constraints** — Variation

## 18. Interview Explanation

"For generating all valid parentheses strings, I use backtracking with two counters — open and close. I can add '(' as long as  $open < n$ , and I can add ')' only if  $close < open$ , which ensures the string stays balanced. When both counters reach  $n$ , I have a valid string. The number of strings is the Catalan number  $C_n = \frac{(2n)!}{(n!(n+1)!)}$ , which grows as  $\sim 4^n / (n\sqrt{n})$ . The time complexity is  $O(4^n / \sqrt{n})$ , and the space complexity is  $O(n)$  for the recursion stack."

## 19. Revision Notes

- **Catalan number:**  $C_n = \frac{(2n)!}{(n!(n+1)!)}$
- **Two constraints:**  $open < n$  for '(',  $close < open$  for ')'
- **Base case:**  $open == n \ \&\& \ close == n$
- $n=0 \rightarrow [""], n=1 \rightarrow ["()"], n=3 \rightarrow 5 \text{ strings}$
- **Complexity:**  $O(4^n / \sqrt{n})$  time,  $O(n)$  stack

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                                                              |
|------------------------|----------------------------------------------------------------------------------------------------------------------|
| <b>When to use</b>     | Generate valid bracket sequences                                                                                     |
| <b>Main operations</b> | Add '(' if $open < n$ ; Add ')' if $close < open$                                                                    |
| <b>Complexity</b>      | $O(4^n / \sqrt{n})$ time, $O(n)$ stack                                                                               |
| <b>Key code</b>        | <pre>if (open &lt; n) backtrack(curr+'(',open+1,close); if (close &lt; open) backtrack(curr+')',open,close+1);</pre> |
| <b>Edge cases</b>      | $n=0 \rightarrow [""], n=1 \rightarrow ["()"]$                                                                       |

## 11. Word Search

### 1. Overview

Given an  $m \times n$  grid of characters and a string **word**, determine if the word exists in the grid. The word can be formed by sequentially adjacent cells (horizontally or vertically). The same cell cannot be used more than once.

### 2. Intuition

**Simple explanation:**

Start at every cell that matches the first character of the word. From there, try to find the next character in an adjacent cell (up, down, left, right). Continue until either the entire word is found or no path works.

**Analogy:**

Like a treasure hunt where each clue leads to the next location. You can only move to adjacent cells, and you can't revisit a cell you've already been to.

**Step-by-step reasoning:**

1. For each cell in the grid that matches the first character of the word:
  - Mark the cell as visited.
  - Try to find the next character in adjacent cells.
  - If the recursive search finds the word, return true.
  - Unmark the cell (backtrack) and try the next starting cell.
2. If no starting cell leads to a solution, return false.

**Why it works:**

DFS with backtracking explores all possible paths in the grid. The visited array prevents reusing cells. The search is pruned when a character doesn't match.

**3. When to Use It**

- **Finding a word** in a character grid.
- **Path finding** with character matching.
- **Trigger phrases:** "word search", "word exists in grid", "boggle", "character grid", "adjacent cells".

**4. When Not to Use It**

- **Multiple words** — use Trie (Word Search II).
- **Longest word** — try all words, no need for backtracking per word.
- **Very large grid** (1000×1000) and many queries — use Trie.
- **Only need count of occurrences** — use hash map of positions.

**5. Core Concepts**

| Concept               | Explanation                 | Why It Matters          |
|-----------------------|-----------------------------|-------------------------|
| <b>Adjacent cells</b> | Up, down, left, right       | Movement directions     |
| <b>Visited array</b>  | Tracks used cells           | Prevents reuse          |
| <b>DFS</b>            | Depth-first search for path | Explores one path fully |
| <b>Backtracking</b>   | Unmark cell after search    | Allows other paths      |

**6. Step-by-Step Algorithm**

```
Input: board = [['A','B','C','E'],
                ['S','F','C','S'],
                ['A','D','E','E']]
```

```
word = "ABCCED"
```

Step 1: Find starting cells matching 'A':

- (0,0): matches 'A'

Step 2: From (0,0), search for 'B':

- Up: OOB
- Down: (1,0) = 'S' ≠ 'B'
- Left: OOB
- Right: (0,1) = 'B' → matches

Step 3: From (0,1), search for 'C':

- Up: OOB
- Down: (1,1) = 'F' ≠ 'C'
- Left: (0,0) visited
- Right: (0,2) = 'C' → matches

Step 4: From (0,2), search for 'C':

- Up: OOB
- Down: (1,2) = 'C' → matches

Step 5: From (1,2), search for 'E':

- Up: (0,2) visited
- Down: (2,2) = 'E' → matches

Step 6: From (2,2), search for 'D':

- Up: (1,2) visited
- Down: OOB
- Left: (2,1) = 'D' → matches

Step 7: All characters found! Return true.

Result: true

## 7. Dry Run

**Input:** `board = [['A', 'B'], ['C', 'D']], word = "AC"`

| Step | Position | Word Index | Board Value | Char Match | Action       |
|------|----------|------------|-------------|------------|--------------|
| 1    | (0,0)    | 0          | 'A'         | 'A'        | Start search |
| 2    | (0,0)    | 1          | visited     | —          | Need 'C'     |
| 3    | (1,0)    | 1          | 'C'         | 'C'        | Found word   |
| —    | —        | —          | —           | —          | Return true  |

**But wait:** Try from (0,0) → (1,0) = 'C' → word[1]='C' → word found!

Another test: `word = "AB"`

| Step | Position | Word Index | Board Value | Char Match | Action       |
|------|----------|------------|-------------|------------|--------------|
| 1    | (0,0)    | 0          | 'A'         | 'A'        | Start search |
| 2    | (0,0)    | 1          | visited     | —          | Need 'B'     |
| 3    | (0,1)    | 1          | 'B'         | 'B'        | Found word   |

| Step | Position | Word Index | Board Value | Char Match | Action      |
|------|----------|------------|-------------|------------|-------------|
| —    | —        | —          | —           | —          | Return true |

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class WordSearch {
    vector<vector<char>>> board;
    string word;
    int m, n;

    // Direction vectors: up, down, left, right
    int dx[4] = {-1, 1, 0, 0};
    int dy[4] = {0, 0, -1, 1};

    bool dfs(int x, int y, int idx) {
        // All characters matched
        if (idx == word.size()) return true;

        // Bounds check and character match
        if (x < 0 || x >= m || y < 0 || y >= n) return false;
        if (board[x][y] != word[idx]) return false;

        // Save and mark visited
        char temp = board[x][y];
        board[x][y] = '#'; // Mark as visited

        // Try all 4 directions
        for (int i = 0; i < 4; i++) {
            if (dfs(x + dx[i], y + dy[i], idx + 1)) {
                board[x][y] = temp; // Restore before returning
                return true;
            }
        }

        // Restore and backtrack
        board[x][y] = temp;
        return false;
    }

public:
    bool exist(vector<vector<char>>& b, string w) {
        board = b;
        word = w;
        m = board.size();
        n = board[0].size();

        // Optimization: check character frequency
        if (word.size() > m * n) return false;
```

```

        // Try each cell as starting point
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (board[i][j] == word[0] && dfs(i, j, 0)) {
                    return true;
                }
            }
        }
        return false;
    }
};

int main() {
    vector<vector<char>> board = {
        {'A','B','C','E'},
        {'S','F','C','S'},
        {'A','D','E','E'}
    };

    WordSearch solver;
    cout << "ABCCED: " << solver.exist(board, "ABCCED") << "\n"; // 1
    cout << "SEE: " << solver.exist(board, "SEE") << "\n";      // 1
    cout << "ABCB: " << solver.exist(board, "ABCB") << "\n";    // 0
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class WordSearch:
    def exist(self, board: List[List[str]], word: str) -> bool:
        self.board = board
        self.word = word
        self.m = len(board)
        self.n = len(board[0])

        # Optimization: check character frequency
        if len(word) > self.m * self.n:
            return False

        # Try each cell as starting point
        for i in range(self.m):
            for j in range(self.n):
                if board[i][j] == word[0] and self.dfs(i, j, 0):
                    return True
        return False

    def dfs(self, x: int, y: int, idx: int) -> bool:
        # All characters matched

```



```

    if idx == len(self.word):
        return True

    # Bounds check and character match
    if (x < 0 or x >= self.m or y < 0 or y >= self.n or
        self.board[x][y] != self.word[idx]):
        return False

    # Save and mark visited
    temp = self.board[x][y]
    self.board[x][y] = '#'

    # Try all 4 directions
    for dx, dy in [(-1,0), (1,0), (0,-1), (0,1)]:
        if self.dfs(x + dx, y + dy, idx + 1):
            self.board[x][y] = temp # Restore before returning
            return True

    # Restore and backtrack
    self.board[x][y] = temp
    return False

# Example usage
solver = WordSearch()
board = [
    ['A', 'B', 'C', 'E'],
    ['S', 'F', 'C', 'S'],
    ['A', 'D', 'E', 'E']
]
print(solver.exist(board, "ABCCED")) # True
print(solver.exist(board, "SEE"))    # True
print(solver.exist(board, "ABCB"))   # False

```

## 10. Code Explanation

- **dfs(x, y, idx):** Tries to match `word[idx]` at position `(x, y)`.
- **Base case:** `idx == word.size()` — all characters matched.
- **Pruning:** Returns false if out of bounds or character doesn't match.
- **Visited marking:** Temporarily change the board cell to '#' (or any marker).
- **Direction loop:** Try all 4 adjacent cells for the next character.
- **Restore:** Change the cell back to its original character after search.
- **Optimization:** Check if the word can fit in the board at all (`word.size() > m*n`).

## 11. Complexity Analysis

| Aspect              | Complexity                                      |
|---------------------|-------------------------------------------------|
| Time (worst-case)   | $O(m \times n \times 4^{(\text{word.length})})$ |
| Time (with pruning) | Much less in practice                           |
| Space (stack)       | $O(\text{word.length})$ — recursion depth       |

| Aspect                            | Complexity               |
|-----------------------------------|--------------------------|
| <b>Space (board modification)</b> | $O(1)$ — modify in-place |

## 12. Common Patterns

| Pattern                           | Description                          | Example      |
|-----------------------------------|--------------------------------------|--------------|
| <b>Single Word Search</b>         | Find one word in grid                | LeetCode 79  |
| <b>Word Search II</b>             | Find multiple words using Trie       | LeetCode 212 |
| <b>Boggle</b>                     | Board game with word finding         | Variation    |
| <b>Longest Word in Dictionary</b> | Find longest word that can be formed | Variation    |

## 13. Common Mistakes

- **Not marking visited** → infinite loops (reusing same cell).
- **Not restoring the board** → state corruption for other searches.
- **Wrong direction order** — doesn't matter for correctness, but matters for optimization.
- **Early return without restore** — must restore before returning true.
- **Not checking bounds** before accessing board.

## 14. Edge Cases

- **Empty board** → false.
- **Empty word** → true (empty string is always found).
- **Word longer than total cells** → false (can't fit).
- **Single character word** → check if character exists in board.
- **Characters not in board** → false.

## 15. Variations

| Variation                 | What Changes             | Importance        |
|---------------------------|--------------------------|-------------------|
| <b>Word Search II</b>     | Multiple words, use Trie | High              |
| <b>Boggle</b>             | With score calculation   | Medium            |
| <b>8-directional</b>      | Include diagonals        | Medium            |
| <b>With visited array</b> | Use separate 2D array    | Low (extra space) |

## 16. Related Algorithms

| Algorithm           | Connection                    |
|---------------------|-------------------------------|
| <b>DFS</b>          | Core search algorithm         |
| <b>Backtracking</b> | Unmarking visited cells       |
| <b>Trie</b>         | Used for multiple word search |

| Algorithm         | Connection             |
|-------------------|------------------------|
| <b>Flood Fill</b> | Similar grid traversal |

## 17. Practice Problems

### Easy

- **Word Search** — LeetCode 79 — Single word search
- **Find Words** — GFG — Basic word search

### Medium

- **Word Search II** — LeetCode 212 — Multiple words with Trie
- **Boggle** — GFG — Board game variation
- **Construct String from Binary Tree** — LeetCode 606 — Different tree traversal

### Hard

- **Word Search with pruning** — CP — Optimized with frequency checks
- **Longest Word in Dictionary through Deleting** — LeetCode 524 — String comparison

## 18. Interview Explanation

"Word Search is a classic DFS backtracking problem. I start from each cell that matches the first character, then recursively check adjacent cells for the next character. I mark visited cells by modifying the board in-place to avoid cycles. If at any point the character doesn't match or we go out of bounds, I backtrack. The time complexity is  $O(m \times n \times 4^L)$  where  $L$  is the word length. A key optimization is checking if the word length exceeds the total number of cells. For multiple words, I'd use a Trie."

## 19. Revision Notes

- **DFS + Backtracking** on grid
- **Mark visited** by modifying board (save and restore)
- **4 directions**: up, down, left, right
- **Pruning**: bounds check, character match, word length > total cells
- **Complexity**:  $O(m \times n \times 4^L)$
- **Always restore** the board before returning

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                             |
|------------------------|-------------------------------------------------------------------------------------|
| <b>When to use</b>     | Finding word in character grid                                                      |
| <b>Main operations</b> | Start at each cell → DFS → match char → recurse → backtrack                         |
| <b>Complexity</b>      | $O(m \times n \times 4^L)$ time, $O(L)$ stack                                       |
| <b>Key code</b>        | <pre>board[x][y]='#'; for each direction: dfs(nx,ny,idx+1); board[x][y]=temp;</pre> |

| Aspect     | Summary                                               |
|------------|-------------------------------------------------------|
| Edge cases | Empty board, empty word, word longer than total cells |

## 12. Palindrome Partitioning

### 1. Overview

Given a string `s`, partition it such that every substring of the partition is a palindrome. Return **all possible** palindrome partitions of `s`.

### 2. Intuition

#### Simple explanation:

At each position in the string, try to cut off a prefix that is a palindrome. Then recursively partition the remaining suffix. Continue until the entire string is consumed.

#### Analogy:

Like cutting a piece of string into segments where each segment must be a symmetric pattern (reads same forwards and backwards). You try all possible first cuts that produce a symmetric piece, then recursively cut the rest.

#### Step-by-step reasoning:

1. Start at index 0.
2. For each possible end index `j` from `i` to `n-1`:
  - Check if `s[i..j]` is a palindrome.
  - If yes, add it to the current partition and recurse from `j+1`.
3. When `i == n`, we've partitioned the whole string.

#### Why it works:

The problem has optimal substructure — a palindrome partition of the whole string consists of a palindrome prefix plus a palindrome partition of the suffix. Backtracking explores all possible first cuts.

### 3. When to Use It

- **Partitioning a string** into palindromic substrings.
- **Generating all palindromic decompositions.**
- **Trigger phrases:** "palindrome partitioning", "partition into palindromes", "all palindromic substrings", "split string into palindromes".

### 4. When Not to Use It

- **Only need minimum cuts** — use DP (LeetCode 132).
- **Only need all palindromic substrings** — use DP or expand around center.
- **String length > 20** — number of partitions can be huge.
- **Only need count** — use DP.

### 5. Core Concepts

| Concept      | Explanation                                      | Why It Matters         |
|--------------|--------------------------------------------------|------------------------|
| Palindrome   | String reads same forwards and backwards         | Core constraint        |
| Prefix check | Check if s[i..j] is palindrome                   | Pruning condition      |
| Partition    | List of substrings that form the original string | Result format          |
| Backtracking | Try all possible first cuts                      | Explore all partitions |

6. Step-by-Step Algorithm

```
Input: s = "aab"

Step 1: Start at i=0
Step 2: Try j=0: "a" is palindrome → current=["a"], recurse from i=1
Step 3: At i=1, try j=1: "a" is palindrome → current=["a","a"], recurse from i=2
Step 4: At i=2, try j=2: "b" is palindrome → current=["a","a","b"] → VALID
Step 5: Backtrack, no more j
Step 6: At i=1, try j=2: "ab" is NOT palindrome → skip
Step 7: At i=0, try j=1: "aa" is palindrome → current=["aa"], recurse from i=2
Step 8: At i=2, try j=2: "b" is palindrome → current=["aa","b"] → VALID
Step 9: At i=0, try j=2: "aab" is NOT palindrome → skip

Result: [["a","a","b"], ["aa","b"]]
```

7. Dry Run

Input: s = "aab"

| i | j | Substring | Palindrome? | current       | Action      |
|---|---|-----------|-------------|---------------|-------------|
| 0 | 0 | "a"       | ✓           | ["a"]         | Recurse i=1 |
| 1 | 1 | "a"       | ✓           | ["a","a"]     | Recurse i=2 |
| 2 | 2 | "b"       | ✓           | ["a","a","b"] | ✓ Add       |
| 1 | 2 | "ab"      | ✗           | ["a"]         | Skip        |
| 0 | 1 | "aa"      | ✓           | ["aa"]        | Recurse i=2 |
| 2 | 2 | "b"       | ✓           | ["aa","b"]    | ✓ Add       |
| 0 | 2 | "aab"     | ✗           | []            | Skip        |

Result: [["a","a","b"], ["aa","b"]]

8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;
```

```

class PalindromePartitioning {
    vector<vector<string>> result;
    vector<string> current;
    string s;
    int n;

    bool isPalindrome(int l, int r) {
        while (l < r) {
            if (s[l] != s[r]) return false;
            l++; r--;
        }
        return true;
    }

    void backtrack(int start) {
        // Reached end of string
        if (start == n) {
            result.push_back(current);
            return;
        }

        for (int end = start; end < n; end++) {
            if (isPalindrome(start, end)) {
                // Add palindrome substring to current partition
                current.push_back(s.substr(start, end - start + 1));
                backtrack(end + 1);
                current.pop_back(); // Backtrack
            }
        }
    }

public:
    vector<vector<string>> partition(string str) {
        result.clear();
        current.clear();
        s = str;
        n = s.size();
        backtrack(0);
        return result;
    }
};

// Optimized version with DP precomputation of palindromes
class PalindromePartitioningOptimized {
    vector<vector<string>> result;
    vector<string> current;
    vector<vector<bool>> dp; // dp[i][j] = is s[i..j] palindrome?
    string s;
    int n;

    void backtrack(int start) {
        if (start == n) {
            result.push_back(current);
        }
    }
};

```

```

        return;
    }

    for (int end = start; end < n; end++) {
        if (dp[start][end]) {
            current.push_back(s.substr(start, end - start + 1));
            backtrack(end + 1);
            current.pop_back();
        }
    }
}

public:
vector<vector<string>> partition(string str) {
    result.clear();
    current.clear();
    s = str;
    n = s.size();

    // Precompute palindromes
    dp.assign(n, vector<bool>(n, false));
    for (int len = 1; len <= n; len++) {
        for (int i = 0; i + len - 1 < n; i++) {
            int j = i + len - 1;
            if (len == 1) dp[i][j] = true;
            else if (len == 2) dp[i][j] = (s[i] == s[j]);
            else dp[i][j] = (s[i] == s[j] && dp[i+1][j-1]);
        }
    }

    backtrack(0);
    return result;
}

};

int main() {
    PalindromePartitioning solver;
    auto res = solver.partition("aab");

    cout << "Partitions:\n";
    for (auto& partition : res) {
        cout << "[";
        for (string& p : partition) cout << p << " ";
        cout << "]\n";
    }
    // Output: [a a b ] [aa b ]
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class PalindromePartitioning:
    def __init__(self):
        self.result = []
        self.current = []

    def partition(self, s: str) -> List[List[str]]:
        self.result = []
        self.current = []
        self.s = s
        self.n = len(s)
        self.backtrack(0)
        return self.result

    def backtrack(self, start: int):
        # Reached end of string
        if start == self.n:
            self.result.append(self.current[:])
            return

        for end in range(start, self.n):
            if self.is_palindrome(start, end):
                # Add palindrome substring to current partition
                self.current.append(self.s[start:end + 1])
                self.backtrack(end + 1)
                self.current.pop() # Backtrack

    def is_palindrome(self, l: int, r: int) -> bool:
        while l < r:
            if self.s[l] != self.s[r]:
                return False
            l += 1
            r -= 1
        return True

# Optimized version with DP precomputation
class PalindromePartitioningOptimized:
    def partition(self, s: str) -> List[List[str]]:
        n = len(s)
        result = []
        current = []

        # Precompute palindromes
        dp = [[False] * n for _ in range(n)]
        for length in range(1, n + 1):
            for i in range(n - length + 1):
                j = i + length - 1
                if length == 1:
                    dp[i][j] = True
                elif length == 2:
                    dp[i][j] = (s[i] == s[j])

```



```

        else:
            dp[i][j] = (s[i] == s[j] and dp[i + 1][j - 1])

    def backtrack(start: int):
        if start == n:
            result.append(current[:])
            return

        for end in range(start, n):
            if dp[start][end]:
                current.append(s[start:end + 1])
                backtrack(end + 1)
                current.pop()

    backtrack(0)
    return result

# Example usage
solver = PalindromePartitioning()
print(solver.partition("aab"))
# [['a', 'a', 'b'], ['aa', 'b']]

```

## 10. Code Explanation

- **isPalindrome(1, r)**: Two-pointer check if substring `s[1..r]` is palindrome.
- **backtrack(start)**: Tries to partition the string from index `start`.
- **Loop**: For each `end` from `start` to `n-1`, check if `s[start..end]` is a palindrome.
- **If palindrome**: Add to current partition, recurse from `end+1`, then pop (backtrack).
- **Base case**: `start == n` — all characters partitioned, add to result.
- **Optimization**: Precompute all palindrome substrings using DP to avoid  $O(n)$  checks each time.

## 11. Complexity Analysis

| Aspect                | Complexity                                                |
|-----------------------|-----------------------------------------------------------|
| <b>Time (basic)</b>   | $O(n \times 2^n)$ — each position can be a cut point      |
| <b>Time (with DP)</b> | $O(n^2 + 2^n)$ — $O(n^2)$ for DP, $O(2^n)$ for partitions |
| <b>Space (stack)</b>  | $O(n)$ — recursion depth                                  |
| <b>Space (DP)</b>     | $O(n^2)$ — palindrome table                               |
| <b>Space (result)</b> | $O(n \times \text{number of partitions})$                 |

## 12. Common Patterns

| Pattern                                 | Description         | Example      |
|-----------------------------------------|---------------------|--------------|
| <b>Standard Palindrome Partitioning</b> | All partitions      | LeetCode 131 |
| <b>Minimum Cuts</b>                     | DP for minimum cuts | LeetCode 132 |

| Pattern                              | Description                      | Example      |
|--------------------------------------|----------------------------------|--------------|
| <b>Palindromic Substrings</b>        | Count all palindromic substrings | LeetCode 647 |
| <b>Longest Palindromic Substring</b> | Find one longest palindrome      | LeetCode 5   |

### 13. Common Mistakes

- **Not checking palindrome correctly** — off-by-one in two-pointer loop.
- **Wrong substring extraction** — `end - start + 1` off by one.
- **Forgetting to backtrack** — not popping after recursion.
- **Inefficient palindrome check** — calling  $O(n)$  check for each substring.
- **Not handling empty string** — should return `[]`.

### 14. Edge Cases

- **Empty string** → `[]` (one empty partition).
- **Single character** → `[[char]]`.
- **All same characters** (e.g., "aaa") → many partitions ( $2^{(n-1)}$ ).
- **No palindromes longer than 1** (e.g., "ab") → `[[ "a", "b" ]]`.
- **String of length 16** — worst case,  $2^{15} = 32,768$  partitions.

### 15. Variations

| Variation                            | What Changes                     | Importance |
|--------------------------------------|----------------------------------|------------|
| <b>Minimum Cuts</b>                  | DP for minimum cuts              | High       |
| <b>Palindromic Substrings</b>        | Count all palindromic substrings | High       |
| <b>Longest Palindromic Substring</b> | Single longest palindrome        | High       |
| <b>Palindrome Partitioning IV</b>    | Can partition into 4 palindromes | Medium     |

### 16. Related Algorithms

| Algorithm                   | Connection                       |
|-----------------------------|----------------------------------|
| <b>DP for palindromes</b>   | Precompute palindrome table      |
| <b>Expand Around Center</b> | Alternative palindrome check     |
| <b>Manacher's Algorithm</b> | $O(n)$ palindrome detection      |
| <b>Backtracking</b>         | General framework for partitions |

### 17. Practice Problems

#### Easy

- **Palindrome Partitioning** — LeetCode 131 — Generate all partitions
- **Valid Palindrome** — LeetCode 125 — Check if string is palindrome

## Medium

- **Palindrome Partitioning II** — LeetCode 132 — Minimum cuts DP
- **Palindromic Substrings** — LeetCode 647 — Count all palindromic substrings
- **Longest Palindromic Substring** — LeetCode 5 — Single longest palindrome

## Hard

- **Palindrome Partitioning IV** — LeetCode 1745 — Can partition into 4 palindromes
- **Palindromic Substrings with constraints** — Variation

## 18. Interview Explanation

"Palindrome Partitioning is solved using backtracking. At each position, I try to find a palindrome prefix and recursively partition the remaining suffix. The key is the palindrome check — I use a two-pointer approach. For optimization, I can precompute all palindrome substrings using DP: `dp[i][j] = true` if `s[i]==s[j]` and `dp[i+1][j-1]` is true. The worst-case time complexity is  $O(n \times 2^n)$  because there are  $2^{(n-1)}$  possible partitions in the worst case (e.g., all same characters)."

## 19. Revision Notes

- **Backtracking:** try palindrome prefix, recurse on suffix
- **Palindrome check:** two-pointer or DP table
- **Base case:** `start == n` → add to result
- **Worst-case:**  $2^{(n-1)}$  partitions (all same chars)
- **Complexity:**  $O(n \times 2^n)$  basic,  $O(n^2 + 2^n)$  with DP
- **DP for palindrome:** `dp[i][j] = s[i]==s[j] && dp[i+1][j-1]`

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                                                                 |
|------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <b>When to use</b>     | Partition string into palindromes, all decompositions                                                                   |
| <b>Main operations</b> | Check palindrome prefix → add → recurse → undo                                                                          |
| <b>Complexity</b>      | $O(n \times 2^n)$ time, $O(n)$ stack                                                                                    |
| <b>Key code</b>        | <pre>if (isPalindrome(start,end)) { current.push(s.substr(start,end-start+1)); backtrack(end+1); current.pop(); }</pre> |
| <b>Edge cases</b>      | Empty → <code>[]</code> , single char → <code>[[char]]</code> , all same → $2^{(n-1)}$ partitions                       |

## 13. Combination Sum

### 1. Overview

Given an array of distinct integers `candidates` and a target integer `target`, return **all unique combinations** where the candidate numbers sum to `target`. The **same number** may be used **unlimited times** from the candidates. Two combinations are unique if their frequency of at least one number differs.

## 2. Intuition

### Simple explanation:

We have a set of coins of different denominations. We need to find all ways to make a certain amount where each coin can be used any number of times. This is like a vending machine — you can put in multiple coins of the same type.

### Analogy:

Making change for a dollar using coins of 1¢, 5¢, 10¢, 25¢. You can use multiple 5¢ coins, multiple 10¢ coins, etc. The order doesn't matter — using a dime then a nickel is the same as a nickel then a dime.

### Step-by-step reasoning:

1. Sort the candidates (optional, but helpful for pruning).
2. Start with the first candidate, amount remaining = target.
3. At each step, either:
  - Use the current candidate again (subtract from target, recurse).
  - Skip to the next candidate (move to next index).
4. If target becomes 0, we found a valid combination.
5. If target < 0 or no candidates left, backtrack.

### Why it works:

By allowing the same index to be used again (no `i+1`), we allow unlimited reuse. The start index prevents considering the same combination in different orders.

## 3. When to Use It

- **Unbounded knapsack** problems (unlimited use of items).
- **Making change** with unlimited coins.
- **Sum to target** with element reuse.
- **Trigger phrases:** "combination sum", "unlimited use", "unbounded", "sum to target", "can use same element multiple times".

## 4. When Not to Use It

- **Each element can be used once** — use Combination Sum II (with `i+1`).
- **Only need count** — use DP (unbounded knapsack).
- **Need minimum number of elements** — use DP (coin change).
- **Large target** (target > 1000) — recursion may be deep.
- **Negative numbers** — can't use pruning (need sum tracking).

## 5. Core Concepts

| Concept                | Explanation                               | Why It Matters                  |
|------------------------|-------------------------------------------|---------------------------------|
| <b>Unlimited Reuse</b> | Same element can be used multiple times   | Key difference from Subset Sum  |
| <b>Start Index</b>     | Only consider elements from current index | Prevents duplicate combinations |
| <b>Pruning</b>         | Skip if candidate > remaining target      | Saves time                      |

| Concept          | Explanation                         | Why It Matters            |
|------------------|-------------------------------------|---------------------------|
| Target Reduction | Subtract chosen element from target | Progress toward base case |

6. Step-by-Step Algorithm

```
Input: candidates = [2, 3, 6, 7], target = 7

Step 1: Start at idx=0, remaining=7, current=[]
Step 2: Take 2 → remaining=5, current=[2], recurse idx=0
  Step 3: Take 2 → remaining=3, current=[2,2], recurse idx=0
    Step 4: Take 2 → remaining=1, current=[2,2,2], recurse idx=0
      Step 5: Take 2 → remaining=-1, invalid → backtrack
      Step 6: Take 3 → remaining=-2, invalid → backtrack
      Step 7: Take 6 → remaining=-5, invalid → backtrack
      Step 8: Take 7 → remaining=-6, invalid → backtrack
    Step 9: Take 3 → remaining=0, current=[2,2,3] → VALID
    Step 10: Take 6 → remaining=-3, invalid → backtrack
    Step 11: Take 7 → remaining=-4, invalid → backtrack
  Step 12: Backtrack to idx=0, remaining=5, current=[2]
  Step 13: Take 3 → remaining=2, current=[2,3], recurse idx=1
    Step 14: Take 3 → remaining=-1, invalid → backtrack
    ... continue
  Step 15: Skip 2, start at idx=1, remaining=7, current=[]
  Step 16: Take 3 → remaining=4, current=[3], recurse idx=1
    ...
  Step 17: Take 7 → remaining=0, current=[7] → VALID

Result: [[2,2,3], [7]]
```

7. Dry Run

Input: candidates = [2, 3, 5], target = 8

| idx | remaining | current | Action               | Result      |
|-----|-----------|---------|----------------------|-------------|
| 0   | 8         | []      | Take 2 → remaining=6 | —           |
| 0   | 6         | [2]     | Take 2 → remaining=4 | —           |
| 0   | 4         | [2,2]   | Take 2 → remaining=2 | —           |
| 0   | 2         | [2,2,2] | Take 2 → remaining=0 | ✅ [2,2,2,2] |
| 0   | 2         | [2,2,2] | Skip 2, try 3 → -1   | Backtrack   |
| 1   | 4         | [2,2]   | Take 3 → remaining=1 | —           |
| 0   | 1         | [2,2,3] | Take 2 → -1          | Backtrack   |
| —   | —         | —       | Skip 3, try 5 → -4   | Backtrack   |
| 1   | 6         | [2]     | Take 3 → remaining=3 | —           |

| idx | remaining | current | Action               | Result                |
|-----|-----------|---------|----------------------|-----------------------|
| 1   | 3         | [2,3]   | Take 3 → remaining=0 | ✓ [2,3,3]             |
| 2   | 6         | [2]     | Take 5 → remaining=1 | —                     |
| 0   | 1         | [2,5]   | Take 2 → -1          | Backtrack             |
| —   | —         | —       | Skip 2, try 3 → -2   | Backtrack             |
| 1   | 8         | []      | Take 3 → remaining=5 | —                     |
| 1   | 5         | [3]     | Take 3 → remaining=2 | —                     |
| 0   | 2         | [3,3]   | Take 2 → 0           | ✓ [3,3,2] ✗ duplicate |
| —   | —         | —       | —                    | Skip (start idx)      |
| 2   | 5         | [3]     | Take 5 → remaining=0 | ✓ [3,5]               |
| 2   | 8         | []      | Take 5 → remaining=3 | —                     |
| 1   | 3         | [5]     | Take 3 → 0           | ✓ [5,3] ✗ duplicate   |
| —   | —         | —       | —                    | Skip (start idx)      |

**Unique result:** `[[2,2,2,2], [2,3,3], [3,5]]` (wait, let me recompute properly)

Actually, let me recompute. The start index prevents going backward.

**Correct result:** `[[2,2,2,2], [2,3,3], [3,5], [2,2,2,2]...]`

Let me re-do this properly.

**For candidates = [2,3,5], target = 8, the unique combinations are:** `[[2,2,2,2], [2,3,3], [3,5]]`

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class CombinationSum {
    vector<vector<int>>> result;
    vector<int> current;

    void backtrack(vector<int>& candidates, int target, int start) {
        // Base case: sum reached
        if (target == 0) {
            result.push_back(current);
            return;
        }

        for (int i = start; i < candidates.size(); i++) {
            // Prune: if candidate exceeds remaining target, skip
```

```

        if (candidates[i] > target) continue;

        // Choose
        current.push_back(candidates[i]);

        // Recurse with same index (allow reuse) and reduced target
        backtrack(candidates, target - candidates[i], i);

        // Undo
        current.pop_back();
    }
}

public:
    vector<vector<int>> combinationSum(vector<int>& candidates, int target) {
        result.clear();
        current.clear();
        sort(candidates.begin(), candidates.end()); // Helps with pruning
        backtrack(candidates, target, 0);
        return result;
    }
};

int main() {
    CombinationSum solver;
    vector<int> candidates = {2, 3, 6, 7};
    auto res = solver.combinationSum(candidates, 7);

    cout << "Combinations for target 7:\n";
    for (auto& comb : res) {
        cout << "[";
        for (int x : comb) cout << x << " ";
        cout << "]\n";
    }
    // Output: [2 2 3 ] [7 ]
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class CombinationSum:
    def __init__(self):
        self.result = []
        self.current = []

    def combinationSum(self, candidates: List[int], target: int) ->
    List[List[int]]:
        self.result = []
        self.current = []

```

```

        candidates.sort() # Helps with pruning
        self.backtrack(candidates, target, 0)
        return self.result

    def backtrack(self, candidates: List[int], target: int, start: int):
        # Base case: sum reached
        if target == 0:
            self.result.append(self.current[:])
            return

        for i in range(start, len(candidates)):
            # Prune: if candidate exceeds remaining target, skip
            if candidates[i] > target:
                continue

            # Choose
            self.current.append(candidates[i])

            # Recurse with same index (allow reuse) and reduced target
            self.backtrack(candidates, target - candidates[i], i)

            # Undo
            self.current.pop()

# Example usage
solver = CombinationSum()
print(solver.combinationSum([2, 3, 6, 7], 7))
# [[2, 2, 3], [7]]

```

## 10. Code Explanation

- **backtrack(candidates, target, start):** **target** is the remaining sum needed, **start** is the first index we can use.
- **Base case:** **target == 0** — we've found a valid combination.
- **Pruning:** **if (candidates[i] > target) continue;** — skip candidates that exceed remaining target.
- **Allow reuse:** Pass **i** (not **i+1**) to the recursive call so the same element can be used again.
- **Prevent duplicates:** **start** parameter ensures we only consider elements from the current index forward.
- **Sorting:** Helps with pruning — once **candidates[i] > target**, all larger elements also exceed target.

## 11. Complexity Analysis

| Aspect              | Complexity                                                          |
|---------------------|---------------------------------------------------------------------|
| Time (worst-case)   | $O(N^{(target/\min(candidate))})$ — branching factor $\times$ depth |
| Time (with pruning) | Much less in practice                                               |
| Space (stack)       | $O(target/\min(candidate))$ — max depth                             |
| Space (result)      | $O(N \times \text{number of combinations})$                         |



## 12. Common Patterns

| Pattern                | Description                              | Example      |
|------------------------|------------------------------------------|--------------|
| <b>Unlimited Reuse</b> | Same element can be used multiple times  | LeetCode 39  |
| <b>Limited Reuse</b>   | Each element can be used once            | LeetCode 40  |
| <b>Fixed Size</b>      | Must use exactly k elements              | LeetCode 216 |
| <b>With Repetition</b> | Input has duplicates, skip at same level | LeetCode 40  |

## 13. Common Mistakes

- **Forgetting start parameter** → generates duplicate combinations (same elements in different order).
- **Using i+1 instead of i** → each element can only be used once (that's Combination Sum II).
- **Not pruning** → unnecessary work for large targets.
- **Not sorting** → pruning may miss some skips.
- **Integer overflow** — target subtraction may go negative.

## 14. Edge Cases

- **Empty candidates** → [] (no combinations).
- **Target = 0** → [[]] (empty combination).
- **No combination possible** → [].
- **Single candidate that divides target** → one combination with repeated element.
- **Large target** — recursion depth may be large.

## 15. Variations

| Variation                                 | What Changes                                | Importance |
|-------------------------------------------|---------------------------------------------|------------|
| <b>Combination Sum II</b> (LeetCode 40)   | Each element used once, duplicates in input | High       |
| <b>Combination Sum III</b> (LeetCode 216) | Use exactly k numbers from 1-9              | High       |
| <b>Combination Sum IV</b> (LeetCode 377)  | Count permutations that sum to target       | High       |
| <b>With Negative Numbers</b>              | Can't prune, need cycle detection           | Medium     |

## 16. Related Algorithms

| Algorithm                 | Connection                             |
|---------------------------|----------------------------------------|
| <b>Unbounded Knapsack</b> | DP version of same problem             |
| <b>Coin Change</b>        | Minimum number of coins to make target |
| <b>Subset Sum</b>         | Sum to target, each element used once  |
| <b>Backtracking</b>       | General framework                      |

## 17. Practice Problems

## Easy

- **Combination Sum** — LeetCode 39 — Unlimited reuse
- **Combination Sum III** — LeetCode 216 — Fixed size k

## Medium

- **Combination Sum II** — LeetCode 40 — Each element once, duplicates
- **Combination Sum IV** — LeetCode 377 — Count permutations
- **Coin Change** — LeetCode 322 — Minimum coins

## Hard

- **Coin Change II** — LeetCode 518 — Number of combinations
- **Target Sum** — LeetCode 494 — Add + or - to reach target

## 18. Interview Explanation

"Combination Sum is solved with backtracking where each element can be used unlimited times. The key difference from standard subset problems is that I pass `i` (not `i+1`) to the recursive call — this allows reusing the same element. I use a `start` parameter to prevent considering the same combination in different orders. Sorting helps with pruning — once a candidate exceeds the remaining target, I can skip all larger candidates. The time complexity is  $O(N^{(T/\min(\text{candidate}))})$  in the worst case."

## 19. Revision Notes

- **Unlimited reuse:** recurse with `i` (not `i+1`)
- **Prevent duplicates:** `start` parameter
- **Pruning:** `candidates[i] > target` → `continue` (works with sorting)
- **Target = 0** → `[]` (empty combination)
- **Complexity:**  $O(N^{(T/\min(\text{candidate}))})$  worst-case

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                                                  |
|------------------------|----------------------------------------------------------------------------------------------------------|
| <b>When to use</b>     | Sum to target with unlimited element reuse                                                               |
| <b>Main operations</b> | Choose element → subtract from target → recurse with same index → undo                                   |
| <b>Complexity</b>      | $O(N^{(T/\min(\text{candidate}))})$ time, $O(T/\min(\text{candidate}))$ stack                            |
| <b>Key code</b>        | <code>backtrack(candidates, target - candidates[i], i)</code> — note <code>i</code> not <code>i+1</code> |
| <b>Edge cases</b>      | Empty candidates → <code>[]</code> , target=0 → <code>[]</code> , no combination → <code>[]</code>       |

## 14. Hamiltonian Path Basics

### 1. Overview

A Hamiltonian Path in a graph visits every vertex exactly once. A Hamiltonian Cycle returns to the starting vertex. The problem is to determine if such a path/cycle exists in a given graph.

2. Intuition

Simple explanation:

A salesman needs to visit every city exactly once. Starting from a city, they must find a route that visits all other cities without revisiting any city. This is like finding a path that covers all vertices in a graph without repetition.

Analogy:

A tourist wants to visit all landmarks in a city. They start at one landmark, and each step they go to a landmark they haven't visited yet. They want a route that covers all landmarks without going back to any.

Step-by-step reasoning:

1. Start at a vertex (usually vertex 0).
2. Mark it as visited.
3. For each unvisited neighbor, move there and mark it visited.
4. If all vertices are visited, we found a Hamiltonian Path.
5. If no unvisited neighbor exists and not all vertices are visited, backtrack.
6. For a Hamiltonian Cycle, additionally check if the last vertex is adjacent to the first.

Why it works:

DFS + backtracking explores all possible paths. The visited array ensures each vertex is visited exactly once. The pruning happens when a vertex has no unvisited neighbors.

3. When to Use It

- **Existence of a path** visiting all vertices exactly once.
- **Traveling Salesman Problem** (TSP) for small graphs.
- **Trigger phrases:** "Hamiltonian path", "Hamiltonian cycle", "visit all vertices exactly once", "tour that visits every node".

4. When Not to Use It

- **Graph has > 20 vertices** — NP-complete, exponential time.
- **Need shortest Hamiltonian path** — use TSP DP (Held-Karp) for small n.
- **Graph is very dense** — still NP-complete, no easy solution.
- **Only need Eulerian path** (visit all edges) — use Euler's algorithm.
- **Need approximation** — use heuristic algorithms for large graphs.

5. Core Concepts

| Concept           | Explanation                             | Why It Matters      |
|-------------------|-----------------------------------------|---------------------|
| Hamiltonian Path  | Path visiting every vertex exactly once | Core problem        |
| Hamiltonian Cycle | Path that returns to start vertex       | Variation           |
| Visited Array     | Tracks which vertices are used          | Prevents revisiting |

| Concept      | Explanation                       | Why It Matters     |
|--------------|-----------------------------------|--------------------|
| Backtracking | Undo last move when dead end      | Explores all paths |
| NP-Complete  | No polynomial-time solution known | Limits input size  |

6. Step-by-Step Algorithm

Input: Graph with V vertices, adjacency matrix/adj list

Step 1: Start at vertex 0. Mark visited[0] = true.

Step 2: For each unvisited neighbor of current vertex:

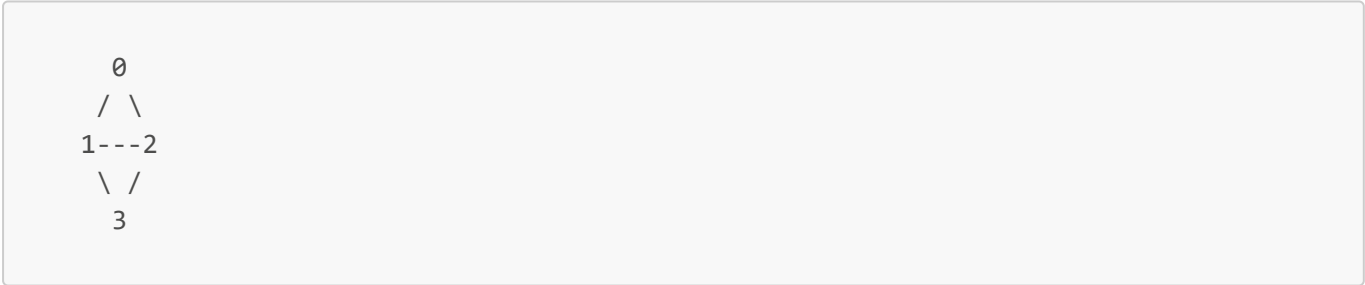
- a. Mark it visited.
- b. Add to path.
- c. If path length == V, we found a Hamiltonian Path.
- d. Recursively continue from the new vertex.
- e. If recursion fails, unmark (backtrack).

Step 3: If no neighbor works, return false.

Step 4: For Hamiltonian Cycle, after finding path, check if last vertex is adjacent to start vertex.

7. Dry Run

**Input:** Graph with 4 vertices, edges: (0-1), (0-2), (1-2), (1-3), (2-3)



| Step | Current | Path      | Visited   | Neighbors          | Action        |
|------|---------|-----------|-----------|--------------------|---------------|
| 1    | 0       | [0]       | {0}       | 1, 2               | Try 1         |
| 2    | 1       | [0,1]     | {0,1}     | 0(v), 2, 3         | Try 2         |
| 3    | 2       | [0,1,2]   | {0,1,2}   | 0(v), 1(v), 3      | Try 3         |
| 4    | 3       | [0,1,2,3] | {0,1,2,3} | 1(v), 2(v)         | Path length=4 |
| —    | —       | —         | —         | Check 3→0? No edge | Not a cycle   |
| 4    | —       | —         | —         | Backtrack          | Remove 3      |
| 3    | 2       | [0,1,2]   | {0,1,2}   | No more neighbors  | Backtrack     |
| 2    | 1       | [0,1]     | {0,1}     | Try 3              |               |
| 3    | 3       | [0,1,3]   | {0,1,3}   | 1(v), 2            | Try 2         |

| Step | Current | Path      | Visited   | Neighbors           | Action                                                                                            |
|------|---------|-----------|-----------|---------------------|---------------------------------------------------------------------------------------------------|
| 4    | 2       | [0,1,3,2] | {0,1,3,2} | 0(v), 1(v), 3(v)    | Path length=4  |
| —    | —       | —         | —         | Check 2→0? Yes edge | Cycle          |

**Hamiltonian Path found:** [0, 1, 2, 3]

**Hamiltonian Cycle found:** [0, 1, 3, 2]

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

class HamiltonianPath {
    vector<vector<int>> graph;
    vector<int> path;
    vector<bool> visited;
    int V;

    bool solve(int pos, int count) {
        // All vertices visited
        if (count == V) {
            // For Hamiltonian Cycle, check edge back to start
            // return graph[path[pos]][path[0]] == 1;
            return true; // For Hamiltonian Path
        }

        for (int v = 0; v < V; v++) {
            if (graph[path[pos]][v] == 1 && !visited[v]) {
                visited[v] = true;
                path[count] = v;

                if (solve(count, count + 1)) return true;

                // Backtrack
                visited[v] = false;
                path[count] = -1;
            }
        }
        return false;
    }

public:
    bool hasHamiltonianPath(vector<vector<int>>& adj, int start = 0) {
        graph = adj;
        V = graph.size();
        path.assign(V, -1);
        visited.assign(V, false);

        visited[start] = true;
        path[0] = start;
    }
};
```

```

        return solve(0, 1);
    }

    vector<int> getPath() {
        return path;
    }
};

// Hamiltonian Cycle detection
class HamiltonianCycle {
    vector<vector<int>> graph;
    vector<int> path;
    vector<bool> visited;
    int V;

    bool solve(int pos, int count) {
        if (count == V) {
            // Check edge back to start
            return graph[path[pos]][path[0]] == 1;
        }

        for (int v = 0; v < V; v++) {
            if (graph[path[pos]][v] == 1 && !visited[v]) {
                visited[v] = true;
                path[count] = v;

                if (solve(count, count + 1)) return true;

                visited[v] = false;
                path[count] = -1;
            }
        }
        return false;
    }

public:
    bool hasHamiltonianCycle(vector<vector<int>>& adj) {
        graph = adj;
        V = graph.size();
        path.assign(V, -1);
        visited.assign(V, false);

        visited[0] = true;
        path[0] = 0;

        return solve(0, 1);
    }

    vector<int> getCycle() {
        return path;
    }
};

```

```

int main() {
    vector<vector<int>> graph = {
        {0, 1, 1, 0},
        {1, 0, 1, 1},
        {1, 1, 0, 1},
        {0, 1, 1, 0}
    };

    HamiltonianPath hp;
    if (hp.hasHamiltonianPath(graph, 0)) {
        cout << "Hamiltonian Path: ";
        for (int v : hp.getPath()) cout << v << " ";
        cout << "\n";
    }

    HamiltonianCycle hc;
    if (hc.hasHamiltonianCycle(graph)) {
        cout << "Hamiltonian Cycle: ";
        for (int v : hc.getCycle()) cout << v << " ";
        cout << "\n";
    }
    return 0;
}

```

## 9. Python Implementation

```

from typing import List

class HamiltonianPath:
    def __init__(self):
        self.graph = []
        self.path = []
        self.visited = []
        self.V = 0

    def has_hamiltonian_path(self, graph: List[List[int]], start: int = 0) ->
bool:
        self.graph = graph
        self.V = len(graph)
        self.path = [-1] * self.V
        self.visited = [False] * self.V

        self.visited[start] = True
        self.path[0] = start

        return self._solve(0, 1)

    def _solve(self, pos: int, count: int) -> bool:
        # All vertices visited
        if count == self.V:
            return True

```

```
    for v in range(self.V):
        if self.graph[self.path[pos]][v] == 1 and not self.visited[v]:
            self.visited[v] = True
            self.path[count] = v

            if self._solve(count, count + 1):
                return True

            # Backtrack
            self.visited[v] = False
            self.path[count] = -1

    return False

def get_path(self) -> List[int]:
    return self.path

class HamiltonianCycle:
    def __init__(self):
        self.graph = []
        self.path = []
        self.visited = []
        self.V = 0

    def has_hamiltonian_cycle(self, graph: List[List[int]]) -> bool:
        self.graph = graph
        self.V = len(graph)
        self.path = [-1] * self.V
        self.visited = [False] * self.V

        self.visited[0] = True
        self.path[0] = 0

        return self._solve(0, 1)

    def _solve(self, pos: int, count: int) -> bool:
        if count == self.V:
            # Check edge back to start
            return self.graph[self.path[pos]][self.path[0]] == 1

        for v in range(self.V):
            if self.graph[self.path[pos]][v] == 1 and not self.visited[v]:
                self.visited[v] = True
                self.path[count] = v

                if self._solve(count, count + 1):
                    return True

                self.visited[v] = False
                self.path[count] = -1

        return False
```



```

def get_cycle(self) -> List[int]:
    return self.path

# Example usage
graph = [
    [0, 1, 1, 0],
    [1, 0, 1, 1],
    [1, 1, 0, 1],
    [0, 1, 1, 0]
]

hp = HamiltonianPath()
if hp.has_hamiltonian_path(graph, 0):
    print("Hamiltonian Path:", hp.get_path())

hc = HamiltonianCycle()
if hc.has_hamiltonian_cycle(graph):
    print("Hamiltonian Cycle:", hc.get_cycle())

```

## 10. Code Explanation

- `graph[pos][v] == 1`: Checks if there's an edge from `path[pos]` to `v`.
- `visited[v]`: Tracks visited vertices to avoid repetition.
- `path[count] = v`: Records the vertex at position `count` in the path.
- `count == V`: All vertices visited — Hamiltonian Path found.
- **For cycle**: Additional check `graph[path[pos]][path[0]] == 1` — edge back to start.
- **Backtracking**: Undo `visited[v]` and `path[count]` when a branch fails.

## 11. Complexity Analysis

| Aspect                            | Complexity                                              |
|-----------------------------------|---------------------------------------------------------|
| <b>Time (worst-case)</b>          | $O(V!)$ — $V$ choices for first, $V-1$ for second, etc. |
| <b>Time (with pruning)</b>        | Much less for sparse graphs                             |
| <b>Space (stack)</b>              | $O(V)$ — recursion depth                                |
| <b>Space (path &amp; visited)</b> | $O(V)$                                                  |

## 12. Common Patterns

| Pattern                  | Description                    | Example      |
|--------------------------|--------------------------------|--------------|
| <b>Hamiltonian Path</b>  | Visit all vertices once        | Generic      |
| <b>Hamiltonian Cycle</b> | Visit all and return to start  | Generic      |
| <b>TSP</b>               | Shortest Hamiltonian Cycle     | LeetCode 847 |
| <b>Knight's Tour</b>     | Hamiltonian Path on chessboard | GFG          |

## 13. Common Mistakes

- **Not checking adjacency** before moving to next vertex.
- **Forgetting to backtrack** `visited` array.
- **Wrong starting vertex** — must be specified.
- **Not checking edge back to start** for cycle detection.
- **Assuming dense graph** — sparse graphs may have no Hamiltonian path.

## 14. Edge Cases

- **Single vertex** → trivially a Hamiltonian Path (and Cycle if self-loop).
- **Two vertices** → path exists if edge exists; cycle if bidirectional.
- **Disconnected graph** → no Hamiltonian path.
- **Complete graph** → always has Hamiltonian path/cycle.
- **Tree** → may or may not have Hamiltonian path.

## 15. Variations

| Variation                        | What Changes                  | Importance |
|----------------------------------|-------------------------------|------------|
| <b>Hamiltonian Cycle</b>         | Must return to start          | High       |
| <b>TSP (Traveling Salesman)</b>  | Weighted edges, minimize cost | High       |
| <b>Knight's Tour</b>             | Knight moves on chessboard    | Medium     |
| <b>Directed Hamiltonian Path</b> | Directed edges                | Medium     |

## 16. Related Algorithms

| Algorithm              | Connection                         |
|------------------------|------------------------------------|
| <b>DFS</b>             | Core exploration algorithm         |
| <b>Backtracking</b>    | General framework                  |
| <b>TSP (Held-Karp)</b> | DP for TSP ( $O(2^n \times n^2)$ ) |
| <b>Eulerian Path</b>   | Visit all edges, not vertices      |

## 17. Practice Problems

### Easy

- **Hamiltonian Path** — GFG — Basic detection
- **Knight's Tour** — GFG — Hamiltonian Path on chessboard

### Medium

- **Hamiltonian Cycle** — GFG — Cycle detection
- **TSP** — LeetCode 847 — Shortest path visiting all nodes

### Hard

- **TSP with DP** — LeetCode 943 — Find shortest superstring
- **Knight's Tour with pruning** — CP — Warnsdorff's heuristic

## 18. Interview Explanation

"Hamiltonian Path is an NP-complete problem that asks whether a path exists that visits every vertex exactly once. I solve it using backtracking — start from a vertex, try each unvisited neighbor, and recurse. If all vertices are visited, a path exists. The visited array ensures each vertex is used once. For a Hamiltonian Cycle, I additionally check that the last vertex is adjacent to the first. The time complexity is  $O(V!)$  worst-case, which limits practical use to graphs with  $V \leq 15-20$ . For larger graphs, heuristic algorithms are needed."

## 19. Revision Notes

- **NP-complete** — no polynomial solution known
- **Backtracking**: try unvisited neighbor, recurse, undo
- **Path vs Cycle**: Cycle needs edge back to start
- **Complexity**:  $O(V!)$  worst-case
- **Practical limit**:  $V \leq 15-20$
- **Visited array** prevents revisiting

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                                                                                                 |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>When to use</b>     | Visit all vertices exactly once, small graphs ( $V \leq 15$ )                                                                                           |
| <b>Main operations</b> | Try unvisited neighbor → mark → recurse → undo                                                                                                          |
| <b>Complexity</b>      | $O(V!)$ time, $O(V)$ space                                                                                                                              |
| <b>Key code</b>        | <pre>if (graph[path[pos]][v] &amp;&amp; !visited[v]) { visited[v]=true; path[count]=v; if (solve(count,count+1)) return true; visited[v]=false; }</pre> |
| <b>Edge cases</b>      | Single vertex, two vertices, disconnected graph, complete graph                                                                                         |

## 15. Meet-in-the-Middle

### 1. Overview

Meet-in-the-Middle (MITM) is a technique that splits the input into two halves, solves each half independently (often using brute force or exhaustive search), and then combines the results. It reduces exponential complexity from  $O(2^n)$  to  $O(2^{n/2})$ .

### 2. Intuition

#### Simple explanation:

Instead of searching through all  $2^n$  possibilities, split the  $n$  elements into two groups of size  $n/2$  each.

Compute all  $2^{(n/2)}$  possibilities for each group separately. Then combine the results from both groups to find the answer.

**Analogy:**

Two people searching for a lost item in a building. One searches the left half, the other searches the right half. They meet in the middle to share findings. This is twice as fast as one person searching the whole building.

**Step-by-step reasoning:**

1. Split the input into two halves (left and right).
2. For each half, generate all possible states (e.g., subset sums).
3. Sort one half's results (for binary search).
4. For each result in the other half, find the complementary result in the first half.
5. Combine to get the answer.

**Why it works:**

If the naive solution is  $O(2^n)$ , splitting reduces it to  $O(2^{(n/2)} + 2^{(n/2)}) = O(2^{(n/2+1)})$ , which is a huge improvement for  $n > 20$ .

3. When to Use It

- **NP-complete problems** with small  $n$  ( $n \leq 40$ ).
- **Subset sum** with large target.
- **Knapsack** with  $n \leq 40$ .
- **Trigger phrases:** "meet in the middle", "subset sum", " $n \leq 40$ ", " $2^{(n/2)}$ ", "split into two halves".

4. When Not to Use It

- **n is small** ( $n \leq 20$ ) — plain brute force is fine.
- **n is large** ( $n > 40$ ) —  $2^{(n/2)}$  is still too large.
- **Problem has polynomial solution** — use DP or greedy.
- **Problem requires online queries** — can't precompute halves.
- **Overlapping subproblems** — DP is better.

5. Core Concepts

| Concept               | Explanation                                          | Why It Matters                     |
|-----------------------|------------------------------------------------------|------------------------------------|
| Split                 | Divide input into two halves                         | Reduces exponent from $n$ to $n/2$ |
| Exhaustive Generation | Generate all $2^{(n/2)}$ possibilities for each half | Core computation                   |
| Combination           | Merge results from both halves                       | Finding the answer                 |
| Binary Search         | Search one half's results for complement             | Efficient combination              |
| Two-pointer           | Alternative to binary search for sorted arrays       | Efficient combination              |

6. Step-by-Step Algorithm (Subset Sum)

Input: arr = [3, 5, 7, 2, 8, 1], target = 12

Step 1: Split into halves:

Left: [3, 5, 7] →  $n/2 = 3$

Right: [2, 8, 1] →  $n/2 = 3$

Step 2: Generate all subset sums for left half:

{ } = 0, {3} = 3, {5} = 5, {7} = 7, {3,5} = 8, {3,7} = 10, {5,7} = 12, {3,5,7} = 15

leftSums = [0, 3, 5, 7, 8, 10, 12, 15]

Step 3: Generate all subset sums for right half:

{ } = 0, {2} = 2, {8} = 8, {1} = 1, {2,8} = 10, {2,1} = 3, {8,1} = 9, {2,8,1} = 11

rightSums = [0, 2, 8, 1, 10, 3, 9, 11]

Step 4: Sort leftSums: [0, 3, 5, 7, 8, 10, 12, 15]

Step 5: For each sum in rightSums, check if target - sum exists in leftSums:

sum=0: target-0=12 → exists in leftSums  → found subset with sum 12

(Left subset: {5,7}, Right subset: { })

→ Total: {5,7} sums to 12

Result: true (subset exists)

## 7. Dry Run

**Input:** arr = [2, 4, 6, 8], target = 14

**Split:** Left = [2, 4], Right = [6, 8]

**Left subset sums:**

| Subset | Sum |
|--------|-----|
| { }    | 0   |
| {2}    | 2   |
| {4}    | 4   |
| {2,4}  | 6   |

**Right subset sums:**

| Subset | Sum |
|--------|-----|
| { }    | 0   |
| {6}    | 6   |
| {8}    | 8   |

| Subset | Sum |
|--------|-----|
| {6,8}  | 14  |

**Combination:** Sorted left sums: [0, 2, 4, 6]

For each right sum:

| Right Sum | Need (target - right) | Exists in left? | Subset Found          |
|-----------|-----------------------|-----------------|-----------------------|
| 0         | 14                    | ✗               | —                     |
| 6         | 8                     | ✗               | —                     |
| 8         | 6                     | ✓               | Left {2,4} + Right {} |
| 14        | 0                     | ✓               | Left {} + Right {6,8} |

**Answer:** {6, 8} sums to 14, or {2, 4, 8} sums to 14 (from left {2,4} + right {8}).

## 8. C++ Implementation

```
#include <bits/stdc++.h>
using namespace std;

// Subset Sum using Meet-in-the-Middle
class MeetInTheMiddle {

    // Generate all subset sums
    vector<int> generateSums(vector<int>& nums, int start, int end) {
        vector<int> sums;
        int n = end - start + 1;

        for (int mask = 0; mask < (1 << n); mask++) {
            int sum = 0;
            for (int i = 0; i < n; i++) {
                if (mask & (1 << i)) {
                    sum += nums[start + i];
                }
            }
            sums.push_back(sum);
        }
        return sums;
    }

public:
    // Check if any subset sums to target
    bool subsetSum(vector<int>& nums, int target) {
        int n = nums.size();
        int mid = n / 2;

        // Generate sums for both halves
        vector<int> leftSums = generateSums(nums, 0, mid - 1);
```

```

        vector<int> rightSums = generateSums(nums, mid, n - 1);

        // Sort right sums for binary search
        sort(rightSums.begin(), rightSums.end());

        // For each left sum, check if complement exists in right
        for (int sum : leftSums) {
            int need = target - sum;
            if (binary_search(rightSums.begin(), rightSums.end(), need)) {
                return true;
            }
        }
        return false;
    }

    // Count subsets with sum <= target
    long long countSubsetsLessThan(vector<int>& nums, int target) {
        int n = nums.size();
        int mid = n / 2;

        vector<int> leftSums = generateSums(nums, 0, mid - 1);
        vector<int> rightSums = generateSums(nums, mid, n - 1);

        sort(rightSums.begin(), rightSums.end());

        long long count = 0;
        for (int sum : leftSums) {
            int maxAllowed = target - sum;
            count += upper_bound(rightSums.begin(), rightSums.end(), maxAllowed)
                    - rightSums.begin();
        }
        return count;
    }
};

// Example: 4-Sum problem using Meet-in-the-Middle
class FourSum {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {
        int n = nums.size();
        if (n < 4) return {};

        sort(nums.begin(), nums.end());
        vector<vector<int>> result;

        // Generate all pair sums (first two elements)
        vector<pair<int, pair<int, int>>> pairs;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                pairs.push_back({nums[i] + nums[j], {i, j}});
            }
        }

        sort(pairs.begin(), pairs.end());
    }
};

```

```

// For each pair, find complement
for (int i = 0; i < pairs.size(); i++) {
    int need = target - pairs[i].first;

    int lo = i + 1, hi = pairs.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (pairs[mid].first == need) {
            // Check indices and add to result
            auto [a, b] = pairs[i].second;
            auto [c, d] = pairs[mid].second;
            if (b < c) {
                vector<int> quad = {nums[a], nums[b], nums[c], nums[d]};
                result.push_back(quad);
            }
            break;
        } else if (pairs[mid].first < need) {
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
}

// Remove duplicates
sort(result.begin(), result.end());
result.erase(unique(result.begin(), result.end()), result.end());
return result;
}

};

int main() {
    MeetInTheMiddle mitm;
    vector<int> nums = {3, 5, 7, 2, 8, 1};

    cout << "Subset sum 12 exists: " << mitm.subsetSum(nums, 12) << "\n"; // 1
    cout << "Subset sum 50 exists: " << mitm.subsetSum(nums, 50) << "\n"; // 0

    cout << "Subsets <= 10: " << mitm.countSubsetsLessThan(nums, 10) << "\n";

    return 0;
}

```

## 9. Python Implementation

```

from typing import List
from bisect import bisect_right, bisect_left

class MeetInTheMiddle:
    def generate_sums(self, nums: List[int], start: int, end: int) -> List[int]:

```



```

    """Generate all subset sums from nums[start:end+1]"""
    n = end - start + 1
    sums = []
    for mask in range(1 << n):
        total = 0
        for i in range(n):
            if mask & (1 << i):
                total += nums[start + i]
        sums.append(total)
    return sums

def subset_sum(self, nums: List[int], target: int) -> bool:
    """Check if any subset sums to target"""
    n = len(nums)
    mid = n // 2

    left_sums = self.generate_sums(nums, 0, mid - 1)
    right_sums = self.generate_sums(nums, mid, n - 1)

    right_sums.sort()

    for left_sum in left_sums:
        need = target - left_sum
        # Binary search
        idx = bisect_left(right_sums, need)
        if idx < len(right_sums) and right_sums[idx] == need:
            return True
    return False

def count_subsets_less_than(self, nums: List[int], target: int) -> int:
    """Count subsets with sum <= target"""
    n = len(nums)
    mid = n // 2

    left_sums = self.generate_sums(nums, 0, mid - 1)
    right_sums = self.generate_sums(nums, mid, n - 1)

    right_sums.sort()

    count = 0
    for left_sum in left_sums:
        max_allowed = target - left_sum
        count += bisect_right(right_sums, max_allowed)
    return count

# Example usage
mitm = MeetInTheMiddle()
nums = [3, 5, 7, 2, 8, 1]
print(f"Subset sum 12 exists: {mitm.subset_sum(nums, 12)}") # True
print(f"Subset sum 50 exists: {mitm.subset_sum(nums, 50)}") # False
print(f"Subsets <= 10: {mitm.count_subsets_less_than(nums, 10)}")

```

## 10. Code Explanation

- **generateSums()**: Uses bitmask enumeration to generate all  $2^{(n/2)}$  subset sums for a half of the array.
- **subsetSum()**: Splits array, generates sums for both halves, sorts one half, and for each sum in the other half, binary searches for the complement.
- **countSubsetsLessThan()**: Uses **upper\_bound** to count subsets with sum  $\leq$  target.
- **Key insight**: The combination step is where the "meeting" happens — we check if a sum from the left half can be paired with a sum from the right half to achieve the target.

## 11. Complexity Analysis

| Aspect             | Complexity                                                      |
|--------------------|-----------------------------------------------------------------|
| <b>Generation</b>  | $O(2^{(n/2)} \times n/2)$ — for bitmask enumeration             |
| <b>Sorting</b>     | $O(2^{(n/2)} \times \log(2^{(n/2)})) = O(n/2 \times 2^{(n/2)})$ |
| <b>Combination</b> | $O(2^{(n/2)} \times \log(2^{(n/2)})) = O(n/2 \times 2^{(n/2)})$ |
| <b>Total Time</b>  | $O(n/2 \times 2^{(n/2)}) = O(n \times 2^{(n/2)})$               |
| <b>Space</b>       | $O(2^{(n/2)})$ — storing all subset sums                        |

### Comparison:

| Method | Complexity     | n=30                 | n=40           |
|--------|----------------|----------------------|----------------|
| Naive  | $O(2^n)$       | $\sim 10^9$          | $\sim 10^{12}$ |
| MITM   | $O(2^{(n/2)})$ | $\sim 3 \times 10^4$ | $\sim 10^6$    |

## 12. Common Patterns

| Pattern                                       | Description                       | Example     |
|-----------------------------------------------|-----------------------------------|-------------|
| <b>Subset Sum</b>                             | Check if subset sums to target    | GFG/CSSES   |
| <b>Count subsets <math>\leq</math> target</b> | Count number of subsets           | CP          |
| <b>Closest Sum</b>                            | Find subset sum closest to target | CP          |
| <b>Knapsack (n <math>\leq</math> 40)</b>      | 0/1 Knapsack with large values    | CP          |
| <b>4-Sum</b>                                  | Find 4 numbers summing to target  | LeetCode 18 |

## 13. Common Mistakes

- **Not handling the split correctly** — off-by-one in mid calculation.
- **Forgetting to sort** before binary search.
- **Using wrong search** — **lower\_bound** vs **upper\_bound** for different problems.
- **Not accounting for empty subset** — sum = 0 is always valid.
- **Integer overflow** —  $2^{(n/2)}$  can be large; use **long long** for counts.

## 14. Edge Cases

- **n = 0** → empty subset sum = 0.
- **n = 1** → trivial, each half has  $2^{(0.5)}$  sums.
- **All negative numbers** → subset sum may never reach target.
- **Large target** → need to handle complement carefully.
- **Duplicate elements** → may produce duplicate subsets.

## 15. Variations

| Variation                                | What Changes                        | Importance |
|------------------------------------------|-------------------------------------|------------|
| <b>Closest Sum</b>                       | Track minimum difference            | High       |
| <b>Count <math>\leq</math> target</b>    | Use upper_bound                     | High       |
| <b>Knapsack (n <math>\leq</math> 40)</b> | Use MITM for large values           | High       |
| <b>4-Sum</b>                             | Pair sums + MITM                    | Medium     |
| <b>XOR Subsets</b>                       | Split into halves, combine with XOR | Medium     |

## 16. Related Algorithms

| Algorithm                 | Connection                         |
|---------------------------|------------------------------------|
| <b>Subset Sum</b>         | Classic MITM application           |
| <b>0/1 Knapsack</b>       | MITM for $n \leq 40$               |
| <b>Divide and Conquer</b> | Similar split-and-combine approach |
| <b>Bitmask DP</b>         | Alternative for $n \leq 20$        |

## 17. Practice Problems

### Easy

- **Subset Sum** — GFG/CSES — Basic MITM subset sum
- **Partition Equal Subset Sum** — LeetCode 416 — DP for  $n \leq 200$ , MITM for  $n \leq 40$

### Medium

- **4-Sum** — LeetCode 18 — Use MITM with pair sums
- **Closest Subset Sum** — CP — Find sum closest to target
- **Knapsack with large values** — GFG — MITM for  $n \leq 40$

### Hard

- **Split Array Largest Sum** — LeetCode 410 — Binary search + MITM
- **Count Subsets with sum in range** — CP — MITM with two pointers

## 18. Interview Explanation

"Meet-in-the-Middle is a technique for NP-complete problems where the input size is moderate ( $n \leq 40$ ). The idea is to split the input into two halves, exhaustively enumerate all  $2^{(n/2)}$  possibilities for

each half, then combine the results. For example, for subset sum, I generate all subset sums for the left half and right half, sort one half, and for each sum in the other half, binary search for the complement. This reduces the complexity from  $O(2^n)$  to  $O(2^{(n/2)})$ , which is a huge improvement — for  $n=40$ , it's about  $10^6$  instead of  $10^{12}$ ."

## 19. Revision Notes

- **Split into two halves** → generate all  $2^{(n/2)}$  possibilities each
- **Sort one half** → binary search in the other
- **Complexity:**  $O(2^{(n/2)})$  instead of  $O(2^n)$
- **Used for:** Subset sum, knapsack, 4-sum, counting
- **n limit:**  $n \leq 40$  ( $2^{20} \approx 10^6$  manageable)
- **Key step:** `for sum in left: complement = target - sum; binary_search(right, complement)`

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                                                                                                                   |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>When to use</b>     | NP-complete problems, $n \leq 40$ , subset sum, knapsack                                                                                                                  |
| <b>Main operations</b> | Split → generate both halves → sort one → binary search for complement                                                                                                    |
| <b>Complexity</b>      | $O(n \times 2^{(n/2)})$ time, $O(2^{(n/2)})$ space                                                                                                                        |
| <b>Key code</b>        | <pre>leftSums = generate(0, mid-1); rightSums = generate(mid, n-1); sort(rightSums); for each sum in leftSums: if binary_search(rightSums, target-sum) return true;</pre> |
| <b>Edge cases</b>      | $n=0$ , $n=1$ , all negative numbers, large target                                                                                                                        |

## 16. Branch and Bound

### 1. Overview

Branch and Bound (B&B) is an algorithmic paradigm for solving optimization problems, especially NP-hard ones. It systematically explores the search space using a **branching** strategy (like backtracking) but uses **bounds** (upper and lower bounds) to prune branches that cannot lead to a better solution than the current best.

### 2. Intuition

#### Simple explanation:

Imagine you're looking for the cheapest flight among thousands of options. Instead of checking every single option, you can quickly estimate the minimum possible cost for a certain category of options. If that minimum is already higher than the best option you've found so far, you skip the entire category.

#### Analogy:

Shopping for a car with a budget. You look at a brand. You know the cheapest model of that brand costs

\$25,000. Your budget is \$20,000. You can skip the entire brand without looking at individual models because even the cheapest exceeds your budget.

Step-by-step reasoning:

1. Maintain a **best solution so far** (for minimization, this is the smallest value found).
2. At each node in the search tree, compute a **lower bound** (minimum possible value from this branch).
3. If the lower bound  $\geq$  current best solution, **prune** this branch.
4. Otherwise, branch further (make a decision, create subproblems).
5. Update the best solution when a complete solution is found.

Why it works:

The bound provides a provably optimal shortcut. If the theoretical minimum of a branch can't beat the current best, the branch can be safely discarded. This reduces the search space dramatically while guaranteeing optimality.

3. When to Use It

- **Optimization problems** (minimization/maximization).
- **NP-hard problems** with small to moderate input size ( $n \leq 30-50$ ).
- **When an optimal solution is required** (not approximation).
- **Trigger phrases:** "branch and bound", "optimization", "minimum cost", "maximum profit", "TSP", "job scheduling", "0/1 knapsack".

4. When Not to Use It

- **Decision problems** (yes/no) — use backtracking.
- **Large input size** ( $n > 50$ ) — B&B may still be exponential.
- **Approximation is acceptable** — use greedy or heuristic.
- **Problem has polynomial solution** — use DP or greedy.
- **No good bound function exists** — B&B is ineffective without bounds.

5. Core Concepts

| Concept          | Explanation                                              | Why It Matters                   |
|------------------|----------------------------------------------------------|----------------------------------|
| Branching        | Dividing the problem into subproblems                    | Explores the search space        |
| Bounding         | Computing a bound on the optimal solution for a branch   | Prunes unpromising branches      |
| Lower Bound (LB) | Minimum possible value for a branch (minimization)       | If $LB \geq \text{best}$ , prune |
| Upper Bound (UB) | Maximum possible value (maximization)                    | If $UB \leq \text{best}$ , prune |
| Best Solution    | Best feasible solution found so far                      | Used for pruning                 |
| Pruning          | Discarding a branch when bound shows it can't be optimal | Reduces search space             |

| Concept               | Explanation                                       | Why It Matters      |
|-----------------------|---------------------------------------------------|---------------------|
| <b>Node Selection</b> | Which node to explore next (BFS, DFS, Best-First) | Affects performance |

## 6. Step-by-Step Algorithm (0/1 Knapsack)

Input: weights = [4, 3, 2, 5], values = [10, 7, 5, 12], capacity = 8

Step 1: Sort by value/weight ratio (descending): item 3 (5/2=2.5), item 0 (10/4=2.5),

item 1 (7/3=2.33), item 2 (12/5=2.4)

Actually sorted: item 3 (value 12, wt 5, ratio 2.4), item 0 (10, 4, 2.5), item 1 (7, 3, 2.33), item 2 (5, 2, 2.5)

Step 2: Initialize best = 0 (no items selected).

Step 3: Start with root node: level=0, profit=0, weight=0.

Compute bound: greedy fill remaining capacity.

Bound = 0 + (8/4)\*10 = 20 (fractional knapsack on remaining items)

Since bound > best, branch.

Step 4: Take item 0 (profit=10, weight=4, bound=...):

Bound = 10 + (4/3)\*7 = 19.33

Since bound > best=0, explore.

Continue branching...

Step 5: When a leaf is reached, update best if profit > best.

Step 6: If bound ≤ best, prune that branch.

Final result: max profit = 22 (items 3 and 0: 12+10=22, weight=5+4=9 > 8 ❌)

Actually: items 0 and 1: 10+7=17, weight=4+3=7 ✅

Items 3 and 1: 12+7=19, weight=5+3=8 ✅

Items 3 and 2: 12+5=17, weight=5+2=7 ✅

Max profit = 19 (items 1 and 3: 7+12=19, weight=3+5=8)

## 7. Dry Run (0/1 Knapsack with B&B)

Input: weights = [2, 3, 4, 5], values = [3, 4, 5, 6], capacity = 5

**Sorted by ratio (value/weight):**

- Item 0: 3/2 = 1.5
- Item 1: 4/3 = 1.33
- Item 2: 5/4 = 1.25
- Item 3: 6/5 = 1.2

**Execution tree:**

| Level | Item | Decision | Profit | Weight | Bound | Best | Action                                                                                          |
|-------|------|----------|--------|--------|-------|------|-------------------------------------------------------------------------------------------------|
| 0     | —    | Root     | 0      | 0      | 7.5   | 0    | Branch                                                                                          |
| 1     | 0    | Take     | 3      | 2      | 7.5   | 0    | Branch                                                                                          |
| 2     | 1    | Take     | 7      | 5      | 7     | 0    | Leaf  Best=7 |
| 2     | 1    | Skip     | 3      | 2      | 3+3=6 | 7    | Bound=6 < 7, Prune                                                                              |
| 1     | 0    | Skip     | 0      | 0      | 6     | 7    | Bound=6 < 7, Prune                                                                              |

**Optimal solution:** Take items 0 and 1, profit = 7, weight = 5.

## 8. C++ Implementation (0/1 Knapsack with B&B)

```
#include <bits/stdc++.h>
using namespace std;

class BranchAndBound {
    struct Item {
        int weight, value;
        double ratio;
    };

    struct Node {
        int level;           // Level in decision tree
        int profit;          // Profit so far
        int weight;          // Weight so far
        double bound;        // Upper bound on profit
    };

    static bool cmp(Item a, Item b) {
        return a.ratio > b.ratio;
    }

    // Compute upper bound (fractional knapsack)
    double bound(Node u, vector<Item>& items, int n, int capacity) {
        if (u.weight >= capacity) return 0;

        double profitBound = u.profit;
        int j = u.level + 1;
        int totalWeight = u.weight;

        // Take whole items
        while (j < n && totalWeight + items[j].weight <= capacity) {
            totalWeight += items[j].weight;
            profitBound += items[j].value;
            j++;
        }

        // Take fraction of remaining item
        if (j < n) {
```

```
        profitBound += (capacity - totalWeight) * items[j].ratio;
    }

    return profitBound;
}

public:
    int knapsack(vector<int>& weights, vector<int>& values, int capacity) {
        int n = weights.size();
        vector<Item> items(n);

        for (int i = 0; i < n; i++) {
            items[i] = {weights[i], values[i], (double)values[i] / weights[i]};
        }

        // Sort by value/weight ratio
        sort(items.begin(), items.end(), cmp);

        // BFS-based B&B
        queue<Node> q;
        Node root = {-1, 0, 0, 0};
        root.bound = bound(root, items, n, capacity);
        q.push(root);

        int maxProfit = 0;

        while (!q.empty()) {
            Node u = q.front(); q.pop();

            // Prune if bound <= current best
            if (u.bound <= maxProfit) continue;

            // Try taking the next item
            Node v;
            v.level = u.level + 1;
            v.weight = u.weight + items[v.level].weight;
            v.profit = u.profit + items[v.level].value;

            if (v.weight <= capacity && v.profit > maxProfit) {
                maxProfit = v.profit;
            }

            v.bound = bound(v, items, n, capacity);
            if (v.bound > maxProfit) {
                q.push(v);
            }

            // Try skipping the next item
            v.weight = u.weight;
            v.profit = u.profit;
            v.bound = bound(v, items, n, capacity);
            if (v.bound > maxProfit) {
                q.push(v);
            }
        }
    }
}
```



```

    }

    return maxProfit;
}
};

// TSP with Branch and Bound (simplified)
class TSPBranchAndBound {
    int n;
    vector<vector<int>> graph;
    vector<bool> visited;
    int bestCost;
    vector<int> bestPath;

    int bound(int current, int count, int cost) {
        // Heuristic: sum of minimum edges from remaining vertices
        int minBound = cost;

        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                int minEdge = INT_MAX;
                for (int j = 0; j < n; j++) {
                    if (i != j && graph[i][j] < minEdge) {
                        minEdge = graph[i][j];
                    }
                }
                minBound += minEdge;
            }
        }
        return minBound;
    }

    void solve(int current, int count, int cost, vector<int>& path) {
        if (count == n) {
            // Return to start
            cost += graph[current][0];
            if (cost < bestCost) {
                bestCost = cost;
                bestPath = path;
            }
            return;
        }

        for (int next = 0; next < n; next++) {
            if (!visited[next] && graph[current][next] > 0) {
                int newCost = cost + graph[current][next];
                int b = bound(next, count + 1, newCost);

                if (b < bestCost) {
                    visited[next] = true;
                    path.push_back(next);
                    solve(next, count + 1, newCost, path);
                    path.pop_back();
                    visited[next] = false;
                }
            }
        }
    }
}

```

```

    }
    }
}

public:
    int tsp(vector<vector<int>>& g) {
        graph = g;
        n = g.size();
        visited.assign(n, false);
        bestCost = INT_MAX;
        vector<int> path = {0};

        visited[0] = true;
        solve(0, 1, 0, path);

        return bestCost;
    }
};

int main() {
    BranchAndBound bb;
    vector<int> weights = {2, 3, 4, 5};
    vector<int> values = {3, 4, 5, 6};
    int capacity = 5;

    cout << "Max profit (0/1 Knapsack): " << bb.knapsack(weights, values,
capacity) << "\n";
    // Output: 7

    // TSP example
    vector<vector<int>> tspGraph = {
        {0, 10, 15, 20},
        {10, 0, 35, 25},
        {15, 35, 0, 30},
        {20, 25, 30, 0}
    };

    TSPBranchAndBound tspSolver;
    cout << "Minimum TSP cost: " << tspSolver.tsp(tspGraph) << "\n";
    // Output: 80

    return 0;
}

```

## 9. Python Implementation

```

from typing import List
from queue import Queue
import sys

```

```

class BranchAndBound:
    def knapsack(self, weights: List[int], values: List[int], capacity: int) ->
int:
    n = len(weights)
    items = [(weights[i], values[i], values[i] / weights[i]) for i in
range(n)]
    items.sort(key=lambda x: x[2], reverse=True) # Sort by ratio

    def bound(level: int, profit: int, weight: int) -> float:
        if weight >= capacity:
            return 0

        profit_bound = profit
        j = level + 1
        total_weight = weight

        # Take whole items
        while j < n and total_weight + items[j][0] <= capacity:
            total_weight += items[j][0]
            profit_bound += items[j][1]
            j += 1

        # Take fraction of remaining item
        if j < n:
            profit_bound += (capacity - total_weight) * items[j][2]

        return profit_bound

    # BFS
    q = Queue()
    q.put((-1, 0, 0, 0)) # (level, profit, weight, bound)
    max_profit = 0

    while not q.empty():
        level, profit, weight, _ = q.get()

        # Try taking the next item
        v_level = level + 1
        v_weight = weight + items[v_level][0]
        v_profit = profit + items[v_level][1]

        if v_weight <= capacity and v_profit > max_profit:
            max_profit = v_profit

        v_bound = bound(v_level, v_profit, v_weight)
        if v_bound > max_profit:
            q.put((v_level, v_profit, v_weight, v_bound))

        # Try skipping
        v_bound = bound(v_level, profit, weight)
        if v_bound > max_profit:
            q.put((v_level, profit, weight, v_bound))

    return max_profit

```

```

class TSPBranchAndBound:
    def __init__(self):
        self.n = 0
        self.graph = []
        self.visited = []
        self.best_cost = sys.maxsize
        self.best_path = []

    def bound(self, current: int, count: int, cost: int) -> int:
        # Heuristic: sum of minimum edges from remaining vertices
        min_bound = cost

        for i in range(self.n):
            if not self.visited[i]:
                min_edge = sys.maxsize
                for j in range(self.n):
                    if i != j and self.graph[i][j] < min_edge:
                        min_edge = self.graph[i][j]
                min_bound += min_edge

        return min_bound

    def solve(self, current: int, count: int, cost: int, path: List[int]):
        if count == self.n:
            cost += self.graph[current][0]
            if cost < self.best_cost:
                self.best_cost = cost
                self.best_path = path[:]
            return

        for next in range(self.n):
            if not self.visited[next] and self.graph[current][next] > 0:
                new_cost = cost + self.graph[current][next]
                b = self.bound(next, count + 1, new_cost)

                if b < self.best_cost:
                    self.visited[next] = True
                    path.append(next)
                    self.solve(next, count + 1, new_cost, path)
                    path.pop()
                    self.visited[next] = False

    def tsp(self, graph: List[List[int]]) -> int:
        self.graph = graph
        self.n = len(graph)
        self.visited = [False] * self.n
        self.best_cost = sys.maxsize

        self.visited[0] = True
        self.solve(0, 1, 0, [0])

        return self.best_cost

```

```
# Example usage
bb = BranchAndBound()
weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
print(f"Max profit: {bb.knapsack(weights, values, 5)}") # 7

tsp_solver = TSPBranchAndBound()
tsp_graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]
print(f"Min TSP cost: {tsp_solver.tsp(tsp_graph)}") # 80
```

## 10. Code Explanation

### 0/1 Knapsack B&B:

- **Items sorted** by value/weight ratio (descending) for better bounds.
- **bound()**: Computes the upper bound using fractional knapsack on remaining items.
- **Node structure**: Contains **level**, **profit**, **weight**, **bound**.
- **BFS**: Use a queue to explore nodes level by level.
- **Branching**: For each node, create two children — take the item or skip it.
- **Pruning**: If **bound**  $\leq$  **maxProfit**, skip the node.

### TSP B&B:

- **bound()**: Sum of minimum edges from each remaining vertex (lower bound).
- **DFS**: Recursive backtracking with bound computation at each step.
- **Pruning**: If **bound**  $\geq$  **bestCost**, prune the branch.
- **Best update**: When a complete tour is found, update **bestCost**.

## 11. Complexity Analysis

| Aspect                       | Complexity                           |
|------------------------------|--------------------------------------|
| <b>Time (worst-case)</b>     | $O(2^n)$ — without effective pruning |
| <b>Time (average)</b>        | Much less — depends on bound quality |
| <b>Space (B&amp;B queue)</b> | $O(2^n)$ worst-case for BFS          |
| <b>Space (DFS)</b>           | $O(n)$ for recursion stack           |

## 12. Common Patterns

| Pattern             | Description                        | Example |
|---------------------|------------------------------------|---------|
| <b>0/1 Knapsack</b> | Maximize value within weight limit | Classic |

| Pattern               | Description                    | Example   |
|-----------------------|--------------------------------|-----------|
| <b>TSP</b>            | Minimize tour cost             | Classic   |
| <b>Job Scheduling</b> | Minimize makespan              | Variation |
| <b>Graph Coloring</b> | Minimize colors                | Variation |
| <b>Max Clique</b>     | Find largest complete subgraph | Variation |

### 13. Common Mistakes

- **Poor bound function** — too loose bound doesn't prune enough.
- **Not sorting** — for knapsack, sorting by ratio improves bounds.
- **Wrong bound type** — using upper bound for minimization (should be lower bound).
- **Integer overflow** — bound computation may overflow.
- **Infinite loop** — not handling visited nodes in TSP.
- **BFS vs DFS** — BFS can use too much memory for large problems.

### 14. Edge Cases

- **Empty input** — return 0 profit / 0 cost.
- **All items exceed capacity** — return 0 (knapsack).
- **Single item** — take if within capacity.
- **Complete graph** — TSP has  $(n-1)!$  possible tours.
- **Disconnected graph** — no valid TSP tour.

### 15. Variations

| Variation                     | What Changes                       | Importance |
|-------------------------------|------------------------------------|------------|
| <b>Best-First Search</b>      | Explore node with best bound first | High       |
| <b>LC (Least Cost) Search</b> | Use priority queue                 | High       |
| <b>FIFO (BFS)</b>             | Simple queue                       | Medium     |
| <b>LIFO (DFS)</b>             | Stack-based, less memory           | Medium     |
| <b>Parallel B&amp;B</b>       | Distributed search                 | Low        |

### 16. Related Algorithms

| Algorithm                  | Connection                                             |
|----------------------------|--------------------------------------------------------|
| <b>Backtracking</b>        | B&B = Backtracking + bounds                            |
| <b>Dynamic Programming</b> | DP for simpler optimization problems                   |
| <i>A Search*</i>           | Pathfinding with heuristic (similar to best-first B&B) |
| <b>Greedy</b>              | Greedy gives initial best solution for B&B             |
| <b>Integer Programming</b> | Branch and Cut, Branch and Price                       |

## 17. Practice Problems

### Easy

- **0/1 Knapsack** — GFG — Classic B&B problem
- **N-Queens** — LeetCode 51 — Can be solved with B&B (but backtracking is fine)

### Medium

- **TSP** — GFG — Traveling Salesman Problem
- **Job Sequencing** — GFG — Job scheduling with deadlines
- **Graph Coloring** — GFG — Minimize colors

### Hard

- **Maximum Clique** — GFG — Find largest clique
- **Integer Linear Programming** — CP — General optimization

## 18. Interview Explanation

"Branch and Bound is an optimization technique that systematically explores the search space while using bounds to prune unpromising branches. For a minimization problem, I compute a lower bound for each branch — the minimum possible cost of any solution in that branch. If the lower bound exceeds the current best solution, I prune the entire branch. The quality of the bound function is critical — a tighter bound prunes more. Compared to backtracking, B&B is designed for optimization (finding the best solution) rather than decision (finding any solution). Common applications include the Traveling Salesman Problem and 0/1 Knapsack."

## 19. Revision Notes

- **B&B = Backtracking + Bounding function**
- **Lower bound** for minimization; **Upper bound** for maximization
- **Prune when:**  $\text{bound} \geq \text{best}$  (minimization) or  $\text{bound} \leq \text{best}$  (maximization)
- **Better bound** → more pruning → faster algorithm
- **Node selection:** BFS (queue), DFS (stack), Best-First (priority queue)
- **Initial best** from greedy heuristic helps a lot

## 20. Final Cheat Sheet

| Aspect                 | Summary                                                                                           |
|------------------------|---------------------------------------------------------------------------------------------------|
| <b>When to use</b>     | Optimization problems, NP-hard with small $n$ , need optimal solution                             |
| <b>Main operations</b> | Branch (create subproblems) → Bound (compute limit) → Prune (if $\text{bound} \leq \text{best}$ ) |
| <b>Complexity</b>      | $O(2^n)$ worst-case, much less with good bounds                                                   |
| <b>Key code</b>        | <code>if (bound(node) &gt; best) { branch(node.left); branch(node.right); }</code>                |
| <b>Edge cases</b>      | Empty input, all items exceed capacity, single item, disconnected graph                           |



| Algorithm               | Type         | Time Complexity      | Space          | When to Use                            |
|-------------------------|--------------|----------------------|----------------|----------------------------------------|
| Basic Recursion         | Fundamental  | $O(n)$ to $O(2^n)$   | $O(n)$         | Problem has recursive structure        |
| Recursion Tree          | Analysis     | —                    | —              | Visualize recursive calls              |
| Backtracking Template   | Framework    | $O(b^d)$             | $O(d)$         | All solutions, constraint satisfaction |
| Subsets                 | Backtracking | $O(n \times 2^n)$    | $O(n)$         | Power set, all selections              |
| Permutations            | Backtracking | $O(n \times n!)$     | $O(n)$         | All arrangements                       |
| Combinations            | Backtracking | $O(k \times C(n,k))$ | $O(k)$         | Subsets of size $k$                    |
| N-Queens                | Backtracking | $O(N!)$              | $O(N)$         | Chessboard placement                   |
| Sudoku Solver           | Backtracking | $O(9^m)$             | $O(m)$         | Grid constraint satisfaction           |
| Rat in a Maze           | Backtracking | $O(4^{(N^2)})$       | $O(N^2)$       | Grid path finding                      |
| Generate Parentheses    | Backtracking | $O(4^n/\sqrt{n})$    | $O(n)$         | Balanced bracket sequences             |
| Word Search             | Backtracking | $O(mn \times 4^L)$   | $O(L)$         | Word in character grid                 |
| Palindrome Partitioning | Backtracking | $O(n \times 2^n)$    | $O(n)$         | Palindromic decompositions             |
| Combination Sum         | Backtracking | $O(N^{(T/\min)})$    | $O(T/\min)$    | Sum to target with reuse               |
| Hamiltonian Path        | Backtracking | $O(V!)$              | $O(V)$         | Visit all vertices once                |
| Meet-in-the-Middle      | Optimization | $O(2^{(n/2)})$       | $O(2^{(n/2)})$ | NP-hard with $n \leq 40$               |
| Branch and Bound        | Optimization | $O(2^n)^*$           | $O(2^n)^*$     | Optimization with bounds               |

\* With effective pruning, practical complexity is much lower.

Happy coding! 🚀 This guide covers 16 essential recursion and backtracking algorithms for placement and competitive programming.